Design Patterns in Java

#	Scenario / Pattern	Level	Target Role
1	Singleton — Central Audit Writer in a Multi-Threaded System	L3	Backend / Platform
2	Factory — Notification channels (Email/SMS/Push)	L1	Backend / Integrations
3	Builder — Customizable travel itinerary	L1	Backend / Product
4	Adapter — Unify cloud storage SDKs (S3/GCS/Azure)	L2	Backend / Cloud
5	Strategy — Shipping fee calculation	L1	E-commerce Backend
6	Decorator — Styling chat messages	L2	Frontend / Full-stack
7	Proxy — Product details lazy-load in e-commerce	L2	Backend / Perf
8	Strategy — Robot behavior in a simulation	L2	Backend / Games
9	Observer — Stock price alerts	L1	Backend / Realtime
10	Command — Smart TV remote with undo	L2	Embedded / App
11	Factory Method — Payment methods in a fintech app	L2	Fintech Backend
12	Strategy — Dynamic themes in a code editor	L1	Frontend
13	Observer — News portal subscribers per category	L1	Backend / PubSub
14	Strategy — Discount engines in an online store	L2	E-commerce Backend
15	Factory — Media file parsers in a player	L1	Media / Backend
16	Singleton — Configuration manager in a cloud service	L2	SRE / Platform
17	Factory — Food ordering app items	L1	Backend
18	Decorator — Online character skins	L1	Gaming
19	Observer — Weather app alerts	L1	Mobile / Backend
20	Command — Music player remote with undo	L2	App / UX
21	Strategy — Shipping methods (Standard/Express/Drone)	L1	Logistics

22	Abstract Factory — Gadget store product/variant families	L2	E-commerce
23	Factory — Notifications in chat app (Text/Image/Voice)	L1	Messaging
24	Command — Job scheduler (sync & async)	L3	Platform
25	Builder — SaaS subscription plans	L1	SaaS
26	Factory Method — Online education bookings	L2	EdTech
27	Decorator — Ride-sharing features	L2	Mobility
28	Observer — Cryptocurrency price alerts	L1	Fintech
29	Command — Restaurant ordering system with undo	L2	POS
30	Strategy — Adaptive learning difficulty	L2	EdTech
31	Factory — Shipment order types in logistics	L1	Logistics
32	Builder — Video-conferencing plan configurator	L2	SaaS
33	Strategy — Route selection in maps	L3	Maps
34	Factory Method — Document exporters workflow	L2	Docs
35	Strategy — Incident alert channels	L2	SRE
36	Factory Method — Drone delivery fleet depots	L3	Logistics
37	Factory Method — Data ingestion processors	L3	Data Platform
38	Decorator — Email composer add-ons	L2	Productivity
39	Strategy — MOBA hero roles	L1	Gaming
40	Command — Text editor actions with undo	L2	Productivity
41	Singleton — Thread-safe cache manager	L3	Platform
42	Factory Method — Ride-hailing booking types	L2	Mobility
43	Abstract Factory — Cross-platform UI toolkit	L3	UI
44	Builder — Flexible resume construction	L1	App
45	Decorator — Dynamic pizza toppings	L1	FoodTech
46	Adapter — Unify payment gateways (PayPal/Stripe/Razorpay)	L2	Fintech

47	Proxy — Lazy-loading large images in a gallery	L1	Frontend
48	Strategy — Pluggable discount logic in travel booking	L2	Travel
49	Observer — Stock portfolio subscriptions	L1	Fin/Wealth
50	Command — Drawing actions with undo	L2	Graphics
51	Prototype — Rich document templates	L2	Productivity
52	Composite — Nested dashboard widgets	L2	UI
53	Bridge — Media app controls vs devices	L3	Media
54	Flyweight — Memory-efficient glyph rendering	L3	Graphics
55	Facade — One-click export across subsystems	L1	App
56	Chain of Responsibility — API security pipeline	L3	Platform
57	Interpreter — Mini rule language for discounts/flags	L4	Arch / DSL
58	Iterator — Lazy PagedCollection over an API	L2	SDK
59	Mediator — Coordinating a busy chat room	L2	Realtime
60	Memento — Safe undo/redo for a diagram editor	L2	Graphics
61	State — Payment lifecycle (Init→Auth→Capture→Refund/Fail)	L3	Fintech
62	Template Method — Pluggable import sources, fixed algorithm	L2	Data
63	Visitor — Extensible cart ops (GST, discounts, shipping)	L3	E-commerce
64	Producer–Consumer — Back-pressure thumbnail pipeline	L3	Media
65	Thread Pool / Executor tuning — 1,000s short tasks	L3	Platform
66	Read–Write Lock — Hot catalog (many reads, few writes)	L2	Backend
67	CompletableFuture composition — fan-out to 3 services	L3	Backend / Reactive
68	DI vs Service Locator — refactor legacy static registry	L2	Arch
69	Repository / DAO — Postgres + Redis behind clean API	L2	Data / Backend

70	Template Method vs Strategy in Spring — fixed payment skeleton + pluggable risk checks	L3	Spring Backend
71	Circuit Breaker / Retry / Timeout — flaky Price service	L3	SRE / Backend
72	Bulkhead & Rate Limiter — isolating a noisy tenant	L3	SRE / Multi-tenant
73	Idempotency — duplicate payment callbacks	L3	Fintech
74	Outbox & Saga — Order→Payment→Inventory across services	L4	Distributed Systems
75	Publisher–Subscriber — audit events to many downstreams	L4	Platform / Data
76	Object Pool — expensive but short-lived parsers	L3	Perf / JVM
77	Flyweight Pitfalls — keep extrinsic state out of equality	L3	Graphics / Perf
78	Plugin architecture — drop-in pricing/risk via ServiceLoader	L4	Platform / Extensibility

Scenario 1: Singleton — Central Audit Writer in a Multi-Threaded System

In a microservices setup, every service instance must log events into the same audit file. If multiple writer objects are created in the JVM, logs can get interleaved or duplicated. We need exactly one writer instance shared across all threads.

Answer:

I'll design a Singleton so only one AuditWriter ever exists in the JVM. The key challenges here are:

- Preventing multiple instantiations under concurrency.
- Ensuring lazy initialization (created only when first accessed).
- Keeping the code simple and avoiding performance overhead.

For this, the *Initialization-on-Demand Holder Idiom* is a clean solution. It leverages class loading guarantees: the inner static class is only loaded when getInstance() is called, and class initialization in Java is thread-safe.

Code:

```
public final class AuditWriter {
    // private constructor prevents external instantiation
    private AuditWriter() {
        // Initialize file handles, buffered writers, formatters, etc.
    }

    // static inner class holds the instance
    private static class Holder {
        private static final AuditWriter INSTANCE = new AuditWriter();
    }

    // global access point
    public static AuditWriter getInstance() {
        return Holder.INSTANCE;
    }

    // example append method
    public void append(String event) {
        // In real code, use BufferedWriter or NIO to append to file safely
        System.out.println("[AUDIT] " + event);
    }
}

// Usage
class Demo {
    public static void main(String[] args) {
```

```
        AuditWriter w1 = AuditWriter.getInstance();
        AuditWriter w2 = AuditWriter.getInstance();

        w1.append("service=start");
        System.out.println(w1 == w2); // true
    }
}
```

Reasoning to interviewer:

- The constructor is private, so no one can create objects directly.
- The Holder class isn't loaded until getInstance() is called, ensuring lazy initialization.
- Class loading in Java is guaranteed to be thread-safe, so even if 100 threads call getInstance() at the same time, only one instance is created.
- This avoids explicit synchronization, unlike double-checked locking which is harder to get right before Java 5.
- Alternative valid approaches:
  • Enum-based Singleton (simple, serializable by default).
  • Double-checked locking with volatile (slightly more verbose).

Future scope:

- Make append() truly thread-safe with ReentrantLock or a concurrent queue feeding a single writer thread.
- Use asynchronous logging frameworks (like Logback/Log4j2) if audit throughput is high.
- Add rotation policies (daily audit files) and persistence to cloud storage for compliance.
- Expose monitoring hooks (e.g., number of events written) via JMX.

Scenario 2: Factory — notification channels (email / sms / push)

A customer engagement service must send alerts using different channels like Email, SMS, and Push. Each channel uses a different SDK, so we need one central creation point to decide which implementation to return.

Answer:

I'll design a Notifier interface with one send method. Concrete classes implement channel-specific logic. A factory class will centralize the creation of the right notifier. Instead of scattering new calls across the code, clients just ask the factory. This makes it easy to add a new channel later (e.g., WhatsApp).

Code:

```
import java.util.EnumMap;
import java.util.Map;
import java.util.Objects;
import java.util.function.Supplier;

interface Notifier {
    void send(String to, String message);
}

class EmailNotifier implements Notifier {
    public void send(String to, String message) {
        System.out.println("Email to " + to + ": " + message);
    }
}

class SmsNotifier implements Notifier {
    public void send(String to, String message) {
        System.out.println("SMS to " + to + ": " + message);
    }
}

class PushNotifier implements Notifier {
    public void send(String to, String message) {
        System.out.println("Push to " + to + ": " + message);
    }
}

enum Channel { EMAIL, SMS, PUSH }

class NotifierFactory {
    private static final Map<Channel, Supplier<Notifier>> REGISTRY = new
EnumMap<>(Channel.class);
    static {
        REGISTRY.put(Channel.EMAIL, EmailNotifier::new);
```

```
        REGISTRY.put(Channel.SMS, SmsNotifier::new);
        REGISTRY.put(Channel.PUSH, PushNotifier::new);
    }

    public static Notifier of(Channel channel) {
        Supplier<Notifier> sup = REGISTRY.get(Objects.requireNonNull(channel));
        if (sup == null) throw new IllegalArgumentException("Unknown channel: " +
channel);
        return sup.get();
    }
}

// Usage
class DemoFactory {
    public static void main(String[] args) {
        Notifier n = NotifierFactory.of(Channel.SMS);
        n.send("+91-99999", "Order shipped!");
    }
}
```

Reasoning to interviewer:

- Factory hides the creation details, so the client code only depends on the interface, not implementations.
- Enum + registry avoids long switch statements and keeps the design open for extension.
- If the notifier objects are heavy to initialize, we can change the supplier to return cached singletons.
- Works smoothly with dependency injection frameworks as well.

Future scope:

- Add new channels like WhatsApp or Slack by only adding new classes and updating the registry.
- Implement retry policies and error handling for failed sends.
- Allow runtime configuration to enable or disable certain channels.
- Support composite notifiers that can fan out to multiple channels for high-priority alerts.

Scenario 3: Builder — customizable travel itinerary

A travel app must build an itinerary with optional parts like hotel, meals, pickup, and seat preference. Not every user chooses all options, so we need an immutable object with a flexible construction style.

Answer:

I'll use the Builder pattern to separate required and optional fields. Origin and destination are mandatory, so they go in the builder's constructor. Optional methods like withHotel() or seat() add flexibility. The final object will be immutable, safe to share, and easy to extend.

Code:

```
import java.util.Objects;

public final class TripPlan {
    private final String origin;
    private final String destination;
    private final boolean hotelIncluded;
    private final boolean airportPickup;
    private final String seatPref;
    private final boolean insurance;

    private TripPlan(Builder b) {
        this.origin = b.origin;
        this.destination = b.destination;
        this.hotelIncluded = b.hotelIncluded;
        this.airportPickup = b.airportPickup;
        this.seatPref = b.seatPref;
        this.insurance = b.insurance;
    }

    public static class Builder {
        private final String origin;
        private final String destination;
        private boolean hotelIncluded;
        private boolean airportPickup;
        private String seatPref;
        private boolean insurance;

        public Builder(String origin, String destination) {
            this.origin = validate(origin, "origin");
            this.destination = validate(destination, "destination");
            if (this.origin.equalsIgnoreCase(this.destination)) {
                throw new IllegalArgumentException("Origin and destination must
differ");
            }
        }
```

```
        public Builder withHotel() { this.hotelIncluded = true; return this; }
        public Builder withAirportPickup() { this.airportPickup = true; return this; }
        public Builder seat(String pref) { this.seatPref = validate(pref, "seat"); return
this; }
        public Builder withInsurance() { this.insurance = true; return this; }

        public TripPlan build() { return new TripPlan(this); }

        private static String validate(String s, String field) {
            Objects.requireNonNull(s, field + " required");
            String t = s.trim();
            if (t.isEmpty()) throw new IllegalArgumentException(field + " required");
            return t;
        }
    }

    @Override
    public String toString() {
        return "TripPlan{" +
                "origin='" + origin + '\'' +
                ", destination='" + destination + '\'' +
                ", hotel=" + hotelIncluded +
                ", pickup=" + airportPickup +
                ", seat='" + seatPref + '\'' +
                ", insurance=" + insurance +
                '}';
    }
}

// Usage
class DemoBuilder {
    public static void main(String[] args) {
        TripPlan plan = new TripPlan.Builder("Pune", "Bengaluru")
                .withHotel()
                .withAirportPickup()
                .seat("WINDOW")
                .withInsurance()
                .build();
        System.out.println(plan);
    }
}
```

Reasoning to interviewer:

- Builder avoids telescoping constructors and makes the code expressive.
- Mandatory fields (origin, destination) are enforced in the builder's constructor.
- The final object is immutable (all fields final, no setters).

- Fluent API style makes the construction readable and reduces mistakes.

Future scope:

- Replace seat preference with an enum for stricter type safety.
- Add pricing logic inside the builder to calculate total cost automatically.
- Support deep copies with a toBuilder() method for editing an existing plan.
- Introduce validation rules like mandatory insurance for international trips.

Scenario 4: Adapter — unify different cloud storage SDKs

Different providers (S3, GCS, Azure) expose different upload APIs. I want my application to talk to one interface (StorageClient) and plug in the right adapter behind it. That way, checkout, media, and reporting modules don't care which vendor we use.

Answer:

I'll define a small internal target interface StorageClient with upload(bucket, key, data). For each provider, I'll create an adapter that wraps the vendor SDK and translates our method into the provider's call (object/param names, exceptions). Selection of the concrete adapter happens at the edge (config/DI). Adding Azure or MinIO later means adding one class; the rest of the code stays unchanged.

Code:

```
import java.util.Objects;

// Target
interface StorageClient {
    void upload(String bucket, String key, byte[] data);
}

// Adaptees (simulate vendor SDKs)
class S3Lib { void put(String b, String k, byte[] d) { System.out.println("S3 put " + b
+ "/" + k); } }
class GcsLib { void write(String b, String k, byte[] d) { System.out.println("GCS
write " + b + "/" + k); } }
class AzureBlobLib { void upload(String container, String blob, byte[] d) {
System.out.println("Azure upload " + container + "/" + blob); } }

// Adapters
class S3ClientAdapter implements StorageClient {
    private final S3Lib s3 = new S3Lib();
    @Override public void upload(String bucket, String key, byte[] data) {
        Objects.requireNonNull(bucket); Objects.requireNonNull(key);
Objects.requireNonNull(data);
        s3.put(bucket, key, data);
    }
}

class GcsClientAdapter implements StorageClient {
    private final GcsLib gcs = new GcsLib();
    @Override public void upload(String bucket, String key, byte[] data) {
        Objects.requireNonNull(bucket); Objects.requireNonNull(key);
Objects.requireNonNull(data);
        gcs.write(bucket, key, data);
    }
}
```

```
class AzureBlobAdapter implements StorageClient {
    private final AzureBlobLib azure = new AzureBlobLib();
    @Override public void upload(String bucket, String key, byte[] data) {
        Objects.requireNonNull(bucket); Objects.requireNonNull(key);
Objects.requireNonNull(data);
        azure.upload(bucket, key, data);
    }
}

// Simple selector (could be wired via DI/config)
enum Provider { S3, GCS, AZURE }
class StorageClientFactory {
    static StorageClient forProvider(Provider p) {
        return switch (p) {
            case S3 -> new S3ClientAdapter();
            case GCS -> new GcsClientAdapter();
            case AZURE -> new AzureBlobAdapter();
        };
    }
}

// Usage
class DemoAdapter {
    public static void main(String[] args) {
        StorageClient client = StorageClientFactory.forProvider(Provider.S3);
        client.upload("media", "logo.png", new byte[]{1,2,3});

        client = StorageClientFactory.forProvider(Provider.GCS);
        client.upload("media", "cover.jpg", new byte[]{4,5,6});
    }
}
```

Reasoning to interviewer:

- The app speaks one language (StorageClient), while adapters perform the translation to S3/GCS/Azure APIs. That isolates third-party drift and SDK upgrades.
- Construction is centralized (factory/DI), so switching providers is a config change, not a code rewrite.
- Validation lives in adapters; we can also translate vendor exceptions to our own StorageException, keeping callers vendor-agnostic.
- This design lets us mock StorageClient in tests without pulling real SDKs.

Future scope:

- Add streaming/multipart uploads (S3 multipart, GCS resumable) behind the same interface for large files.
- Implement retries and idempotency keys (e.g., object metadata) to handle transient failures.
- Support pre-signed URL generation as another method on StorageClient for client-side uploads.
- Emit metrics (latency, bytes, error codes) per provider for SRE visibility; add circuit breaking if a provider degrades.

Scenario 5: Strategy — shipping fee calculation

In an e-commerce checkout, shipping can be weight-based, flat promo, or express (base + distance). I don't want if/else jungles in checkout; I want to plug the right algorithm based on cart/user and compute.

Answer:

I'll define a ShippingStrategy with fee(weightKg, distanceKm). Each algorithm becomes its own class. Checkout depends on the interface, so I can pick a strategy at runtime (promo flag, user preference, service level). If rules change, I add a new strategy without touching Checkout.

Code:

```
interface ShippingStrategy {
    double fee(double weightKg, int distanceKm);
}

class WeightBasedShipping implements ShippingStrategy {
    @Override public double fee(double weightKg, int distanceKm) {
        // base + per-kg
        return 40 + (Math.max(0, weightKg) * 20);
    }
}

class FlatRateShipping implements ShippingStrategy {
    @Override public double fee(double weightKg, int distanceKm) {
        return 99; // promo
    }
}

class ExpressShipping implements ShippingStrategy {
    @Override public double fee(double weightKg, int distanceKm) {
        // faster, distance-driven
        return 150 + Math.max(0, distanceKm) * 5;
    }
}

class Checkout {
    private ShippingStrategy strategy;
    public Checkout(ShippingStrategy strategy) { this.strategy = strategy; }
    public void setStrategy(ShippingStrategy s) { this.strategy = s; }

    public double total(double itemsTotal, double weightKg, int distanceKm) {
        return itemsTotal + strategy.fee(weightKg, distanceKm);
    }
}

// Optional: a selector to pick strategy via business rules
```

```
enum ShippingMode { WEIGHT, FLAT, EXPRESS }
class ShippingSelector {
    static ShippingStrategy choose(ShippingMode mode, boolean promoActive) {
        if (promoActive) return new FlatRateShipping();
        return switch (mode) {
            case WEIGHT -> new WeightBasedShipping();
            case FLAT   -> new FlatRateShipping();
            case EXPRESS-> new ExpressShipping();
        };
    }
}

// Usage
class DemoStrategy {
    public static void main(String[] args) {
        Checkout c = new
Checkout(ShippingSelector.choose(ShippingMode.WEIGHT, false));
        System.out.println(c.total(1200, 3.5, 12)); // weight-based

        c.setStrategy(ShippingSelector.choose(ShippingMode.FLAT, true)); // promo
        System.out.println(c.total(1200, 3.5, 12));

        c.setStrategy(new ExpressShipping()); // user picks express
        System.out.println(c.total(1200, 3.5, 750));
    }
}
```

Reasoning to interviewer:

- Strategy removes branching from Checkout and localizes pricing logic per algorithm, which makes audits and experiments easier.
- Runtime selection is simple: choose by user's choice, SLA, or flags from a feature-config service.
- This keeps us open for extension—"Eco" or "Same-Day" is a new class, not a cascade of ifs.
- Testing is isolated: each strategy can be unit-tested with boundary cases (0 kg, long distances).

Future scope:

- Add surge pricing or slab rates (e.g., first 5 km cheaper, beyond that higher) as new strategies.
- Inject region-specific rates from configuration, not hard-coded numbers.

- Compose strategies (e.g., base rule + holiday surcharge) using decorators for complex pricing.
- Expose an explainability method (e.g., breakdown() returning components) for invoice transparency.

Scenario 6: Decorator — styling chat messages

Users can stack styles like bold, italic, strikethrough in any order. We need a base message and pluggable wrappers that add formatting without modifying the base class.

Answer:

I'll define a simple Message interface with render(). PlainMessage holds the raw text. Each style (bold, italic, strike) is a decorator that wraps another Message and augments render(). Because decorators share the same interface, they can be composed freely (new Italic(new Bold(new Plain(...)))). Adding a new style is just a new decorator class—no changes to existing ones.

Code:

```
interface Message {
    String render();
}

final class PlainMessage implements Message {
    private final String content;
    public PlainMessage(String content) { this.content = content; }
    @Override public String render() { return content; }
}

abstract class MessageDecorator implements Message {
    protected final Message inner;
    protected MessageDecorator(Message inner) { this.inner = inner; }
    @Override public String render() { return inner.render(); }
}

final class BoldMessage extends MessageDecorator {
    public BoldMessage(Message inner) { super(inner); }
    @Override public String render() { return "**" + super.render() + "**"; } //
markdown-like
}

final class ItalicMessage extends MessageDecorator {
    public ItalicMessage(Message inner) { super(inner); }
    @Override public String render() { return "_" + super.render() + "_"; }
}

final class StrikeMessage extends MessageDecorator {
    public StrikeMessage(Message inner) { super(inner); }
    @Override public String render() { return "~~" + super.render() + "~~"; }
}

// Usage
class DemoDecorator {
```

```
    public static void main(String[] args) {
        Message m1 = new BoldMessage(new PlainMessage("Hello"));
        Message m2 = new ItalicMessage(new BoldMessage(new PlainMessage("Hi
Rajat")));
        Message m3 = new StrikeMessage(new ItalicMessage(new
BoldMessage(new PlainMessage("Outdated text"))));
        System.out.println(m1.render()); // **Hello**
        System.out.println(m2.render()); // _**Hi Rajat**_
        System.out.println(m3.render()); // ~~_**Outdated text**_~~
    }
}
```

Reasoning to interviewer:

- Decorator keeps the base type small and stable; behavior is added via composition, not inheritance chains.
- All decorators implement Message, so they are interchangeable and stackable.
- Order matters (bold → italic vs italic → bold), which is naturally expressed by nesting.
- This is easy to test: each decorator has a tiny, focused responsibility.

Future scope:

- Add renderers for HTML or ANSI (e.g., … or terminal codes) via alternate decorators.
- Introduce a SanitizedMessage decorator to escape unsafe content before styling.
- Add a TruncatedMessage decorator for max-length previews in the chat list.
- Cache results in a MemoizedMessage if the same combination is rendered repeatedly.

Scenario 7: Proxy — product details in e-commerce

On product listing pages we want instant load (name, price), but heavy data (reviews, seller history, high-res images) should load only when the user opens details.

Answer:

I'll expose a Product interface with two views: showBasic() and showFullDetails(). ProductProxy holds only cheap fields and defers creation of RealProduct until full details are requested. The first time showFullDetails() is called, the proxy instantiates RealProduct (which does the expensive work), then delegates. Subsequent calls reuse the loaded instance.

Code:

```
interface Product {
    void showBasic();
    void showFullDetails();
}

final class RealProduct implements Product {
    private final String name;
    private final String price;
    private final String reviews; // heavy
    private final String sellerHistory; // heavy

    public RealProduct(String name, String price) {
        this.name = name;
        this.price = price;
        // Simulate expensive loads
        this.reviews = loadReviews();
        this.sellerHistory = loadSellerHistory();
    }

    private String loadReviews() {
        System.out.println("Fetching reviews from DB...");
        return "Rated 4.8/5 (12,341 reviews)";
    }

    private String loadSellerHistory() {
        System.out.println("Fetching seller history...");
        return "Top Seller • 98% positive in last 12 months";
    }

    @Override public void showBasic() {
        System.out.println(name + " - " + price);
    }

    @Override public void showFullDetails() {
```

```
        System.out.println(name + " - " + price + " | " + reviews + " | " + sellerHistory);
    }
}

final class ProductProxy implements Product {
    private final String name;
    private final String price;
    private RealProduct real; // lazily initialized

    public ProductProxy(String name, String price) {
        this.name = name;
        this.price = price;
    }

    @Override public void showBasic() {
        // Fast: just fields already in memory
        System.out.println(name + " - " + price);
    }

    @Override public void showFullDetails() {
        if (real == null) {
            // Lazy load on first demand
            real = new RealProduct(name, price);
        }
        real.showFullDetails();
    }
}

// Usage
class DemoProxy {
    public static void main(String[] args) {
        Product p = new ProductProxy("Laptop", "₹65,000");
        p.showBasic();        // fast
        p.showFullDetails();  // triggers heavy fetches
        p.showFullDetails();  // reuses cached RealProduct (no re-fetch)
    }
}
```

Reasoning to interviewer:

- This is a virtual proxy: it stands in for a heavyweight object and creates it only when needed.
- The listing page can render thousands of proxies quickly (name/price), while details are fetched on demand.
- The proxy also gives a control point for cross-cutting concerns—timing metrics, caching, or permissions—without touching RealProduct.

- Memory and latency are predictable: we only pay the cost for items the user actually inspects.

Future scope:

- Add eviction or soft references if many product details are opened to avoid memory bloat.
- Prefetch top N popular products in the background during idle time for perceived speed.
- Wrap network/DB calls in retries and circuit breakers inside RealProduct or behind a repository.
- Split heavy fields behind individual proxies (e.g., ReviewsProxy, ImagesProxy) for even finer-grained loading.

Scenario 8: Strategy — robot behavior in a simulation

A robot needs to switch behavior at runtime (explore, attack, repair) based on environment signals (enemy spotted, low health, unknown terrain). We want clean swapping without if/else chains all over the code.

Answer:

I'll define a Mode strategy interface with execute(context). Each behavior (Explore/Attack/Repair) is a concrete strategy. The Robot holds a reference to the current Mode and delegates behavior to it. When conditions change (e.g., health < 30%), we swap the strategy via setMode(...). This keeps the robot logic simple and makes adding a new mode like "Defend" trivial.

Code:

```
import java.util.Objects;

// Strategy
interface Mode {
    void execute(RobotContext ctx);
}

// Concrete strategies
class ExploreMode implements Mode {
    @Override public void execute(RobotContext ctx) {
        System.out.println("Exploring terrain: scanning area " + ctx.sectorId());
        ctx.consumeBattery(2);
    }
}

class AttackMode implements Mode {
    @Override public void execute(RobotContext ctx) {
        if (ctx.missiles() > 0) {
            System.out.println("Engaging target with missiles!");
            ctx.fireMissile();
        } else {
            System.out.println("No missiles left, switching to Explore.");
            ctx.robot().setMode(new ExploreMode());
        }
        ctx.consumeBattery(5);
    }
}

class RepairMode implements Mode {
    @Override public void execute(RobotContext ctx) {
        System.out.println("Repairing systems...");
        ctx.recoverHealth(10);
        ctx.consumeBattery(3);
    }
```

```
}

// Context data the strategies may need
final class RobotContext {
    private final Robot robot;
    private int battery;      // %
    private int health;       // %
    private int missiles;
    private final String sectorId;

    RobotContext(Robot robot, int battery, int health, int missiles, String sectorId) {
        this.robot = robot; this.battery = battery; this.health = health; this.missiles =
missiles; this.sectorId = sectorId;
    }

    public Robot robot() { return robot; }
    public String sectorId() { return sectorId; }
    public int battery() { return battery; }
    public int health() { return health; }
    public int missiles() { return missiles; }

    public void consumeBattery(int delta) { battery = Math.max(0, battery - delta); }
    public void recoverHealth(int delta) { health = Math.min(100, health + delta); }
    public void fireMissile() { if (missiles > 0) missiles--; }
}

// Robot holding the current strategy
class Robot {
    private Mode mode;
    private final RobotContext ctx;

    public Robot(Mode initial, RobotContext ctx) {
        this.mode = Objects.requireNonNull(initial);
        this.ctx = Objects.requireNonNull(ctx);
    }

    public void setMode(Mode m) { this.mode = Objects.requireNonNull(m); }
    public void tick() { mode.execute(ctx); } // one simulation step

    // Example trigger based on state (could be called by simulation engine)
    public void evaluateAndSwitch() {
        if (ctx.health() < 30) setMode(new RepairMode());
        else if (ctx.missiles() > 0) setMode(new AttackMode());
        else setMode(new ExploreMode());
    }
}

// Usage
class DemoStrategyRobot {
    public static void main(String[] args) {
```

```
        Robot dummy = new Robot(new ExploreMode(), new RobotContext(null, 100,
50, 1, "A-12"));
        RobotContext ctx = new RobotContext(dummy, 100, 50, 1, "A-12");
        Robot bot = new Robot(new ExploreMode(), ctx);

        bot.tick();             // explore
        bot.setMode(new AttackMode());
        bot.tick();             // attack
        bot.evaluateAndSwitch();    // switch based on state
        bot.tick();             // next mode executes
    }
}
```

Reasoning to interviewer:

- We keep behavior isolated per mode, so changes in one don't ripple through others.
- The Robot depends on the Mode interface, enabling hot-swapping without conditionals.
- Context is passed in (RobotContext) so strategies can read/write relevant state (battery, missiles).
- Testability improves: each mode can be unit-tested independently with a fake context.

Future scope:

- Add time-sliced behaviors (e.g., explore for N ticks then reevaluate).
- Introduce composite strategies (patrol pattern = explore segments in sequence).
- Externalize mode selection via a rules engine or behavior tree for complex AI.
- Telemetry hooks inside strategies to record decisions for replay/debugging.

Scenario 9: Observer — stock price alerts

Investors subscribe to stocks and want instant alerts when prices change. Subscriptions change frequently (subscribe/unsubscribe), and multiple investors can watch the same stock.

Answer:

I'll model Stock as a subject with a list of observers (Investor). When setPrice(...) is called, the stock notifies its current observers. Investors can be added/removed at any time. This decouples the price producer from consumers—new investor types don't affect the Stock class.

Code:

```
import java.util.ArrayList;
import java.util.List;
import java.util.function.Predicate;

// Observer
interface Investor {
    void notify(String stock, double price);
}

// Concrete observer
class Trader implements Investor {
    private final String name;
    private final Predicate<Double> filter; // optional: only alert if condition matches

    public Trader(String name) { this(name, p -> true); }
    public Trader(String name, Predicate<Double> filter) {
        this.name = name; this.filter = filter;
    }

    @Override public void notify(String stock, double price) {
        if (filter.test(price)) {
            System.out.println(name + " alerted: " + stock + " at " + price);
        }
    }
}

// Subject
interface Stock {
    void addInvestor(Investor i);
    void removeInvestor(Investor i);
    void notifyAllInvestors();
    void setPrice(double price);
    double getPrice();
}
```

```
class StockTicker implements Stock {
    private final String symbol;
    private double price;
    private final List<Investor> investors = new ArrayList<>();

    public StockTicker(String symbol) { this.symbol = symbol; }

    @Override public void setPrice(double price) {
        this.price = price;
        notifyAllInvestors();
    }

    @Override public double getPrice() { return price; }

    @Override public void addInvestor(Investor i) { investors.add(i); }
    @Override public void removeInvestor(Investor i) { investors.remove(i); }

    @Override public void notifyAllInvestors() {
        // iterate over a snapshot to avoid ConcurrentModification if observers mutate
on notify
        List<Investor> snapshot = List.copyOf(investors);
        for (Investor i : snapshot) i.notify(symbol, price);
    }
}

// Usage
class DemoObserver {
    public static void main(String[] args) {
        StockTicker aapl = new StockTicker("AAPL");
        Investor ravi = new Trader("Ravi");
        Investor sneha = new Trader("Sneha", p -> p > 200); // only if price > 200

        aapl.addInvestor(ravi);
        aapl.addInvestor(sneha);

        aapl.setPrice(195.5); // notifies Ravi (Sneha filter skips)
        aapl.setPrice(205.0); // notifies both
        aapl.removeInvestor(ravi);
        aapl.setPrice(210.0); // notifies Sneha only
    }
}
```

Reasoning to interviewer:

- Subject/observer separation keeps StockTicker ignorant of who is listening and why.
- Using a snapshot during notification prevents concurrent modification if observers unsubscribe in response to an alert.

- Optional filters in Trader show how subscribers can encode their own thresholds without complicating StockTicker.
- This scales to many observer types (mobile push, email, webhooks) with zero changes to the subject.

Future scope:

- Make notifications asynchronous with an event bus (e.g., queue per stock) to avoid slow observers blocking price updates.
- Add backpressure or batching if price ticks are very frequent.
- Persist subscriber preferences (thresholds, channels) and honor do-not-disturb windows.
- Support topic wildcards or portfolios (subscribe to “all FAANG stocks above X”).

Scenario 10: Command — smart TV remote

A single remote should control multiple devices (TV, Soundbar, Gaming Console) and support undo. Each button press should encapsulate an action so it can be invoked and reversed later.

Answer:

I'll define a Command interface with execute() and undo(). Each device operation (TV On, TV Off, Soundbar Volume Up) is wrapped in a concrete command. The Remote doesn't know about devices—it only stores and triggers commands. For undo, I'll keep a stack of executed commands and call their undo() when required.

Code:

```
import java.util.Stack;

// Command interface
interface Command {
    void execute();
    void undo();
}

// Receiver 1
class TV {
    public void powerOn() { System.out.println("TV is ON"); }
    public void powerOff() { System.out.println("TV is OFF"); }
}

// Receiver 2
class Soundbar {
    public void volumeUp() { System.out.println("Soundbar volume increased"); }
    public void volumeDown() { System.out.println("Soundbar volume decreased"); }
}

// Receiver 3
class GamingConsole {
    public void start() { System.out.println("Console started"); }
    public void stop() { System.out.println("Console stopped"); }
}

// Concrete commands
class TVOnCommand implements Command {
    private final TV tv;
    public TVOnCommand(TV tv) { this.tv = tv; }
    public void execute() { tv.powerOn(); }
    public void undo() { tv.powerOff(); }
}
```

```
class TVOffCommand implements Command {
    private final TV tv;
    public TVOffCommand(TV tv) { this.tv = tv; }
    public void execute() { tv.powerOff(); }
    public void undo() { tv.powerOn(); }
}

class VolumeUpCommand implements Command {
    private final Soundbar sb;
    public VolumeUpCommand(Soundbar sb) { this.sb = sb; }
    public void execute() { sb.volumeUp(); }
    public void undo() { sb.volumeDown(); }
}

class ConsoleStartCommand implements Command {
    private final GamingConsole gc;
    public ConsoleStartCommand(GamingConsole gc) { this.gc = gc; }
    public void execute() { gc.start(); }
    public void undo() { gc.stop(); }
}

// Invoker
class Remote {
    private final Stack<Command> history = new Stack<>();
    public void press(Command cmd) {
        cmd.execute();
        history.push(cmd);
    }
    public void undoLast() {
        if (!history.isEmpty()) {
            history.pop().undo();
        }
    }
}

// Usage
class DemoCommand {
    public static void main(String[] args) {
        TV tv = new TV();
        Soundbar sb = new Soundbar();
        GamingConsole gc = new GamingConsole();
        Remote remote = new Remote();

        remote.press(new TVOnCommand(tv));
        remote.press(new VolumeUpCommand(sb));
        remote.press(new ConsoleStartCommand(gc));

        // Undo last (console stop)
        remote.undoLast();
        // Undo again (soundbar volume down)
```

```
        remote.undoLast();
    }
}
```

Reasoning to interviewer:

- Each command encapsulates a request, so the remote is decoupled from actual devices.
- Undo works naturally by calling the opposite action stored in the command.
- Stack ensures multiple undo operations are possible, maintaining command history.
- Adding new devices just means writing new command classes, not touching the remote code.

Future scope:

- Add redo functionality with a second stack.
- Macro commands (sequence of actions like “Movie Mode” → TV on, console off, soundbar volume set).
- Schedule commands for delayed execution (timer-based).
- Persist history so the system can recover device state after restart.

Scenario 11: Factory Method — payment methods in a fintech app

A wallet app supports UPI, Card, and NetBanking. Each has different processing flows. We want a clean way to add new payment options without rewriting client code.

Answer:

I'll define a Payment interface with pay(). Each method (UPI, Card, NetBanking) implements it. Instead of clients instantiating directly, we introduce a PaymentFactory abstract class with a createPayment(). Each concrete factory returns the correct type. The client code depends only on the factory abstraction, so adding a new payment type means creating a new factory—no changes elsewhere.

Code:

```
// Product interface
interface Payment {
    void pay(double amount);
}

// Concrete products
class UpiPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Paid " + amount + " via UPI");
    }
}

class CardPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Paid " + amount + " using Credit Card");
    }
}

class NetBankingPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Paid " + amount + " via NetBanking");
    }
}

// Factory Method
abstract class PaymentFactory {
    public abstract Payment createPayment();
}

class UpiPaymentFactory extends PaymentFactory {
    public Payment createPayment() { return new UpiPayment(); }
}

class CardPaymentFactory extends PaymentFactory {
```

```
    public Payment createPayment() { return new CardPayment(); }
}

class NetBankingPaymentFactory extends PaymentFactory {
    public Payment createPayment() { return new NetBankingPayment(); }
}

// Usage
class DemoFactoryMethod {
    public static void main(String[] args) {
        PaymentFactory factory = new UpiPaymentFactory();
        Payment p1 = factory.createPayment();
        p1.pay(500);

        factory = new CardPaymentFactory();
        Payment p2 = factory.createPayment();
        p2.pay(1500);
    }
}
```

Reasoning to interviewer:

- Factory Method encapsulates object creation, so clients don't new directly.
- Open for extension: adding WalletPayment only requires WalletPayment and WalletPaymentFactory.
- Client code doesn't change—it always talks to PaymentFactory and Payment.
- Each factory can encapsulate setup logic (e.g., connect to UPI gateway, load card rules).

Future scope:

- Replace concrete factories with a registry or config-driven lookup (avoid new).
- Integrate with dependency injection so factories are wired at runtime.
- Add cross-cutting logic in factories (logging, fraud checks).
- Support multiple providers per method (e.g., multiple UPI gateways) via specialized factories.

Scenario 12: Strategy — dynamic themes in a code editor

A desktop editor should switch themes (Light, Dark, Solarized) at runtime without disturbing the rest of the app. The UI should simply “ask the current theme” how to paint.

Answer:

I’ll treat a theme as a strategy: one interface that knows how to apply colors/fonts to the editor UI. The Editor holds a Theme reference and delegates to it during render or when the user switches themes. To keep this realistic, the theme gets a lightweight EditorUI facade (tabs, gutter, editor area) and sets colors there—no if/else spread across the codebase.

Code:

```
import java.util.Objects;

// Strategy
interface Theme {
    void apply(EditorUI ui);
}

// Concrete strategies
class LightTheme implements Theme {
    @Override public void apply(EditorUI ui) {
        ui.setBackground("#FFFFFF");
        ui.setForeground("#222222");
        ui.setGutter("#F3F3F3");
        ui.setAccent("#1E90FF");
        System.out.println("Light theme applied.");
    }
}

class DarkTheme implements Theme {
    @Override public void apply(EditorUI ui) {
        ui.setBackground("#0F141A");
        ui.setForeground("#E6EDF3");
        ui.setGutter("#1B232C");
        ui.setAccent("#58A6FF");
        System.out.println("Dark theme applied.");
    }
}

class SolarizedTheme implements Theme {
    @Override public void apply(EditorUI ui) {
        ui.setBackground("#002B36"); // base03
```

```
        ui.setForeground("#93A1A1"); // base1
        ui.setGutter("#073642");     // base02
        ui.setAccent("#B58900");     // yellow
        System.out.println("Solarized theme applied.");
    }
}

// Minimal UI facade the theme can paint on
class EditorUI {
    void setBackground(String hex) {}
    void setForeground(String hex) {}
    void setGutter(String hex) {}
    void setAccent(String hex) {}
}

// Context
class Editor {
    private final EditorUI ui = new EditorUI();
    private Theme theme;

    public void setTheme(Theme theme) {
        this.theme = Objects.requireNonNull(theme);
        // Immediate apply on switch
        this.theme.apply(ui);
    }

    public void render() {
        // Render cycle asks current theme to ensure consistency
        if (theme != null) theme.apply(ui);
        System.out.println("Rendering editor...");
    }
}

// Usage
class DemoTheme {
    public static void main(String[] args) {
        Editor editor = new Editor();
        editor.setTheme(new DarkTheme());
        editor.render();

        editor.setTheme(new SolarizedTheme());
        editor.render();
    }
}
```

Reasoning to interviewer:

- Strategy isolates “how to paint” from “when to paint.” The editor only knows it has a Theme.

- Swapping strategies at runtime is a simple setTheme(...), no conditionals sprinkled in rendering code.
- The EditorUI facade prevents theme code from touching deep internals—good for stability and tests.
- Re-applying on render() keeps late-loaded components consistent (tabs that appear after a theme switch).

Future scope:

- Add a HighContrastTheme without touching Editor or other themes.
- Persist last-selected theme in user settings and restore on startup.
- Live-preview: apply a theme to a preview pane before committing.
- Theming tokens (semantic roles like editor.bg, accent.primary) to support brand packs and accessibility.

Scenario 13: Observer — news portal subscribers

Readers subscribe to categories (Politics, Sports, Tech). Whenever a new article lands in a category, only that category's subscribers should be notified, and they can unsubscribe anytime.

Answer:

I'll model each category as a subject with a subscriber list. A Reader implements Subscriber. Publishing an article on a category triggers notifications to a snapshot of current subscribers. To support multiple categories cleanly, I'll use a NewsHub that holds a map from category → subject.

Code:

```
import java.util.*;

// Observer
interface Subscriber {
    void notify(String category, String article);
}

// Concrete observer
class Reader implements Subscriber {
    private final String name;
    public Reader(String name) { this.name = name; }
    @Override public void notify(String category, String article) {
        System.out.println(name + " notified [" + category + "]: " + article);
    }
}

// Subject
interface NewsCategory {
    void addSubscriber(Subscriber s);
    void removeSubscriber(Subscriber s);
    void publish(String article);
    String name();
}

class CategoryTopic implements NewsCategory {
    private final String name;
    private final List<Subscriber> subscribers = new ArrayList<>();
    CategoryTopic(String name) { this.name = name; }

    @Override public String name() { return name; }

    @Override public void addSubscriber(Subscriber s) { subscribers.add(s); }

    @Override public void removeSubscriber(Subscriber s) { subscribers.remove(s);
}
```

```
    @Override public void publish(String article) {
        // notify over a snapshot to avoid ConcurrentModification if someone
unsubscribes during notify
        List<Subscriber> snap = List.copyOf(subscribers);
        for (Subscriber s : snap) s.notify(name, article);
    }
}

// Coordinator for multiple categories
class NewsHub {
    private final Map<String, CategoryTopic> topics = new HashMap<>();

    public NewsCategory category(String name) {
        return topics.computeIfAbsent(name.toUpperCase(Locale.ROOT),
CategoryTopic::new);
    }

    public void publish(String category, String article) {
        category(category).publish(article);
    }
}

// Usage
class DemoNews {
    public static void main(String[] args) {
        NewsHub hub = new NewsHub();
        NewsCategory sports = hub.category("Sports");
        NewsCategory tech = hub.category("Tech");

        Subscriber neha = new Reader("Neha");
        Subscriber arjun = new Reader("Arjun");

        sports.addSubscriber(neha);
        sports.addSubscriber(arjun);
        tech.addSubscriber(neha);

        hub.publish("Sports", "India wins the World Cup!");
        hub.publish("Tech", "JDK 22 features explained.");
        sports.removeSubscriber(arjun);
        hub.publish("Sports", "Transfer window: Big signing announced.");
    }
}
```

Reasoning to interviewer:

- Subject/observer separation keeps categories decoupled from subscriber logic; readers can be mobile apps, emailers, or webhooks with the same interface.
- The NewsHub makes it easy to manage many categories without duplicating subject code.
- Snapshot iteration avoids failures when a subscriber unsubscribes during notification.
- Category name normalization (toUpperCase) prevents duplicate topics due to casing.

Future scope:

- Make notifications asynchronous via a queue so slow subscribers don’t block publishing.
- Add per-subscriber filters (e.g., only “breaking” articles) or digest batching.
- Track delivery metrics and retries; move dead subscribers to a quarantine list.
- Support hierarchical topics (e.g., “Tech/AI”) with parent–child fan-out.

Scenario 14: Strategy — discount engines in an online store

In an e-commerce checkout, pricing rules change often: festival promos, loyalty discounts, or location-based taxes. I want to swap this logic at runtime without touching the rest of checkout.

Answer:

I'll define a PricingRule strategy with apply(basePrice, ctx). Each discount/tax is a concrete strategy. The Checkout holds a reference to the current strategy and delegates price calculation. I'll also show a simple selector that picks a rule based on runtime flags (e.g., user is loyal, region requires GST). For realism, I'll clamp totals to a minimum (no negative totals) and keep rounding in one place.

Code:

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.Objects;

// Strategy
interface PricingRule {
    double apply(double basePrice, PricingContext ctx);
}

// Concrete strategies
class FestivalDiscount implements PricingRule {
    @Override public double apply(double basePrice, PricingContext ctx) {
        return basePrice * 0.80; // 20% off
    }
}

class LoyaltyDiscount implements PricingRule {
    @Override public double apply(double basePrice, PricingContext ctx) {
        return basePrice - 100.0; // flat 100 off
    }
}

class TaxAppliedPricing implements PricingRule {
    @Override public double apply(double basePrice, PricingContext ctx) {
        double rate = ctx.taxRatePercent(); // e.g., 18.0
        return basePrice * (1.0 + rate / 100.0);
    }
}

// Context for pricing decisions (region, user flags, etc.)
```

```
record PricingContext(boolean isLoyal, boolean isFestival, double taxRatePercent)
{}

// Selector (could be config/rules-engine driven)
class PricingSelector {
    static PricingRule choose(PricingContext ctx) {
        if (ctx.isFestival()) return new FestivalDiscount();
        if (ctx.isLoyal())    return new LoyaltyDiscount();
        return new TaxAppliedPricing(); // default to tax-only
    }
}

// Checkout uses the strategy
class Checkout {
    private PricingRule rule;

    public void setRule(PricingRule rule) { this.rule = Objects.requireNonNull(rule); }

    public double total(double basePrice, PricingContext ctx) {
        double raw = rule.apply(basePrice, ctx);
        double clamped = Math.max(0.0, raw);
        return round(clamped);
    }

    private static double round(double v) {
        return new BigDecimal(v).setScale(2,
RoundingMode.HALF_UP).doubleValue();
    }
}

// Usage
class DemoStrategyPricing {
    public static void main(String[] args) {
        Checkout cart = new Checkout();
        PricingContext ctx = new PricingContext(true, false, 18.0); // loyal user, no
festival, 18% tax region

        cart.setRule(PricingSelector.choose(ctx)); // Loyalty
        System.out.println("Total: " + cart.total(1000, ctx)); // 900.00

        // Switch at runtime (festival goes live)
        ctx = new PricingContext(true, true, 18.0);
        cart.setRule(PricingSelector.choose(ctx)); // Festival
        System.out.println("Total: " + cart.total(1000, ctx)); // 800.00
    }
}
```

Reasoning to interviewer:

- Strategy isolates each pricing rule; checkout code never changes when promos/taxes change.
- A small PricingContext carries dynamic inputs (loyalty, tax rate). Strategies stay pure and testable.
- The selector demonstrates runtime swapping; in production this might be a rules engine or feature flags.
- Central rounding and clamping avoid scattered money math bugs.

Future scope:

- Compose multiple rules (e.g., festival then tax) via a chain/decorator if business demands stacking.
- Externalize rates and thresholds to config so ops can tune without redeploy.
- Add explainability: return a breakdown (base, discounts, tax) for invoices and audits.
- Support region-specific caps/floors (e.g., max 50% discount) inside dedicated strategies.

Scenario 15: Factory — media file parsers in a player

A player must handle MP3, WAV, FLAC, each with different decoding logic. The client should ask for “type=FLAC” and get the right parser without knowing the details.

Answer:

I’ll define an AudioFile interface with play(file). Each format has a concrete implementation. A factory maps a format to a creator. To keep it open for new formats, I’ll avoid a big switch by using an enum + registry of suppliers. Adding OGG later is just a new class and one registry entry—client code remains unchanged.

Code:

```
import java.util.EnumMap;
import java.util.Locale;
import java.util.Map;
import java.util.Objects;
import java.util.function.Supplier;

// Product
interface AudioFile {
    void play(String file);
}

// Concrete products
class Mp3File implements AudioFile {
    @Override public void play(String file) { System.out.println("Playing MP3: " +
file); }
}

class WavFile implements AudioFile {
    @Override public void play(String file) { System.out.println("Playing WAV: " +
file); }
}

class FlacFile implements AudioFile {
    @Override public void play(String file) { System.out.println("Playing FLAC: " +
file); }
}

// Supported formats
enum AudioType { MP3, WAV, FLAC }

// Factory with registry (open for extension)
class AudioFactory {
```

```
    private static final Map<AudioType, Supplier<AudioFile>> REGISTRY = new
EnumMap<>(AudioType.class);
    static {
        REGISTRY.put(AudioType.MP3,  Mp3File::new);
        REGISTRY.put(AudioType.WAV,  WavFile::new);
        REGISTRY.put(AudioType.FLAC, FlacFile::new);
    }

    public AudioFile getFile(String type) {
        Objects.requireNonNull(type, "type");
        AudioType t = AudioType.valueOf(type.toUpperCase(Locale.ROOT));
        Supplier<AudioFile> sup = REGISTRY.get(t);
        if (sup == null) throw new IllegalArgumentException("Unsupported format: " +
type);
        return sup.get();
    }
}

// Usage
class DemoAudioFactory {
    public static void main(String[] args) {
        AudioFactory factory = new AudioFactory();
        AudioFile f = factory.getFile("FLAC");
        f.play("song.flac");
    }
}
```

Reasoning to interviewer:

- Factory centralizes creation; the player layer depends only on AudioFile.
- Enum + registry avoids brittle switch statements and keeps us open/closed for new formats.
- If a decoder needs heavy init (DSP buffers), the supplier can return a cached instance or a pooled one.
- This design plugs into DI easily: the registry can be populated from configuration or module scanning.

Future scope:

- Add metadata readers (bitrate, duration) to AudioFile or separate AudioProbe interface.
- Support streaming sources (HTTP, HLS) with different implementations behind the same factory.

- Fallback chaining: if native decoder fails, try a pure-Java decoder transparently.
- Feature flags to enable/disable experimental codecs per platform.

Scenario 16: Singleton — configuration manager in a cloud service

In a cloud service, multiple modules (auth, billing, messaging) need shared settings (DB URLs, secrets, feature flags). If each module loads config separately, it wastes resources and risks inconsistencies. We need one global ConfigManager instance.

Answer:

I'll implement ConfigManager as a thread-safe Singleton using double-checked locking with volatile. This ensures:

- Only one instance is created, even under heavy concurrency.
- Initialization happens lazily (only when first requested).
- Volatile prevents instruction reordering issues with partially constructed objects.

Code:

```
public class ConfigManager {
    private static volatile ConfigManager instance;

    // private constructor blocks external instantiation
    private ConfigManager() {}

    public static ConfigManager getInstance() {
        if (instance == null) { // fast path
            synchronized (ConfigManager.class) {
                if (instance == null) { // double-check
                    instance = new ConfigManager();
                }
            }
        }
        return instance;
    }

    public void loadConfig() {
        System.out.println("Loading configuration...");
        // e.g., read from S3 bucket, Vault, or DB once
    }
}

// Usage
class DemoSingleton {
    public static void main(String[] args) {
        ConfigManager cfg1 = ConfigManager.getInstance();
        ConfigManager cfg2 = ConfigManager.getInstance();
        System.out.println(cfg1 == cfg2); // true
        cfg1.loadConfig();
```

```
    }
}
```

Reasoning to interviewer:

- Private constructor prevents uncontrolled instantiation.
- Volatile + double-checked locking ensures safe publication and avoids performance penalty of full synchronization on every call.
- Lazy initialization means config loads only when needed, not at class load time.
- Suitable for cloud services where multiple microservices/threads rely on the same config.

Future scope:

- Add refresh capability (reload config if updated in source).
- Support environment-specific configs (dev, staging, prod).
- Provide immutable snapshots for thread safety during reload.
- Expose change events so components can adapt to new settings dynamically.

Scenario 17: Factory — food ordering app

A delivery platform serves Pizza, Burger, Salad. Each has different preparation logic. Instead of scattering if/else across order flows, we want a factory to centralize creation.

Answer:

I'll define a FoodItem interface with prepare(). Each food type implements its recipe. A FoodFactory decides which concrete item to create based on input. This isolates client code from object creation and makes adding a new item (e.g., Pasta) trivial.

Code:

```
// Product
interface FoodItem {
    void prepare();
}

// Concrete products
class Pizza implements FoodItem {
    @Override public void prepare() {
        System.out.println("Preparing Pizza with toppings...");
    }
}

class Burger implements FoodItem {
    @Override public void prepare() {
        System.out.println("Grilling Burger with cheese...");
    }
}

class Salad implements FoodItem {
    @Override public void prepare() {
        System.out.println("Mixing fresh Salad...");
    }
}

// Factory
class FoodFactory {
    public FoodItem getFood(String type) {
        switch (type.toUpperCase()) {
            case "PIZZA": return new Pizza();
            case "BURGER": return new Burger();
            case "SALAD": return new Salad();
            default: throw new IllegalArgumentException("Unknown food type: " +
type);
        }
    }
}
```

```
// Usage
class DemoFactoryFood {
    public static void main(String[] args) {
        FoodFactory factory = new FoodFactory();
        FoodItem order1 = factory.getFood("Burger");
        order1.prepare();

        FoodItem order2 = factory.getFood("Salad");
        order2.prepare();
    }
}
```

Reasoning to interviewer:

- Factory encapsulates object creation; clients only deal with the FoodItem interface.
- Avoids duplicated if/else logic in multiple places—centralized control.
- Adding new dishes doesn't affect existing client code, just extend the factory.
- This design supports dynamic menu options (e.g., read available items from config/DB).

Future scope:

- Replace switch with a registry (map of type → supplier) for open/closed design.
- Add pricing, preparation time, and calorie info inside each FoodItem.
- Extend to abstract factory: create whole combos (meal = burger + fries + drink).
- Integrate with order tracking: prepare() can emit events for kitchen workflow.

Scenario 18: Decorator — online character skins

In an RPG game, players may stack multiple skins like Fire Aura, Ice Shield, Golden Armor. We don't want to modify the base Player class whenever a new skin is introduced. Instead, we'll decorate the player object at runtime.

Answer:

I'll define a Player interface with a describe() method. BasicPlayer provides the default warrior. Each skin is a decorator implementing Player, wrapping another Player instance and appending its description. Because decorators share the same interface, they can be nested infinitely, allowing stacked skins.

Code:

```
// Component
interface Player {
    String describe();
}

// Concrete component
class BasicPlayer implements Player {
    @Override public String describe() {
        return "Warrior";
    }
}

// Base decorator
abstract class SkinDecorator implements Player {
    protected final Player player;
    public SkinDecorator(Player player) { this.player = player; }
}

// Concrete decorators
class FireAura extends SkinDecorator {
    public FireAura(Player player) { super(player); }
    @Override public String describe() {
        return player.describe() + " with Fire Aura";
    }
}

class IceShield extends SkinDecorator {
    public IceShield(Player player) { super(player); }
    @Override public String describe() {
        return player.describe() + " with Ice Shield";
    }
}

class GoldenArmor extends SkinDecorator {
    public GoldenArmor(Player player) { super(player); }
```

```
    @Override public String describe() {
        return player.describe() + " wearing Golden Armor";
    }
}

// Usage
class DemoDecoratorSkins {
    public static void main(String[] args) {
        Player hero = new GoldenArmor(new IceShield(new FireAura(new
BasicPlayer())));
        System.out.println(hero.describe());
        // Output: Warrior with Fire Aura with Ice Shield wearing Golden Armor
    }
}
```

Reasoning to interviewer:

- The base Player remains untouched; skins are added as wrappers.
- Because all decorators implement Player, they can be stacked in any order.
- Open/closed principle: adding “Shadow Cloak” is just another decorator class, no modifications required elsewhere.
- This pattern allows dynamic combinations at runtime depending on achievements, purchases, or events.

Future scope:

- Each decorator could also add behaviors (e.g., getDefenseBonus()) instead of just description.
- Chain can be introspected to check which skins are active.
- Add a removeSkin() decorator logic to toggle off certain skins.
- Support serialization of the stack (for saving player state in the game DB).

Scenario 19: Observer — weather app alerts

In a weather app, multiple users subscribe to city updates. When temperature changes, all subscribers must be alerted immediately.

Answer:

I'll design WeatherStation as the subject and Subscriber as the observer. CityWeather holds the temperature and a list of subscribers. Whenever setTemp() is called, it triggers notifySubscribers(). Subscribers (users) implement notifyUpdate(temp). They can register or unregister anytime.

Code:

```
import java.util.ArrayList;
import java.util.List;

// Observer
interface Subscriber {
    void notifyUpdate(double temp);
}

// Concrete observer
class User implements Subscriber {
    private final String name;
    public User(String name) { this.name = name; }

    @Override public void notifyUpdate(double temp) {
        System.out.println(name + " received alert: Temperature is " + temp + "°C");
    }
}

// Subject
interface WeatherStation {
    void register(Subscriber s);
    void unregister(Subscriber s);
    void notifySubscribers();
}

class CityWeather implements WeatherStation {
    private final List<Subscriber> subs = new ArrayList<>();
    private double temp;

    public void setTemp(double temp) {
        this.temp = temp;
        notifySubscribers();
    }

    @Override public void register(Subscriber s) { subs.add(s); }
    @Override public void unregister(Subscriber s) { subs.remove(s); }
```

```
    @Override public void notifySubscribers() {
        for (Subscriber s : List.copyOf(subs)) {
            s.notifyUpdate(temp);
        }
    }
}

// Usage
class DemoObserverWeather {
    public static void main(String[] args) {
        CityWeather station = new CityWeather();
        Subscriber u1 = new User("Aman");
        Subscriber u2 = new User("Sara");

        station.register(u1);
        station.register(u2);

        station.setTemp(42.5);  // Notifies both
        station.unregister(u1);
        station.setTemp(36.8);  // Notifies Sara only
    }
}
```

Reasoning to interviewer:

- Observer cleanly decouples the data producer (CityWeather) from consumers (users).
- Subscribers can join/leave dynamically without affecting the subject's logic.
- Snapshot iteration (List.copyOf) ensures safety if a subscriber unsubscribes while notifications are happening.
- This structure scales to thousands of subscribers per city with minimal changes.

Future scope:

- Make alerts asynchronous (via message queues or push notifications).
- Add filtering—users may subscribe only if temp > 40°C or < 5°C.
- Support multiple weather parameters (humidity, wind, rain) with topic-based subscriptions.
- Persist subscription data in a DB to handle restarts and long-lived sessions.

Scenario 20: Command — music player remote

A music player needs to support Play, Pause, and Stop through a remote. The remote should also support undoing the last action. We want to encapsulate each action as a command so the remote is decoupled from the player logic.

Answer:

I'll model each player operation (play, pause, stop) as a command implementing execute() and undo(). The Remote will invoke commands and keep them in a history stack. Undo simply calls the opposite action stored in the command. This lets us expand to new actions (next, previous, volume up) without touching the remote code.

Code:

```
import java.util.Stack;

// Command interface
interface Command {
    void execute();
    void undo();
}

// Receiver
class MusicPlayer {
    public void play() { System.out.println("Playing song"); }
    public void pause() { System.out.println("Paused song"); }
    public void stop() { System.out.println("Stopped song"); }
}

// Concrete commands
class PlayCommand implements Command {
    private final MusicPlayer player;
    public PlayCommand(MusicPlayer player) { this.player = player; }
    public void execute() { player.play(); }
    public void undo() { player.stop(); }
}

class PauseCommand implements Command {
    private final MusicPlayer player;
    public PauseCommand(MusicPlayer player) { this.player = player; }
    public void execute() { player.pause(); }
    public void undo() { player.play(); }
}

class StopCommand implements Command {
    private final MusicPlayer player;
    public StopCommand(MusicPlayer player) { this.player = player; }
    public void execute() { player.stop(); }
    public void undo() { player.play(); }
```

```
}

// Invoker
class Remote {
    private final Stack<Command> history = new Stack<>();

    public void press(Command c) {
        c.execute();
        history.push(c);
    }

    public void undo() {
        if (!history.isEmpty()) {
            history.pop().undo();
        }
    }
}

// Usage
class DemoCommandMusic {
    public static void main(String[] args) {
        MusicPlayer player = new MusicPlayer();
        Remote remote = new Remote();

        remote.press(new PlayCommand(player));  // Playing
        remote.press(new PauseCommand(player)); // Paused
        remote.undo();                          // Undo pause → Play
        remote.press(new StopCommand(player));  // Stopped
        remote.undo();                          // Undo stop → Play
    }
}
```

Reasoning to interviewer:

- The remote never calls play() or pause() directly—it just knows commands.
- Each command object knows how to undo itself, so we can roll back the last action.
- The stack allows multiple undos in order.
- Adding new buttons (like NextTrackCommand) means creating a new class, not touching existing code.

Future scope:

- Add redo functionality with a second stack.
- Support macro commands (playlist mode: play → next → stop).
- Persist command history for replaying user sessions.

- Integrate with UI buttons where each button is wired to a command.

Scenario 21: Strategy — shipping methods

An e-commerce platform supports multiple shipping types (Standard, Express, Drone). Each has a unique cost formula. Instead of hardcoding conditions, we want to switch algorithms dynamically.

Answer:

I'll define a Shipping strategy interface with a cost(weight) method. Each shipping type implements its cost formula. The Checkout class will hold a Shipping reference and delegate cost calculation to it. At runtime, we simply swap strategies based on user choice or delivery availability.

Code:

```
// Strategy
interface Shipping {
    double cost(double weight);
}

// Concrete strategies
class StandardDelivery implements Shipping {
    public double cost(double weight) { return 50 + weight * 5; }
}

class ExpressDelivery implements Shipping {
    public double cost(double weight) { return 100 + weight * 10; }
}

class DroneDelivery implements Shipping {
    public double cost(double weight) { return 200 + weight * 2; }
}

// Context
class Checkout {
    private Shipping shipping;
    public void setShipping(Shipping s) { this.shipping = s; }
    public double calculate(double weight) { return shipping.cost(weight); }
}

// Usage
class DemoStrategyShipping {
    public static void main(String[] args) {
        Checkout order = new Checkout();

        order.setShipping(new StandardDelivery());
        System.out.println("Standard: " + order.calculate(3)); // 65

        order.setShipping(new ExpressDelivery());
```

```
        System.out.println("Express: " + order.calculate(3));  // 130

        order.setShipping(new DroneDelivery());
        System.out.println("Drone: " + order.calculate(3));    // 206
    }
}
```

Reasoning to interviewer:

- Strategy makes each cost formula a self-contained class, so checkout code stays clean.
- Users or business logic can pick a strategy dynamically (promo, distance, availability).
- Easy to test each shipping method independently.
- Open for extension—adding “Same-Day Delivery” is just a new class, no impact on existing code.

Future scope:

- Add context (distance, destination zone, fuel surcharge) to cost formulas.
- Compose strategies (base cost + holiday surcharge).
- Integrate rules engine for dynamic selection.
- Cache calculations for repeated queries in high-traffic apps.

Scenario 22: Abstract Factory — gadget store

A gadget store sells two product lines (Laptop, Smartphone). Each product comes in two variants (Standard, Premium). We want a flexible way to create product–variant combinations without hardcoding them.

Answer:

I'll use the Abstract Factory Pattern. The abstract factory will define creation methods for both product (Gadget) and variant (Variant). Concrete factories decide which product family (Laptop vs Smartphone) and which variant (Standard vs Premium) to return. This way, the client can swap factories and get consistent product–variant pairs without changing its code.

Code:

```
// Abstract products
interface Gadget { void show(); }
interface Variant { void describe(); }

// Concrete products
class Laptop implements Gadget {
    public void show() { System.out.println("Laptop booked"); }
}
class Smartphone implements Gadget {
    public void show() { System.out.println("Smartphone booked"); }
}

// Concrete variants
class Standard implements Variant {
    public void describe() { System.out.println("Standard edition"); }
}
class Premium implements Variant {
    public void describe() { System.out.println("Premium edition with extras"); }
}

// Abstract factory
interface GadgetFactory {
    Gadget createGadget();
    Variant createVariant();
}

// Concrete factories
class StandardLaptopFactory implements GadgetFactory {
    public Gadget createGadget() { return new Laptop(); }
    public Variant createVariant() { return new Standard(); }
}

class PremiumLaptopFactory implements GadgetFactory {
    public Gadget createGadget() { return new Laptop(); }
```

```
    public Variant createVariant() { return new Premium(); }
}

class StandardPhoneFactory implements GadgetFactory {
    public Gadget createGadget() { return new Smartphone(); }
    public Variant createVariant() { return new Standard(); }
}

class PremiumPhoneFactory implements GadgetFactory {
    public Gadget createGadget() { return new Smartphone(); }
    public Variant createVariant() { return new Premium(); }
}

// Usage
class DemoAbstractFactory {
    public static void main(String[] args) {
        GadgetFactory factory = new PremiumLaptopFactory();
        Gadget g = factory.createGadget();
        Variant v = factory.createVariant();

        g.show();      // Laptop booked
        v.describe();  // Premium edition with extras
    }
}
```

Reasoning to interviewer:

- Abstract Factory ensures families of related objects (Laptop + Premium, Smartphone + Standard) are created together.
- Client code is shielded from knowing *which exact classes* are used.
- To switch from Premium Laptop to Standard Phone, we just change the factory instantiation.
- New product families or variants are added by extending factories, not rewriting client code.

Future scope:

- Support regional variants (e.g., “US Premium Laptop”, “EU Standard Phone”).
- Add configuration-driven factory selection at runtime.
- Extend to bundles (Laptop + Smartphone combo pack factory).
- Cache or pool created objects if expensive to construct.

Scenario 23: Factory — notifications in chat app

A chat app must support sending different notifications (Text, Image, Voice). Each has unique logic, but the client should just specify the type, not worry about object creation.

Answer:

I'll define a Notification interface with send(). Each notification type implements it. A NotificationFactory maps a string type to the correct object. This centralizes creation and makes it easy to extend the system for new types like Video or File notifications.

Code:

```
// Product
interface Notification {
    void send();
}

// Concrete products
class TextNotification implements Notification {
    public void send() { System.out.println("Text notification sent"); }
}

class ImageNotification implements Notification {
    public void send() { System.out.println("Image notification sent"); }
}

class VoiceNotification implements Notification {
    public void send() { System.out.println("Voice notification sent"); }
}

// Factory
class NotificationFactory {
    public Notification get(String type) {
        switch (type.toUpperCase()) {
            case "TEXT": return new TextNotification();
            case "IMAGE": return new ImageNotification();
            case "VOICE": return new VoiceNotification();
            default: throw new IllegalArgumentException("Invalid notification type: " +
type);
        }
    }
}

// Usage
class DemoFactoryNotification {
    public static void main(String[] args) {
        NotificationFactory factory = new NotificationFactory();
        Notification n1 = factory.get("VOICE");
```

```
        n1.send();

        Notification n2 = factory.get("IMAGE");
        n2.send();
    }
}
```

Reasoning to interviewer:

- Factory encapsulates creation, so client code doesn't need to use new directly.
- Centralizing object creation avoids duplicated if/else logic across the app.
- Adding a new type (VideoNotification) just means a new class + factory update.
- Promotes open/closed principle, especially if we migrate to registry-based factories later.

Future scope:

- Replace switch with a registry map (Map<String, Supplier<Notification>>) for more extensibility.
- Allow asynchronous delivery strategies inside notification objects.
- Add metadata (sender, timestamp) into Notification.
- Integrate with multiple channels (push, email, SMS) under the same abstraction.

Scenario 24: Command — job scheduler

A server needs to queue different jobs (backup, report generation, email) and run them later. We want to encapsulate each job as a command so scheduling, ordering, and execution are decoupled from job logic.

Answer:

I'll represent each job as a Job command with a single run() method. The Scheduler acts as the invoker: it accepts jobs, stores them in a queue, and executes them later. This keeps scheduling concerns (when/how to run) separate from job behavior (what to run). I'll also show a variant with delayed execution and a thread pool for realism.

Code:

```
import java.time.Instant;
import java.util.*;
import java.util.concurrent.*;

// Command
interface Job {
    void run();
}

// Receivers / concrete commands
class BackupJob implements Job {
    @Override public void run() { System.out.println("[" + Instant.now() + "] Running
backup job"); }
}

class ReportJob implements Job {
    @Override public void run() { System.out.println("[" + Instant.now() + "]
Generating reports"); }
}

class EmailJob implements Job {
    @Override public void run() { System.out.println("[" + Instant.now() + "] Sending
emails"); }
}

// Invoker (simple sync queue)
class Scheduler {
    private final Queue<Job> queue = new ArrayDeque<>();

    public void addJob(Job j) { queue.offer(j); }

    public void executeAll() {
        while (!queue.isEmpty()) {
            queue.poll().run();
        }
```

```
    }
}

// Optional: delayed + concurrent scheduler (more real-world)
class AsyncScheduler implements AutoCloseable {
    private static final class ScheduledJob implements Delayed {
        final long triggerAtNanos;
        final Job job;
        ScheduledJob(Job job, long delay, TimeUnit unit) {
            this.job = job;
            this.triggerAtNanos = System.nanoTime() + unit.toNanos(delay);
        }
        @Override public long getDelay(TimeUnit unit) { return
unit.convert(triggerAtNanos - System.nanoTime(), TimeUnit.NANOSECONDS); }
        @Override public int compareTo(Delayed o) { return
Long.compare(getDelay(TimeUnit.NANOSECONDS),
o.getDelay(TimeUnit.NANOSECONDS)); }
    }

    private final DelayQueue<ScheduledJob> dq = new DelayQueue<>();
    private final ExecutorService pool =
Executors.newFixedThreadPool(Runtime.getRuntime().availableProcessors());
    private final Thread dispatcher;

    public AsyncScheduler() {
        dispatcher = new Thread(() -> {
            try {
                while (!Thread.currentThread().isInterrupted()) {
                    ScheduledJob sj = dq.take();      // blocks until due
                    pool.submit(sj.job::run);         // execute in pool
                }
            } catch (InterruptedException ignored) { Thread.currentThread().interrupt();
}
        }, "job-dispatcher");
        dispatcher.start();
    }

    public void schedule(Job job, long delay, TimeUnit unit) {
        dq.offer(new ScheduledJob(job, delay, unit));
    }

    @Override public void close() {
        dispatcher.interrupt();
        pool.shutdown();
    }
}

// Usage
class DemoCommandScheduler {
    public static void main(String[] args) throws Exception {
```

```
        // Simple FIFO scheduler
        Scheduler s = new Scheduler();
        s.addJob(new BackupJob());
        s.addJob(new ReportJob());
        s.addJob(new EmailJob());
        s.executeAll();

        // Delayed concurrent scheduler
        try (AsyncScheduler async = new AsyncScheduler()) {
            async.schedule(new BackupJob(), 1, TimeUnit.SECONDS);
            async.schedule(new EmailJob(),  500, TimeUnit.MILLISECONDS);
            async.schedule(new ReportJob(), 1500, TimeUnit.MILLISECONDS);
            Thread.sleep(2200); // wait to observe output
        }
    }
}
```

Reasoning to interviewer:

- Commands encapsulate “what to do”; the scheduler only cares about “when and in what order.”
- Adding a new job type is just a new class implementing Job; the scheduler remains unchanged.
- The async variant shows how to evolve from a simple queue to delayed and parallel execution without touching job classes.
- This design makes it trivial to plug in retries, auditing, or priority queues at the invoker layer.

Future scope:

- Add priority scheduling (e.g., PriorityBlockingQueue) and retry/backoff policies.
- Persist the queue for crash recovery (append-only log, outbox pattern).
- Add job metadata (id, createdAt, tenant) and middleware (metrics, tracing).
- Support cancellation and idempotency keys to avoid duplicate side effects.

Scenario 25: Builder — SaaS subscription plans

A SaaS product offers a base plan name with optional add-ons (Analytics, API Access, Support). Customers mix and match. We want readable construction and an immutable result.

Answer:

I'll make Plan immutable with a nested Builder. The builder's constructor takes the required name. Optional features are toggled via fluent methods. I'll add basic validation (non-empty name) and a convenience toString() for debugging. This keeps construction expressive while preventing invalid states.

Code:

```
import java.util.Objects;

public final class Plan {
    private final String name;
    private final boolean analytics;
    private final boolean apiAccess;
    private final boolean support;

    private Plan(Builder b) {
        this.name = b.name;
        this.analytics = b.analytics;
        this.apiAccess = b.apiAccess;
        this.support = b.support;
    }

    public String name() { return name; }
    public boolean analytics() { return analytics; }
    public boolean apiAccess() { return apiAccess; }
    public boolean support() { return support; }

    @Override public String toString() {
        return "Plan{name='" + name + "', analytics=" + analytics +
                ", apiAccess=" + apiAccess + ", support=" + support + "}";
    }

    public static class Builder {
        private final String name;
        private boolean analytics;
        private boolean apiAccess;
        private boolean support;

        public Builder(String name) {
            this.name = validate(name);
        }
        public Builder withAnalytics() { this.analytics = true; return this; }
```

```
        public Builder withApi()       { this.apiAccess = true; return this; }
        public Builder withSupport()   { this.support = true; return this; }

        public Plan build() { return new Plan(this); }

        private static String validate(String n) {
            Objects.requireNonNull(n, "name required");
            String t = n.trim();
            if (t.isEmpty()) throw new IllegalArgumentException("name required");
            return t;
        }
    }
}

// Usage
class DemoPlanBuilder {
    public static void main(String[] args) {
        Plan enterprise = new Plan.Builder("Enterprise")
                .withAnalytics()
                .withApi()
                .withSupport()
                .build();

        Plan starter = new Plan.Builder("Starter")
                .withApi()
                .build();

        System.out.println(enterprise);
        System.out.println(starter);
    }
}
```

Reasoning to interviewer:

- Builder separates required (name) from optional add-ons; callers compose plans fluently.
- The object is immutable (final fields, no setters), which is safer for concurrent access and caching.
- Validation happens once at construction, preventing invalid state from leaking into the system.
- The API reads naturally in UI/service code, reducing wiring mistakes.

Future scope:

- Add computed pricing in the builder (e.g., base + per-addon), and expose a price() on Plan.

- Introduce tiers as enums and guard incompatible combinations (e.g., Starter cannot have Support SLA-Gold).
- Provide toBuilder() for editing existing plans in an admin console.
- Localize plan names/descriptions and serialize plans to JSON for APIs.

Scenario 26: Factory Method — online education bookings

A portal offers multiple session types (Doubt-clearing, Mentor 1:1, Mock Interview). Each has slightly different booking steps (e.g., mentor sessions ask extra questions). We want creation encapsulated so the UI just “requests a session” and the right process runs.

Answer:

I’ll define a Session product with reserve(). An abstract SessionFactory exposes createSession() and optional hooks for pre/post steps (like validating forms, capturing payments). Concrete factories (Doubt/Mentor/Mock) return the appropriate Session and can customize the flow without changing client code. This keeps the booking controller thin and open for new session types later.

Code:

```
// Product
interface Session {
    void reserve();
}

// Concrete products
class DoubtSession implements Session {
    @Override public void reserve() {
        System.out.println("Reserved a general doubt session.");
    }
}

class MentorSession implements Session {
    @Override public void reserve() {
        System.out.println("Reserved a mentorship session with extra
questionnaire.");
    }
}

class MockInterviewSession implements Session {
    @Override public void reserve() {
        System.out.println("Reserved a mock interview with interviewer
assignment.");
    }
}

// Factory Method
abstract class SessionFactory {
    public final Session book() {
        preChecks();
        Session s = createSession();
        s.reserve();
        postActions();
```

```
        return s;
    }
    protected void preChecks() { /* default no-op; e.g., slot availability */ }
    protected void postActions() { /* default no-op; e.g., send confirmation */ }
    public abstract Session createSession();
}

// Concrete factories
class DoubtSessionFactory extends SessionFactory {
    @Override public Session createSession() { return new DoubtSession(); }
}

class MentorSessionFactory extends SessionFactory {
    @Override protected void preChecks() { System.out.println("Collecting mentor
questionnaire & NDA consent."); }
    @Override public Session createSession() { return new MentorSession(); }
    @Override protected void postActions() { System.out.println("Matching expert
mentor and sharing pre-read links."); }
}

class MockInterviewFactory extends SessionFactory {
    @Override protected void preChecks() { System.out.println("Selecting
interviewer by tech stack & experience level."); }
    @Override public Session createSession() { return new
MockInterviewSession(); }
    @Override protected void postActions() { System.out.println("Sending interview
prep checklist and calendar invite."); }
}

// Usage
class DemoFactoryMethodEdu {
    public static void main(String[] args) {
        SessionFactory f1 = new MentorSessionFactory();
        f1.book();

        SessionFactory f2 = new DoubtSessionFactory();
        f2.book();

        SessionFactory f3 = new MockInterviewFactory();
        f3.book();
    }
}
```

Reasoning to interviewer:

- Factory Method moves object creation behind SessionFactory, so controllers don’t new anything.

- Hooks (preChecks, postActions) let each factory inject custom steps without branching in client code.
- Adding “System Design Workshop” is a new factory + session class—no edits to existing flow.
- Keeps booking logic composable and testable: each factory can be unit tested in isolation.

Future scope:

- Add payment gateways per session type inside factories (e.g., refundable vs non-refundable).
- Slot allocation and waitlist logic can live in preChecks().
- Internationalization (locale-specific templates) in postActions().
- A registry to pick factories by SessionType enum or config for runtime selection.

Scenario 27: Decorator — ride-sharing features

A passenger books a basic ride and optionally adds stackable features like Priority Pickup and Pool Ride. We don't want to keep modifying the core booking class whenever a new feature is introduced.

Answer:

I'll model Ride as the base component with book() (and I'll add cost() to show realistic behavior). Each optional feature is a decorator wrapping another Ride, augmenting description and price. Features compose freely, so order and combinations are flexible without touching BasicRide.

Code:

```
// Component
interface Ride {
    String book();
    double cost();
}

// Concrete component
class BasicRide implements Ride {
    @Override public String book() { return "Basic ride booked"; }
    @Override public double cost() { return 120.0; } // base fare
}

// Base decorator
abstract class RideDecorator implements Ride {
    protected final Ride ride;
    protected RideDecorator(Ride ride) { this.ride = ride; }
}

// Concrete decorators
class PriorityPickup extends RideDecorator {
    public PriorityPickup(Ride ride) { super(ride); }
    @Override public String book() { return ride.book() + " with Priority Pickup"; }
    @Override public double cost() { return ride.cost() + 40.0; } // surcharge
}

class PoolRide extends RideDecorator {
    public PoolRide(Ride ride) { super(ride); }
    @Override public String book() { return ride.book() + " as a Pool Ride"; }
    @Override public double cost() { return Math.max(80.0, ride.cost() - 30.0); } //
discount with a floor
}

class ExtraLuggage extends RideDecorator {
    public ExtraLuggage(Ride ride) { super(ride); }
    @Override public String book() { return ride.book() + " with Extra Luggage"; }
```

```
    @Override public double cost() { return ride.cost() + 25.0; }
}

// Usage
class DemoRideDecorator {
    public static void main(String[] args) {
        Ride r1 = new PriorityPickup(new PoolRide(new BasicRide()));
        System.out.println(r1.book());            // Basic ride booked as a Pool Ride with
Priority Pickup
        System.out.println("Fare: " + r1.cost());  // base 120 -30 +40 = 130

        Ride r2 = new ExtraLuggage(new PriorityPickup(new BasicRide()));
        System.out.println(r2.book());
        System.out.println("Fare: " + r2.cost());  // 120 +40 +25 = 185
    }
}
```

Reasoning to interviewer:

- Decorator cleanly adds cross-cutting ride features without subclass explosion (BasicRideWithXAndY).
- All features implement Ride, so they're stackable and interchangeable.
- Pricing logic sits with the feature that owns it, keeping responsibilities local.
- Easy to test: each decorator's effect on description and cost is small and focused.

Future scope:

- Add surge pricing or time-of-day adjustments as decorators (or a separate pricing strategy).
- Provide a way to inspect active features (e.g., list decorators) for receipts.
- Introduce constraints (e.g., PriorityPickup not allowed with Pool in some regions) via a validator layer.
- Cache book()/cost() for identical combinations if performance becomes critical.

Scenario 28: Observer — cryptocurrency price alerts

In a crypto exchange, multiple investors track coin prices. If Bitcoin drops below a threshold (say $30,000), all subscribers should get notified instantly. The Observer Pattern fits well here: CoinMarket is the subject, Investor is the observer.

Answer:

I'll define an Investor interface with update(). Each Trader implements it to print alerts. CoinMarket manages subscription and notification. BitcoinMarket maintains price and triggers notifications if the threshold is breached. This keeps investors decoupled from the market logic.

Code:

```
import java.util.ArrayList;
import java.util.List;

// Observer
interface Investor {
    void update(String coin, double price);
}

// Concrete observer
class Trader implements Investor {
    private final String name;
    public Trader(String name) { this.name = name; }

    @Override public void update(String coin, double price) {
        System.out.println(name + " notified: " + coin + " dropped to $" + price);
    }
}

// Subject
interface CoinMarket {
    void subscribe(Investor inv);
    void unsubscribe(Investor inv);
    void notifyAllSubs();
}

// Concrete subject
class BitcoinMarket implements CoinMarket {
    private final List<Investor> subs = new ArrayList<>();
    private double price;

    public void setPrice(double price) {
        this.price = price;
        if (price < 30000) notifyAllSubs();
    }
```

```
    @Override public void subscribe(Investor inv) { subs.add(inv); }
    @Override public void unsubscribe(Investor inv) { subs.remove(inv); }
    @Override public void notifyAllSubs() {
        for (Investor i : List.copyOf(subs)) i.update("BTC", price);
    }
}

// Usage
class DemoObserverCrypto {
    public static void main(String[] args) {
        BitcoinMarket btc = new BitcoinMarket();
        btc.subscribe(new Trader("Arjun"));
        btc.subscribe(new Trader("Neha"));

        btc.setPrice(31000); // no notification
        btc.setPrice(29500); // triggers alerts
    }
}
```

Reasoning to interviewer:

- Observer decouples price monitoring from investor alerting.
- Investors can join or leave dynamically without changing BitcoinMarket.
- Using a snapshot (List.copyOf) avoids concurrent modification if someone unsubscribes during notifications.
- Threshold logic is encapsulated inside setPrice(), keeping conditions close to state changes.

Future scope:

- Extend to multiple coins (BTC, ETH, SOL) with a Map<String, Double> in CoinMarket.
- Add per-investor thresholds, so Neha gets alerts at $29k while Arjun at $25k.
- Notify asynchronously via event queues to avoid blocking on slow observers.
- Add persistence for subscriptions so investors remain subscribed across restarts.

Scenario 29: Command — restaurant ordering system

A digital kiosk in a restaurant lets customers order dishes, cancel them, or request a bill. We want each action encapsulated as a command and undoable.

Answer:

I'll define a Command interface with execute() and undo(). Each action (OrderDishCommand, CancelDishCommand, RequestBillCommand) wraps the Kitchen receiver. The Kiosk is the invoker: it runs commands and stores history for undo. This way, the kiosk never calls kitchen methods directly—it just triggers commands.

Code:

```
import java.util.Stack;

// Command interface
interface Command {
    void execute();
    void undo();
}

// Receiver
class Kitchen {
    public void prepare(String dish) { System.out.println("Preparing " + dish); }
    public void cancel(String dish) { System.out.println("Cancelled " + dish); }
    public void printBill() { System.out.println("Bill generated for your table."); }
}

// Concrete commands
class OrderDishCommand implements Command {
    private final Kitchen kitchen;
    private final String dish;
    public OrderDishCommand(Kitchen k, String dish) { this.kitchen = k; this.dish =
dish; }

    @Override public void execute() { kitchen.prepare(dish); }
    @Override public void undo() { kitchen.cancel(dish); }
}

class CancelDishCommand implements Command {
    private final Kitchen kitchen;
    private final String dish;
    public CancelDishCommand(Kitchen k, String dish) { this.kitchen = k; this.dish =
dish; }

    @Override public void execute() { kitchen.cancel(dish); }
    @Override public void undo() { kitchen.prepare(dish); }
}
```

```
class RequestBillCommand implements Command {
    private final Kitchen kitchen;
    public RequestBillCommand(Kitchen k) { this.kitchen = k; }

    @Override public void execute() { kitchen.printBill(); }
    @Override public void undo() { System.out.println("Bill request undone (session
continues)."); }
}

// Invoker
class Kiosk {
    private final Stack<Command> history = new Stack<>();

    public void press(Command c) {
        c.execute();
        history.push(c);
    }

    public void undoLast() {
        if (!history.isEmpty()) {
            history.pop().undo();
        }
    }
}

// Usage
class DemoCommandRestaurant {
    public static void main(String[] args) {
        Kitchen kitchen = new Kitchen();
        Kiosk kiosk = new Kiosk();

        kiosk.press(new OrderDishCommand(kitchen, "Pasta"));
        kiosk.press(new CancelDishCommand(kitchen, "Pasta"));
        kiosk.press(new RequestBillCommand(kitchen));

        kiosk.undoLast(); // undo bill request
        kiosk.undoLast(); // undo cancel → re-prepares pasta
    }
}
```

Reasoning to interviewer:

- Command encapsulates all actions (Order, Cancel, Bill) in objects, decoupling kiosk from kitchen.
- Undo is straightforward because each command knows its inverse.
- History stack allows multiple undo operations in reverse order.

- New features (e.g., “Request Water”) are added via new command classes, not by editing existing code.

Future scope:

- Add redo stack for re-applying undone actions.
- Support macro commands (combo meals = multiple dish orders in one command).
- Log executed commands for audit/billing reconciliation.
- Asynchronous execution for heavy tasks (e.g., large group order printing).

Scenario 30: Strategy — adaptive learning difficulty

An online learning app should adapt quiz difficulty (Easy, Medium, Hard) on the fly based on a student's recent performance. We want to switch the difficulty algorithm at runtime without sprinkling conditionals everywhere.

Answer:

I'll model each difficulty as a DifficultyStrategy that knows how to generate a quiz. The QuizEngine holds a reference to the current strategy and delegates quiz generation to it. A small StudentContext (recent accuracy, streak, time per question) can guide a selector that decides which strategy to use; swapping difficulty then becomes a single setDifficulty(...) call.

Code:

```
// Strategy
interface DifficultyStrategy {
    void generateQuiz(StudentContext ctx);
}

// Concrete strategies
class EasyDifficulty implements DifficultyStrategy {
    @Override public void generateQuiz(StudentContext ctx) {
        System.out.println("Generated Easy Quiz: fundamentals, single-step
questions.");
    }
}
class MediumDifficulty implements DifficultyStrategy {
    @Override public void generateQuiz(StudentContext ctx) {
        System.out.println("Generated Medium Quiz: mixed concepts, 2-step
reasoning.");
    }
}
class HardDifficulty implements DifficultyStrategy {
    @Override public void generateQuiz(StudentContext ctx) {
        System.out.println("Generated Hard Quiz: multi-step, edge cases, time
pressure.");
    }
}

// Context about the learner
final class StudentContext {
    final double recentAccuracy;  // 0.0..1.0
    final int correctStreak;      // consecutive correct
    final double avgTimeSecs;     // avg seconds per question
    StudentContext(double recentAccuracy, int correctStreak, double avgTimeSecs)
{
        this.recentAccuracy = recentAccuracy;
        this.correctStreak = correctStreak;
```

```
        this.avgTimeSecs = avgTimeSecs;
    }
}

// Strategy selector (simple rules; could be an ML model later)
final class DifficultySelector {
    static DifficultyStrategy choose(StudentContext ctx) {
        if (ctx.recentAccuracy >= 0.85 && ctx.correctStreak >= 5 && ctx.avgTimeSecs
<= 25)
            return new HardDifficulty();
        if (ctx.recentAccuracy >= 0.6)
            return new MediumDifficulty();
        return new EasyDifficulty();
    }
}

// Engine that hosts the current strategy
class QuizEngine {
    private DifficultyStrategy strategy;
    public void setDifficulty(DifficultyStrategy s) { this.strategy = s; }
    public void startQuiz(StudentContext ctx) {
        if (strategy == null) throw new IllegalStateException(" Difficulty not set ");
        strategy.generateQuiz(ctx);
    }
}

// Usage
class DemoStrategyAdaptive {
    public static void main(String[] args) {
        QuizEngine engine = new QuizEngine();
        StudentContext ctx1 = new StudentContext(0.55, 1, 38);
        engine.setDifficulty(DifficultySelector.choose(ctx1));
        engine.startQuiz(ctx1);  // Easy

        StudentContext ctx2 = new StudentContext(0.88, 6, 22);
        engine.setDifficulty(DifficultySelector.choose(ctx2));
        engine.startQuiz(ctx2);  // Hard
    }
}
```

Reasoning to interviewer:

- Strategy cleanly separates “how to generate a quiz” from “when to switch difficulty.”
- The engine depends only on the interface, so changing levels is one setter call—no if/else sprawl.

- The selector encapsulates heuristics, keeping strategies pure and easy to unit test.
- Passing StudentContext into generateQuiz allows fine-tuning (e.g., pick specific topics the student struggles with).

Future scope:

- Replace heuristic selector with an ML model or multi-armed bandit.
- Add topic-aware strategies (Easy-Algebra, Hard-Graphs) and compose difficulty + topic.
- Log outcomes to adjust thresholds automatically (closed-loop personalization).
- Introduce cooldown timers to prevent thrashing between difficulties.

Scenario 31: Factory — shipment order types in logistics

A logistics platform creates different shipment orders: Same-Day, Scheduled, International. Each has different prep steps (urgent labeling, slot coordination, customs/insurance). The client should request a type and get the right object without knowing the internals.

Answer:

I'll keep a ShipmentOrder product with a prepare() method. A ShipmentFactory maps a type string (or enum) to the correct concrete product. This centralizes creation and keeps the booking workflow clean. I'll also add a registry-based variant to make adding new types (e.g., "ColdChain") a one-line change.

Code:

```
import java.util.EnumMap;
import java.util.Locale;
import java.util.Map;
import java.util.function.Supplier;

// Product
interface ShipmentOrder { void prepare(); }

// Concrete products
class SameDayOrder implements ShipmentOrder {
    @Override public void prepare() {
        System.out.println("Same-day: assign courier and label as urgent.");
    }
}
class ScheduledOrder implements ShipmentOrder {
    @Override public void prepare() {
        System.out.println("Scheduled: confirm pickup slot and route.");
    }
}
class InternationalOrder implements ShipmentOrder {
    @Override public void prepare() {
        System.out.println("International: attach customs paperwork and insurance.");
    }
}

// Types
enum ShipmentType { SAME_DAY, SCHEDULED, INTERNATIONAL }

// Factory (registry-based for open/closed design)
class ShipmentFactory {
    private static final Map<ShipmentType, Supplier<ShipmentOrder>> REGISTRY
= new EnumMap<>(ShipmentType.class);
    static {
        REGISTRY.put(ShipmentType.SAME_DAY, SameDayOrder::new);
```

```
        REGISTRY.put(ShipmentType.SCHEDULED, ScheduledOrder::new);
        REGISTRY.put(ShipmentType.INTERNATIONAL, InternationalOrder::new);
    }

    public ShipmentOrder create(String kind) {
        ShipmentType t = ShipmentType.valueOf(kind.toUpperCase(Locale.ROOT));
        Supplier<ShipmentOrder> sup = REGISTRY.get(t);
        if (sup == null) throw new IllegalArgumentException("Unknown shipment type:
" + kind);
        return sup.get();
    }
}

// Usage
class DemoLogisticsFactory {
    public static void main(String[] args) {
        ShipmentFactory factory = new ShipmentFactory();
        ShipmentOrder order = factory.create("international");
        order.prepare();
    }
}
```

Reasoning to interviewer:

- Factory centralizes object creation and hides type details from the caller.
- Registry avoids large switch statements and keeps the design open for extension.
- Each order encapsulates its prep steps, so business logic lives with the type that owns it.
- Swapping the creation mechanism (e.g., caching heavy objects or returning pooled instances) doesn’t touch the client.

Future scope:

- Add preconditions and validations per type (volumetric weight, hazardous materials, documents present).
- Introduce an abstract factory for region-specific variants (EU vs US customs).
- Make the registry pluggable via configuration so ops can enable/disable types at runtime.
- Add a quote() method to compute price/ETA consistently across order types.

Scenario 32: Builder — video-conferencing plan configurator

A SaaS provider sells conferencing plans with optional add-ons like Cloud Recording, SSO, Priority SLA, and variable participant limits. Customers can mix and match features to build a tailored plan. We want clean construction while keeping ConferencingPlan immutable.

Answer:

I'll implement ConferencingPlan as an immutable class with a nested Builder. The builder sets required fields (plan name) in the constructor, while optional add-ons are enabled through fluent methods. This ensures readability and safety—no giant constructor with many optional parameters.

Code:

```
// Product
class ConferencingPlan {
    private final String name;
    private final int participants;
    private final boolean cloudRecording;
    private final boolean sso;
    private final boolean prioritySla;

    private ConferencingPlan(Builder b) {
        this.name = b.name;
        this.participants = b.participants;
        this.cloudRecording = b.cloudRecording;
        this.sso = b.sso;
        this.prioritySla = b.prioritySla;
    }

    @Override
    public String toString() {
        return "Plan{" +
                "name='" + name + '\'' +
                ", participants=" + participants +
                ", cloudRecording=" + cloudRecording +
                ", sso=" + sso +
                ", prioritySla=" + prioritySla +
                '}';
    }

    // Builder
    public static class Builder {
        private final String name;
        private int participants = 100; // default
        private boolean cloudRecording;
        private boolean sso;
        private boolean prioritySla;
```

```
        public Builder(String name) { this.name = name; }

        public Builder participants(int n) { this.participants = n; return this; }
        public Builder enableRecording() { this.cloudRecording = true; return this; }
        public Builder enableSSO() { this.sso = true; return this; }
        public Builder enablePrioritySLA() { this.prioritySla = true; return this; }

        public ConferencingPlan build() { return new ConferencingPlan(this); }
    }
}

// Usage
class DemoBuilderConferencing {
    public static void main(String[] args) {
        ConferencingPlan enterprise = new ConferencingPlan.Builder("Enterprise")
                .participants(1000)
                .enableRecording()
                .enableSSO()
                .enablePrioritySLA()
                .build();

        ConferencingPlan starter = new ConferencingPlan.Builder("Starter")
                .participants(200)
                .build();

        System.out.println(enterprise);
        System.out.println(starter);
    }
}
```

Reasoning to interviewer:

- Builder avoids telescoping constructors (e.g., new Plan("Enterprise", true, false, true, …)), making it clear which features are enabled.
- The object is immutable—thread-safe and reliable for caching.
- Default values (like 100 participants) are managed inside the builder.
- This design supports readability and safe customization.

Future scope:

- Add pricing calculation directly in the builder, so clients can see plan cost before calling build().
- Provide validations (e.g., participants limit for Starter plans).
- Add toBuilder() for editing existing plans in an admin dashboard.

- Persist/serialize plans to JSON for API responses.

Scenario 33: Strategy — route selection in maps

A maps app supports different routing behaviors: Fastest, Toll-Free, and Scenic. The user may switch strategies at runtime (e.g., avoid tolls mid-drive). We want clean swapping of routing logic without rewriting Navigator.

Answer:

I'll define RouteStrategy as the strategy interface with navigate(from, to). Each concrete class implements its own logic (fastest highways, toll-free local roads, scenic routes). The Navigator holds a reference to the strategy and delegates navigation to it. Switching strategies is a simple setter call.

Code:

```
// Strategy
interface RouteStrategy {
    void navigate(String from, String to);
}

// Concrete strategies
class FastestRoute implements RouteStrategy {
    @Override public void navigate(String from, String to) {
        System.out.println("Navigating " + from + " → " + to + " via highways
(fastest).");
    }
}
class TollFreeRoute implements RouteStrategy {
    @Override public void navigate(String from, String to) {
        System.out.println("Navigating " + from + " → " + to + " avoiding toll roads.");
    }
}
class ScenicRoute implements RouteStrategy {
    @Override public void navigate(String from, String to) {
        System.out.println("Navigating " + from + " → " + to + " using scenic
byways.");
    }
}

// Context
class Navigator {
    private RouteStrategy strategy;
    public void setStrategy(RouteStrategy s) { this.strategy = s; }
    public void start(String from, String to) {
        if (strategy == null) throw new IllegalStateException("Route strategy not
set!");
        strategy.navigate(from, to);
    }
}
```

```
// Usage
class DemoStrategyRoute {
    public static void main(String[] args) {
        Navigator nav = new Navigator();
        nav.setStrategy(new FastestRoute());
        nav.start("Pune", "Goa");

        nav.setStrategy(new ScenicRoute());
        nav.start("Pune", "Goa");
    }
}
```

Reasoning to interviewer:

- Strategy makes each routing behavior a self-contained class, so Navigator stays clean.
- Swapping strategies at runtime allows dynamic user preferences (avoid tolls, scenic drive).
- Easy to test each strategy independently without mocking Navigator.
- Adding new strategies (e.g., “Eco-Friendly Route”) requires no changes to Navigator.

Future scope:

- Add route scoring based on live traffic, fuel efficiency, or weather.
- Support multi-strategy composition (fastest + toll-free).
- Externalize strategies so the app can load new route plugins dynamically.
- Use ML-driven strategies that personalize based on user driving history.

Scenario 34: Factory Method — document exporters

A reporting service must export the same report into different formats (PDF or Markdown). The export workflow (fetch data → render → write to file/stream) stays the same, but the concrete exporter changes. We want a creation hook so teams can add new formats without touching the workflow.

Answer:

I'll model a Exporter product with export(content). The base ExportJob is the creator and defines run(content)—that's the shared workflow. Subclasses override createExporter() to decide which concrete exporter to use (PDF/Markdown/anything new). This cleanly separates pipeline orchestration from format-specific generation. I'll also show where pre/post hooks (watermarks, headers, analytics) live in the base class so new formats inherit them automatically.

Code:

```
// Product
interface Exporter {
    void export(String content);
}

// Concrete products
class PdfExporter implements Exporter {
    @Override public void export(String content) {
        // Real life: render via PDF lib, embed fonts, paginate
        System.out.println("Generating PDF: " + content);
    }
}
class MarkdownExporter implements Exporter {
    @Override public void export(String content) {
        // Real life: write .md, escape code blocks, add front-matter
        System.out.println("Generating Markdown: " + content);
    }
}

// Creator
abstract class ExportJob {
    // Factory Method — creation hook
    protected abstract Exporter createExporter();

    // Template-ish workflow: fetch → transform → export → audit
    public final void run(String raw) {
        String normalized = preprocess(raw);
        Exporter exporter = createExporter();
        exporter.export(normalized);
        postAudit(normalized);
    }
```

```
    protected String preprocess(String raw) {
        // Common steps: trim, add header/footer tokens, sanitize
        return "[HEADER]\n" + raw.trim() + "\n[FOOTER]";
    }

    protected void postAudit(String content) {
        // Common telemetry/logging
        System.out.println("Export audit logged for " + getClass().getSimpleName());
    }
}

// Concrete creators
class PdfExportJob extends ExportJob {
    @Override protected Exporter createExporter() { return new PdfExporter(); }
}

class MdExportJob extends ExportJob {
    @Override protected Exporter createExporter() { return new
MarkdownExporter(); }
}

// Usage
class DemoFactoryMethodExport {
    public static void main(String[] args) {
        ExportJob job = new PdfExportJob();
        job.run("Quarterly report");
        job = new MdExportJob();
        job.run("Quarterly report");
    }
}
```

Reasoning to interviewer:

- Factory Method gives a single overridable creation point (createExporter()), so adding HTML/Docx is a small subclass, not a workflow rewrite.
- The shared run() enforces consistency (preprocessing, auditing) across all formats—no drift between teams.
- If a new exporter needs special setup (fonts, templates), its concrete creator is the right place to assemble that.
- This plays nicely with dependency injection—createExporter() can pull a bean, not new.

Future scope:

- Add OutputTarget (file, S3, HTTP response) as another variation; keep run() stable.
- Support streaming for huge reports (exporter writes in chunks).
- Add feature flags to switch exporters at runtime (e.g., canary a new PDF engine).
- Introduce error handling and retries in run() so all exporters inherit robustness.

Scenario 35: Strategy — incident alert channels

An incident platform must notify on-call via different channels (Email, Slack, PagerDuty). Teams may change the active channel per rotation or severity. We want the channel pluggable without changing the alerting code.

Answer:

I'll model each channel as an AlertChannel strategy with send(msg). AlertService holds a reference to the current channel and delegates sends to it. Switching is a single use(channel) call—no if/else in the alert path. I'll also show a simple "policy selector" so severity or time-of-day can pick the channel dynamically.

Code:

```
// Strategy
interface AlertChannel {
    void send(String msg);
}

// Concrete strategies
class EmailChannel implements AlertChannel {
    @Override public void send(String msg) { System.out.println("[EMAIL] " + msg);
}
}

class SlackChannel implements AlertChannel {
    @Override public void send(String msg) { System.out.println("[SLACK] " + msg);
}
}

class PagerDutyChannel implements AlertChannel {
    @Override public void send(String msg) { System.out.println("[PAGERDUTY] "
+ msg); }
}

// Context
class AlertService {
    private AlertChannel channel;
    public void use(AlertChannel c) { this.channel = c; }
    public void notify(String msg) {
        if (channel == null) throw new IllegalStateException("Alert channel not set");
        channel.send(msg);
    }
}

// Example: simple runtime policy
enum Severity { INFO, WARN, CRITICAL }

class ChannelPolicy {
```

```
    static AlertChannel forSeverity(Severity s) {
        return switch (s) {
            case INFO -> new EmailChannel();
            case WARN -> new SlackChannel();
            case CRITICAL -> new PagerDutyChannel();
        };
    }
}

// Usage
class DemoStrategyAlerts {
    public static void main(String[] args) {
        AlertService svc = new AlertService();

        svc.use(ChannelPolicy.forSeverity(Severity.WARN));
        svc.notify("CPU > 90% on api-1");

        svc.use(ChannelPolicy.forSeverity(Severity.CRITICAL));
        svc.notify("Database unavailable");
    }
}
```

Reasoning to interviewer:

- Strategy keeps AlertService oblivious to transport details; adding Teams/SMS is a new class, not a refactor.
- Policies decide “which channel when” separately from “how to send,” making experiments safe.
- Each channel can hide setup/retries/rate-limiting internally without changing the caller.
- Testing is straightforward: assert that the correct strategy is chosen and invoked for a given severity.

Future scope:

- Multi-channel fan-out (send to Slack + PagerDuty for CRITICAL) via a composite channel.
- Backoff/retry and dead-letter queues inside channels for reliability.
- Per-team configuration loaded at runtime (YAML/DB) to pick channels without redeploys.
- Add structured payloads and templating so messages render appropriately per channel.

Scenario 36: Factory Method — drone delivery fleet

A startup operates drones of different classes. Micro drones handle lightweight urban deliveries, while HeavyLift drones handle bulky shipments from warehouses. Each depot (urban vs warehouse hub) should decide which drone type to dispatch.

Answer:

I'll define a Drone product with a dispatch(parcelId) method. The abstract DispatchCenter defines send(parcelId) but delegates actual drone creation to the factory method createDrone(). Each concrete depot overrides createDrone() to supply the right type of drone. This way, the workflow (send) is stable, but drone choice varies by subclass.

Code:

```
// Product
interface Drone {
    void dispatch(String parcelId);
}

// Concrete products
class MicroDrone implements Drone {
    @Override public void dispatch(String parcelId) {
        System.out.println("MicroDrone dispatched with " + parcelId + " (light
parcel)");
    }
}

class HeavyLiftDrone implements Drone {
    @Override public void dispatch(String parcelId) {
        System.out.println("HeavyLiftDrone dispatched with " + parcelId + " (bulky
parcel)");
    }
}

// Creator
abstract class DispatchCenter {
    public abstract Drone createDrone();
    public void send(String parcelId) {
        // shared dispatch workflow
        System.out.println("Preparing dispatch docs for " + parcelId);
        Drone d = createDrone();
        d.dispatch(parcelId);
        System.out.println("Tracking initiated for " + parcelId);
    }
}

// Concrete creators
class UrbanCenter extends DispatchCenter {
```

```
    @Override public Drone createDrone() { return new MicroDrone(); }
}

class WarehouseHub extends DispatchCenter {
    @Override public Drone createDrone() { return new HeavyLiftDrone(); }
}

// Usage
class DemoDroneFactoryMethod {
    public static void main(String[] args) {
        DispatchCenter cityDepot = new UrbanCenter();
        cityDepot.send("PKG-001");

        DispatchCenter hubDepot = new WarehouseHub();
        hubDepot.send("PKG-XL-9001");
    }
}
```

Reasoning to interviewer:

- Factory Method (createDrone()) lets each depot decide which drone to use, while send() provides a consistent dispatch pipeline.
- Avoids if/else scattered across the workflow.
- Adding a new depot type (e.g., CoastalHub with amphibious drones) just means adding a new subclass.
- The template-style send() ensures shared pre/post steps (docs, tracking) are enforced across all depots.

Future scope:

- Introduce drone capacity checks (parcel.weight) before choosing.
- Load balancing: depots may pool multiple drone types and select based on queue length.
- Integrate telemetry into send() (battery status, route planning).
- Move createDrone() to use config/registry if drone types grow beyond a few.

Scenario 37: Factory Method — data ingestion processors

A data platform must ingest files of multiple formats (YAML, Parquet, TSV). Each has its own parsing and validation rules. We want the ingestion job flow consistent (run → parse → validate → load), but allow the processor to vary by file type.

Answer:

I'll define an Ingestor product with process(path). The abstract IngestionJob defines the workflow run(path) but delegates creation of the ingestor to the factory method createIngestor(). Each job subclass overrides this method to return the proper parser/validator.

Code:

```
// Product
interface Ingestor {
    void process(String path);
}

// Concrete products
class YamlIngestor implements Ingestor {
    @Override public void process(String path) {
        System.out.println("Parse YAML, validate schema, load: " + path);
    }
}
class ParquetIngestor implements Ingestor {
    @Override public void process(String path) {
        System.out.println("Read Parquet metadata, validate columns, load: " + path);
    }
}
class TsvIngestor implements Ingestor {
    @Override public void process(String path) {
        System.out.println("Split TSV rows, validate delimiters, load: " + path);
    }
}

// Creator
abstract class IngestionJob {
    public abstract Ingestor createIngestor();
    public void run(String path) {
        System.out.println("Starting ingestion job for " + path);
        Ingestor ing = createIngestor();
        ing.process(path);
        System.out.println("Job completed for " + path);
    }
}

// Concrete creators
class YamlJob extends IngestionJob {
```

```
    @Override public Ingestor createIngestor() { return new YamlIngestor(); }
}
class ParquetJob extends IngestionJob {
    @Override public Ingestor createIngestor() { return new ParquetIngestor(); }
}
class TsvJob extends IngestionJob {
    @Override public Ingestor createIngestor() { return new TsvIngestor(); }
}

// Usage
class DemoIngestionFactoryMethod {
    public static void main(String[] args) {
        IngestionJob job1 = new YamlJob();
        job1.run("/configs/app.yaml");

        IngestionJob job2 = new ParquetJob();
        job2.run("s3://bucket/data.snappy.parquet");

        IngestionJob job3 = new TsvJob();
        job3.run("/tmp/data.tsv");
    }
}
```

Reasoning to interviewer:

- The workflow (run) is fixed across formats: log start, process with a format-specific ingestor, log complete.
- The Factory Method (createIngestor()) is the hook that subclasses override to inject the right parser.
- Adding JSON or Avro is just a new Ingestor class + a matching job class.
- Keeps ingestion logic extendable and avoids cluttered switch statements.

Future scope:

- Centralize registration: IngestionJobFactory.get("YAML") → new YamlJob() for dynamic selection.
- Support multiple stages (parse, enrich, validate) as a chain of strategies.
- Add error handling and retries into the template method run().
- Metrics and tracing automatically collected in run(), applied uniformly across formats.

Scenario 38: Decorator — email composer add-ons

An email system should allow optional features like encryption, read-receipts, and signatures without bloating the core class. Users may stack these features in any order, depending on preferences.

Answer:

I'll define a Mail component with a simple send(). BasicMail is the default implementation. Each add-on (Encryption, ReadReceipt, Signature) is a decorator that wraps another Mail object and extends its behavior. Because decorators implement the same interface, they can be chained in any combination at runtime.

Code:

```
// Component
interface Mail {
    String send();
}

// Concrete component
class BasicMail implements Mail {
    @Override public String send() { return "Mail sent."; }
}

// Decorator base
abstract class MailAddon implements Mail {
    protected final Mail inner;
    protected MailAddon(Mail inner) { this.inner = inner; }
}

// Concrete decorators
class Encrypted extends MailAddon {
    public Encrypted(Mail inner) { super(inner); }
    @Override public String send() { return inner.send() + " [Encrypted]"; }
}

class ReadReceipt extends MailAddon {
    public ReadReceipt(Mail inner) { super(inner); }
    @Override public String send() { return inner.send() + " [Read-Receipt
Requested]"; }
}

class Signature extends MailAddon {
    public Signature(Mail inner) { super(inner); }
    @Override public String send() { return inner.send() + " [Signed: Rajat G.]"; }
}

// Usage
```

```
class DemoDecoratorMail {
    public static void main(String[] args) {
        Mail mail = new Signature(new ReadReceipt(new Encrypted(new
BasicMail())));
        System.out.println(mail.send());
        // Output: Mail sent. [Encrypted] [Read-Receipt Requested] [Signed: Rajat G.]
    }
}
```

Reasoning to interviewer:

- Decorator lets us extend behavior dynamically, without modifying BasicMail.
- Features are independent: we can apply only encryption, or encryption + signature, etc.
- No class explosion like EncryptedReadReceiptSignatureMail. Instead, we compose features at runtime.
- This design follows open/closed principle: adding a new feature means a new decorator, not modifying old code.

Future scope:

- Add advanced add-ons: scheduled send, priority marking, or auto-translation.
- Enable toggling decorators on/off at runtime using a configuration manager.
- Support introspection: e.g., list which add-ons are active for a given Mail object.
- Integrate external services like digital certificates or delivery tracking as new decorators.

Scenario 39: Strategy — MOBA hero roles

In a MOBA game, heroes may change playstyles mid-match: Carry, Tank, or Support. Each role defines a different action set, but the hero must swap roles on the fly depending on tactics.

Answer:

I'll define a RoleBehavior strategy with an act() method. Each role (Carry, Tank, Support) provides its own behavior. The Hero context holds the current role strategy and delegates to it. At runtime, swapping the strategy changes the hero's playstyle without altering the hero class.

Code:

```
// Strategy
interface RoleBehavior {
    void act();
}

// Concrete strategies
class CarryBehavior implements RoleBehavior {
    @Override public void act() { System.out.println("Farm fast, deal burst
damage."); }
}
class TankBehavior implements RoleBehavior {
    @Override public void act() { System.out.println("Absorb damage, initiate
fights."); }
}
class SupportBehavior implements RoleBehavior {
    @Override public void act() { System.out.println("Ward map, heal and buff
allies."); }
}

// Context
class Hero {
    private RoleBehavior behavior;
    public void setRole(RoleBehavior b) { this.behavior = b; }
    public void perform() {
        if (behavior == null) throw new IllegalStateException("Role not set");
        behavior.act();
    }
}

// Usage
class DemoStrategyHero {
    public static void main(String[] args) {
        Hero phoenix = new Hero();

        phoenix.setRole(new SupportBehavior());
```

```
        phoenix.perform(); // Support role

        phoenix.setRole(new CarryBehavior());
        phoenix.perform(); // Carry role

        phoenix.setRole(new TankBehavior());
        phoenix.perform(); // Tank role
    }
}
```

Reasoning to interviewer:

- Strategy makes each role modular, so adding a new role (Assassin, Jungler) doesn't touch Hero.
- Hero delegates execution to the current strategy, avoiding conditionals like if(role==carry)….
- Roles are swappable at runtime, which models real gameplay decisions mid-match.
- Each role's behavior is isolated, so testing and balancing are easier.

Future scope:

- Add composite strategies: hybrid roles (e.g., Tank-Support).
- Integrate state (hero's HP, gold, cooldowns) into strategies to influence actions.
- Support AI-driven dynamic switching (auto role swap if the team lacks a tank).
- Extend RoleBehavior with additional methods like specialAbility() for richer gameplay.

Scenario 40: Command — text editor actions with undo

In a text editor, user operations (Insert, Delete, Bold) should be modeled as commands so we can apply them, keep history, and support undo. This way, every action is encapsulated and reversible.

Answer:

I'll define an EditCommand interface with execute() and undo(). The Document is the receiver that modifies text. Each action (InsertText, DeleteSelection, BoldSelection) is a command implementation, holding the necessary state to undo itself. The Editor is the invoker that runs commands and keeps a history stack for undo.

Code:

```
import java.util.Stack;

// Command
interface EditCommand {
    void execute();
    void undo();
}

// Receiver
class Document {
    private final StringBuilder buf = new StringBuilder();

    public void insert(String s) { buf.append(s); }
    public void deleteLast(int n) {
        buf.delete(Math.max(0, buf.length() - n), buf.length());
    }
    public void boldLast(int n) {
        buf.insert(buf.length() - n, "**");
        buf.append("**");
    }
    public String content() { return buf.toString(); }
}

// Concrete commands
class InsertText implements EditCommand {
    private final Document doc;
    private final String text;
    public InsertText(Document d, String t) { this.doc = d; this.text = t; }
    @Override public void execute() { doc.insert(text); }
    @Override public void undo() { doc.deleteLast(text.length()); }
```

```
}

class DeleteSelection implements EditCommand {
    private final Document doc;
    private final int length;
    private String deleted;
    public DeleteSelection(Document d, int len) { this.doc = d; this.length = len; }
    @Override public void execute() {
        String c = doc.content();
        int start = Math.max(0, c.length() - length);
        deleted = c.substring(start);
        doc.deleteLast(length);
    }
    @Override public void undo() { doc.insert(deleted); }
}

class BoldSelection implements EditCommand {
    private final Document doc;
    private final int length;
    public BoldSelection(Document d, int len) { this.doc = d; this.length = len; }
    @Override public void execute() { doc.boldLast(length); }
    @Override public void undo() {
        // naive undo: strip last "**" pairs
        String c = doc.content();
        doc.deleteLast(2); // remove trailing **
        doc.deleteLast(length);
        doc.insert(c.substring(c.length() - length - 2, c.length() - 2));
    }
}

// Invoker
class Editor {
    private final Stack<EditCommand> history = new Stack<>();
    public void run(EditCommand c) { c.execute(); history.push(c); }
    public void undo() { if (!history.isEmpty()) history.pop().undo(); }
}

// Usage
class DemoCommandEditor {
    public static void main(String[] args) {
        Document doc = new Document();
        Editor editor = new Editor();

        editor.run(new InsertText(doc, "Hello "));
        editor.run(new InsertText(doc, "World"));
        System.out.println(doc.content()); // Hello World

        editor.undo(); // undo last insert
        System.out.println(doc.content()); // Hello
```

```
        editor.run(new BoldSelection(doc, 5));
        System.out.println(doc.content()); // Hello ** (bold applied)
    }
}
```

Reasoning to interviewer:

- Each command encapsulates action + undo data (Insert remembers inserted text, Delete stores deleted text).
- The editor doesn’t know about insert/delete/bold internals—it just runs and undoes commands.
- The history stack enables multiple undo levels.
- Adding new actions (Italic, Replace, CopyPaste) only requires new command classes.

Future scope:

- Add a redo stack for redo functionality.
- Use a Caret/Selection object to support arbitrary ranges, not just “last N chars.”
- Persist command logs for collaborative editing (like Google Docs).
- Optimize undo() for performance by using a Memento or snapshot for large documents.

Scenario 41: Singleton — thread-safe cache manager in an e-commerce app

We need a cache that's globally accessible (product details, price lists, inventory snapshots), with exactly one instance per JVM so TTLs, eviction, and memory usage stay consistent across threads.

Answer:

I'll design CacheManager as a Singleton using the initialization-on-demand holder idiom (lazy, thread-safe without synchronization on access). Internally it uses a ConcurrentHashMap and optional TTL support. All threads call CacheManager.getInstance() and work with the same store.

Code:

```
import java.time.Duration;
import java.time.Instant;
import java.util.Objects;
import java.util.Optional;
import java.util.concurrent.*;

public final class CacheManager {
    // value wrapper with TTL
    private static final class Entry {
        final Object value;
        final Instant expiresAt; // null means no TTL
        Entry(Object value, Instant expiresAt) { this.value = value; this.expiresAt =
expiresAt; }
        boolean expired() { return expiresAt != null &&
Instant.now().isAfter(expiresAt); }
    }

    private final ConcurrentHashMap<String, Entry> store = new
ConcurrentHashMap<>();
    private final ScheduledExecutorService janitor =
Executors.newSingleThreadScheduledExecutor(r -> {
        Thread t = new Thread(r, "cache-janitor"); t.setDaemon(true); return t;
    });

    private CacheManager() {
        // periodic eviction of expired keys
        janitor.scheduleAtFixedRate(this::cleanUp, 1, 1, TimeUnit.MINUTES);
    }

    private void cleanUp() {
        for (var e : store.entrySet()) {
            if (e.getValue().expired()) store.remove(e.getKey(), e.getValue());
        }
    }
```

```
    // Holder idiom: lazy, thread-safe singleton
    private static class Holder {
        private static final CacheManager INSTANCE = new CacheManager();
    }

    public static CacheManager getInstance() {
        return Holder.INSTANCE;
    }

    // API
    public void put(String key, Object value) {
        store.put(Objects.requireNonNull(key), new Entry(value, null));
    }

    public void put(String key, Object value, Duration ttl) {
        Instant expires = ttl == null ? null : Instant.now().plus(ttl);
        store.put(Objects.requireNonNull(key), new Entry(value, expires));
    }

    public Optional<Object> get(String key) {
        Entry e = store.get(key);
        if (e == null) return Optional.empty();
        if (e.expired()) { store.remove(key, e); return Optional.empty(); }
        return Optional.ofNullable(e.value);
    }

    public void evict(String key) { store.remove(key); }

    public void clear() { store.clear(); }

    public int size() { return store.size(); }
}

// Usage
class DemoSingletonCache {
    public static void main(String[] args) {
        CacheManager c1 = CacheManager.getInstance();
        CacheManager c2 = CacheManager.getInstance();
        System.out.println(c1 == c2); // true

        c1.put("sku:IPH14", "iPhone 14");
        System.out.println(c2.get("sku:IPH14").orElse("miss"));
    }
}
```

Reasoning to interviewer:

- Holder idiom leverages class loading guarantees for thread-safe lazy init; no double-checked locking ceremony needed.
- ConcurrentHashMap plus a light janitor thread gives non-blocking reads/writes and periodic TTL cleanup.
- APIs are simple: put/get/evict/clear; TTL is optional, which fits mixed workloads (frequently read catalog vs. short-lived price quotes).
- One instance ensures consistent eviction policy and memory accounting across the entire app.

Future scope:

- Pluggable eviction (LRU/size cap) and metrics (hit/miss, evictions) exported via JMX/Micrometer.
- Namespaces per domain (pricing, inventory) and size quotas per namespace.
- Async refresh (“stale-while-revalidate”) to hide latency spikes.
- Serialization guards for large objects and off-heap/Redis backing for very large caches.

Scenario 42: Factory Method — ride-hailing booking (Bike/Auto/Car)

Each ride type has unique pricing and assignment rules: Bike is cheapest and assigned to nearby two-wheelers, Auto has fixed slabs, Car uses surge and rider rating gates. The booking flow remains the same, only the “which ride” details differ.

Answer:

I’ll keep a base creator Booking that defines the workflow (createRide() → quote() → assign() → confirm()), and let concrete creators (BikeBooking, AutoBooking, CarBooking) override the factory method to return the correct Ride. Each Ride encapsulates type-specific pricing/assignment logic.

Code:

```
import java.util.Random;

// Product
interface Ride {
    String type();
    double quote(double km);
    String assignDriver(); // could return driver id
}

// Concrete products
class BikeRide implements Ride {
    public String type() { return "BIKE"; }
    public double quote(double km) { return Math.max(35, 10 * km); }
    public String assignDriver() { return "bike-" + new Random().nextInt(10_000); }
}

class AutoRide implements Ride {
    public String type() { return "AUTO"; }
    public double quote(double km) {
        // base slab + per-km
        return 40 + 12 * km;
    }
    public String assignDriver() { return "auto-" + new Random().nextInt(10_000); }
}

class CarRide implements Ride {
    public String type() { return "CAR"; }
    public double quote(double km) {
        double base = 60 + 15 * km;
        double surge = (km > 8 ? 1.15 : 1.0); // naive surge
        return Math.round(base * surge);
    }
    public String assignDriver() { return "car-" + new Random().nextInt(10_000); }
}
```

```
// Creator
abstract class Booking {
    // Factory Method
    protected abstract Ride createRide();

    // Shared workflow
    public final void book(double km) {
        Ride ride = createRide();
        double fare = ride.quote(km);
        String driver = ride.assignDriver();
        confirm(ride.type(), fare, driver);
    }

    protected void confirm(String type, double fare, String driverId) {
        System.out.println("Booked " + type + " | Fare ₹" + fare + " | Driver " +
driverId);
    }
}

// Concrete creators
class BikeBooking extends Booking {
    @Override protected Ride createRide() { return new BikeRide(); }
}

class AutoBooking extends Booking {
    @Override protected Ride createRide() { return new AutoRide(); }
}

class CarBooking extends Booking {
    @Override protected Ride createRide() { return new CarRide(); }
}

// Usage
class DemoFactoryMethodRides {
    public static void main(String[] args) {
        Booking b1 = new BikeBooking();
        b1.book(4.2);

        Booking b2 = new AutoBooking();
        b2.book(7.0);

        Booking b3 = new CarBooking();
        b3.book(12.5);
    }
}
```

Reasoning to interviewer:

- Factory Method centralizes “how to build a booking” while letting subclasses decide the concrete Ride.
- Pricing and assignment stay close to the ride type, keeping the workflow class clean and open for extension.
- Swapping ride type is just swapping the concrete creator; the booking pipeline (quote → assign → confirm) is identical.
- Easy to test each ride’s pricing independently and verify the common flow once.

Future scope:

- Move surge, driver eligibility, and cancellation rules into each Ride or policy objects.
- Add PremiumCarBooking, EVBikeBooking without touching existing creators.
- Introduce a selection layer that picks the Booking creator at runtime based on ETA, price caps, or user preference.
- Instrument the base Booking with metrics/retries so all ride types inherit resilience.

Scenario 43: Abstract Factory — cross-platform UI toolkit

Different platforms (Windows, macOS, Linux) need their own look-and-feel for buttons and text fields. I want one place to choose the platform, and everywhere else the app just asks “give me a button and a text field” without caring about which OS we’re on.

Answer:

I’ll define product interfaces (Button, TextField) and an abstract factory UIFactory that can create the family of widgets. Each platform has a concrete factory (Windows/macOS/Linux) returning the matching implementations. The client receives a UIFactory (e.g., from environment detection or config) and builds screens purely against interfaces. Swapping platforms is just swapping the factory.

Code:

```
// Products
interface Button { void render(); }
interface TextField { void render(); }

// Concrete products: Windows
class WinButton implements Button { public void render() {
System.out.println("[Win] Button"); } }
class WinTextField implements TextField { public void render() {
System.out.println("[Win] TextField"); } }

// Concrete products: macOS
class MacButton implements Button { public void render() {
System.out.println("[Mac] Button"); } }
class MacTextField implements TextField { public void render() {
System.out.println("[Mac] TextField"); } }

// Concrete products: Linux
class LinuxButton implements Button { public void render() {
System.out.println("[Linux] Button"); } }
class LinuxTextField implements TextField { public void render() {
System.out.println("[Linux] TextField"); } }

// Abstract factory
interface UIFactory {
    Button createButton();
    TextField createTextField();
}

// Concrete factories
class WindowsFactory implements UIFactory {
    public Button createButton() { return new WinButton(); }
    public TextField createTextField() { return new WinTextField(); }
}
```

```
class MacFactory implements UIFactory {
    public Button createButton() { return new MacButton(); }
    public TextField createTextField() { return new MacTextField(); }
}
class LinuxFactory implements UIFactory {
    public Button createButton() { return new LinuxButton(); }
    public TextField createTextField() { return new LinuxTextField(); }
}

// Client code uses only interfaces
class LoginScreen {
    private final Button loginBtn;
    private final TextField userField;

    public LoginScreen(UIFactory f) {
        this.loginBtn = f.createButton();
        this.userField = f.createTextField();
    }
    public void draw() {
        userField.render();
        loginBtn.render();
    }
}

// Usage
class DemoUI {
    public static void main(String[] args) {
        UIFactory f = chooseFactory(System.getProperty("os.name"));
        new LoginScreen(f).draw();
    }

    static UIFactory chooseFactory(String os) {
        os = os == null ? "" : os.toLowerCase();
        if (os.contains("mac"))   return new MacFactory();
        if (os.contains("win"))   return new WindowsFactory();
        return new LinuxFactory();
    }
}
```

Reasoning to interviewer:

- Abstract Factory groups related products so you always get a consistent family (Windows button + Windows text field).
- Client code never names concrete classes; it depends only on interfaces, so platform swapping is trivial.
- New platforms (e.g., “Web”) add a factory and product classes, without touching screen code.

- This keeps theme/look-and-feel concerns out of business logic.

Future scope:

- Extend the family: add Checkbox, Menu, etc., to the same UIFactory.
- Support runtime theming: compose factories or decorate products for “High Contrast” or “Touch-First” modes.
- Cache or pool expensive widgets if the underlying toolkit requires it.
- Plug into dependency injection so platform choice comes from configuration.

Scenario 44: Builder — flexible resume construction

Users may include only some sections (Education, Work, Projects, Skills). We want a readable API that builds an immutable resume with exactly the sections provided—no giant constructor with nulls.

Answer:

I'll make Resume immutable and expose a nested Builder. Required bits (like name/contact) go into the builder's constructor; optional sections are added via fluent methods that accept lists or single entries. The builder validates data and assembles a final Resume object that's safe to pass around or serialize.

Code:

```
import java.util.*;

// Domain models (simple for demo)
record Education(String school, String degree, String year) {}
record Work(String company, String role, String duration) {}
record Project(String title, String summary) {}

final class Resume {
    private final String fullName;
    private final String email;
    private final List<Education> education;
    private final List<Work> work;
    private final List<Project> projects;
    private final List<String> skills;

    private Resume(Builder b) {
        this.fullName = b.fullName;
        this.email = b.email;
        this.education = List.copyOf(b.education);
        this.work = List.copyOf(b.work);
        this.projects = List.copyOf(b.projects);
        this.skills = List.copyOf(b.skills);
    }

    @Override public String toString() {
        return "Resume[" + fullName + ", " + email +
                ", edu=" + education.size() +
                ", work=" + work.size() +
                ", projects=" + projects.size() +
                ", skills=" + skills + "]";
    }

    public static class Builder {
        private final String fullName;
        private final String email;
```

```
        private final List<Education> education = new ArrayList<>();
        private final List<Work> work = new ArrayList<>();
        private final List<Project> projects = new ArrayList<>();
        private final List<String> skills = new ArrayList<>();

        public Builder(String fullName, String email) {
            this.fullName = requireNonBlank(fullName, "fullName");
            this.email = requireNonBlank(email, "email");
        }

        // Section adders (fluent)
        public Builder addEducation(Education e) {
education.add(Objects.requireNonNull(e)); return this; }
        public Builder addWork(Work w)          { work.add(Objects.requireNonNull(w));
return this; }
        public Builder addProject(Project p)    {
projects.add(Objects.requireNonNull(p)); return this; }
        public Builder addSkill(String s)       { skills.add(requireNonBlank(s, "skill"));
return this; }

        // Bulk setters (optional)
        public Builder education(List<Education> list) { education.addAll(list); return
this; }
        public Builder work(List<Work> list)           { work.addAll(list); return this; }
        public Builder projects(List<Project> list)    { projects.addAll(list); return this; }
        public Builder skills(List<String> list)       { list.forEach(this::addSkill); return
this; }

        public Resume build() {
            // minimal validation example
            if (skills.isEmpty() && projects.isEmpty() && work.isEmpty() &&
education.isEmpty()) {
                throw new IllegalStateException("At least one section should be
provided");
            }
            return new Resume(this);
        }

        private static String requireNonBlank(String v, String field) {
            Objects.requireNonNull(v, field + " required");
            String t = v.trim();
            if (t.isEmpty()) throw new IllegalArgumentException(field + " required");
            return t;
        }
    }
}

// Usage
class DemoResumeBuilder {
    public static void main(String[] args) {
```

```
        Resume r = new Resume.Builder("Rajat Gupta", "rajat@example.com")
                .addEducation(new Education("IIT", "B.Tech CS", "2020"))
                .addWork(new Work("Acme Corp", "SDE II", "2021–2024"))
                .addProject(new Project("Billing Service", "Reduced latency by 40%"))
                .addSkill("Java")
                .addSkill("Spring Boot")
                .build();

        System.out.println(r);
    }
}
```

Reasoning to interviewer:

- Builder separates required identity (name/email) from optional sections—callers add only what they need.
- The final object is immutable (List.copyOf, no setters), which is safe for caching and sharing.
- The API reads like the resume itself; easy to wire from a UI form without juggling nulls.
- Validation sits near construction (fail fast), keeping downstream code simpler.

Future scope:

- Add formatting/export (PDF/Markdown) without changing the model—use a separate exporter.
- Provide toBuilder() for editing an existing resume in a multi-step wizard.
- Add localized labels and a template system for different countries.
- Include computed sections (e.g., keyword density, ATS scores) during build for analytics.

Scenario 45: Decorator — dynamic pizza toppings

A customer starts with a base pizza and can stack toppings (Cheese, Olives, Jalapeños) in any order. We don't want subclasses for every combo; we want to compose features at runtime.

Answer:

I'll define a Pizza component with description() and cost(). BasePizza (e.g., Margherita) is the concrete component. Each topping is a decorator that wraps a Pizza, augments description, and adjusts cost. Because decorators implement Pizza, they can be combined freely.

Code:

```
// Component
interface Pizza {
    String description();
    double cost();
}

// Concrete component
class BasePizza implements Pizza {
    @Override public String description() { return "Margherita"; }
    @Override public double cost() { return 199.0; } // base price
}

// Decorator base
abstract class Topping implements Pizza {
    protected final Pizza inner;
    protected Topping(Pizza inner) { this.inner = inner; }
}

// Concrete decorators
class Cheese extends Topping {
    public Cheese(Pizza inner) { super(inner); }
    @Override public String description() { return inner.description() + ", Cheese"; }
    @Override public double cost() { return inner.cost() + 40.0; }
}

class Olives extends Topping {
    public Olives(Pizza inner) { super(inner); }
    @Override public String description() { return inner.description() + ", Olives"; }
    @Override public double cost() { return inner.cost() + 25.0; }
}

class Jalapenos extends Topping {
    public Jalapenos(Pizza inner) { super(inner); }
```

```
    @Override public String description() { return inner.description() + ", Jalapeños";
}
    @Override public double cost() { return inner.cost() + 30.0; }
}

// Usage
class DemoPizzaDecorator {
    public static void main(String[] args) {
        Pizza order = new Jalapenos(new Cheese(new Olives(new BasePizza())));
        System.out.println(order.description()); // Margherita, Olives, Cheese,
Jalapeños
        System.out.println(order.cost());        // 199 + 25 + 40 + 30 = 294.0
    }
}
```

Reasoning to interviewer:

- Decorator composes behavior at runtime; no subclass explosion like CheeseOlivesJalapenosPizza.
- Each topping owns its price logic and text, so changes are localized.
- Callers treat every result as a Pizza, enabling seamless stacking and pricing.
- Order of decorators naturally affects description() and can influence cost rules if needed.

Future scope:

- Add size multipliers (Small/Medium/Large) as decorators, or a separate pricing strategy.
- Introduce constraints (e.g., “double cheese” allowed, “no olives with jalapeños” rule) via a validator before checkout.
- Memoize cost()/description() for repeated access.
- Add nutritional info or allergen flags in decorators for health labeling.

Scenario 46: Adapter — unify payment gateways (PayPal, Stripe, Razorpay)

Gateways expose different SDK shapes (method names, params, auth). Your app should talk to one PaymentProcessor interface and plug in an adapter per provider.

Answer:

I'll define a PaymentProcessor target interface. For every provider, I'll write an adapter that wraps the vendor SDK and translates our unified call (charge(amount, currency, meta)) into the provider-specific API (param mapping, key formats, id handling). Selection happens via config/factory; callers never touch vendor classes.

Code:

```
import java.util.Map;
import java.util.Objects;

// Target
interface PaymentProcessor {
    String charge(long amountInMinor, String currency, Map<String, String>
metadata);
}

// Adaptees (simulated vendor SDKs)
class PayPalSDK {
    public String createPayment(double amountMajor, String curr) {
        System.out.println("PayPal charge " + amountMajor + " " + curr);
        return "PP-" + System.nanoTime();
    }
}
class StripeSDK {
    public String payCents(long cents, String iso, Map<String, String> meta) {
        System.out.println("Stripe charge " + cents + " " + iso + " meta=" + meta);
        return "ST-" + System.nanoTime();
    }
}
class RazorpaySDK {
    public String orderAndCapture(int paise, String iso3) {
        System.out.println("Razorpay charge " + paise + " " + iso3);
        return "RZP-" + System.nanoTime();
    }
}

// Adapters
class PayPalAdapter implements PaymentProcessor {
    private final PayPalSDK sdk = new PayPalSDK();
    @Override public String charge(long amountInMinor, String currency,
Map<String, String> metadata) {
        double major = amountInMinor / 100.0; // minor→major (e.g., cents→dollars)
```

```
        return sdk.createPayment(major, currency);
    }
}

class StripeAdapter implements PaymentProcessor {
    private final StripeSDK sdk = new StripeSDK();
    @Override public String charge(long amountInMinor, String currency,
Map<String, String> metadata) {
        return sdk.payCents(amountInMinor, currency, metadata);
    }
}

class RazorpayAdapter implements PaymentProcessor {
    private final RazorpaySDK sdk = new RazorpaySDK();
    @Override public String charge(long amountInMinor, String currency,
Map<String, String> metadata) {
        // Assume INR paise / ISO code pass-through for demo
        return sdk.orderAndCapture((int) amountInMinor, currency);
    }
}

// Selector / factory for runtime choice
enum Gateway { PAYPAL, STRIPE, RAZORPAY }

class PaymentFactory {
    static PaymentProcessor of(Gateway g) {
        return switch (g) {
            case PAYPAL   -> new PayPalAdapter();
            case STRIPE   -> new StripeAdapter();
            case RAZORPAY -> new RazorpayAdapter();
        };
    }
}

// Usage
class DemoPaymentAdapter {
    public static void main(String[] args) {
        PaymentProcessor processor = PaymentFactory.of(Gateway.STRIPE);
        String id = processor.charge(49900, "USD", Map.of("orderId", "ORD-123"));
        System.out.println("Charge id: " + id);

        processor = PaymentFactory.of(Gateway.RAZORPAY);
        String id2 = processor.charge(129900, "INR", Map.of());
        System.out.println("Charge id: " + id2);
    }
}
```

Reasoning to interviewer:

- The app speaks one interface; adapters handle param units, currency formats, and SDK quirks.
- Swapping gateways is a config change—no business logic rewrites.
- Each adapter is the right place for retries, idempotency keys, or error translation (map vendor errors to domain errors).
- Testing is easy: mock PaymentProcessor for unit tests without real SDKs.

Future scope:

- Add a composite processor for failover (try Stripe, fallback to PayPal).
- Centralize idempotency and signature verification in adapters.
- Config-driven factory (per-tenant gateway) and per-region currency rules.
- Telemetry: emit latency, success rate, and error codes per gateway for SRE dashboards.

Scenario 47: Proxy — lazy-loading large images in a gallery

Thumbnails are cheap and should render immediately, but full-size images are heavy (network + memory). I want a stand-in object that looks like an image to the UI but defers the expensive load until the user actually opens it.

Answer:

I'll define an Image interface with two operations: showThumbnail() (fast) and show() (full image). RealImage does the heavy I/O, while ImageProxy holds just the file/URL and lazily creates/loads the RealImage on first show(). From the UI's perspective, both are Image, so swapping is transparent.

Code:

```
// Subject
interface Image {
    void showThumbnail();
    void show(); // full-size
}

// Real subject (heavy)
class RealImage implements Image {
    private final String url;
    private final byte[] pixels; // simulated buffer

    public RealImage(String url) {
        this.url = url;
        this.pixels = loadFromNetwork(url);
    }

    private byte[] loadFromNetwork(String url) {
        System.out.println("Downloading full image from " + url + " ...");
        // simulate heavy payload
        return new byte[10_000_000];
    }

    @Override public void showThumbnail() {
        System.out.println("Rendering thumbnail (from decoded pixels) for " + url);
    }

    @Override public void show() {
        System.out.println("Rendering FULL image for " + url + " (size=" +
pixels.length + ")");
    }
}

// Proxy (light, defers RealImage creation)
class ImageProxy implements Image {
    private final String url;
```

```
    private RealImage real; // lazily initialized

    public ImageProxy(String url) { this.url = url; }

    @Override public void showThumbnail() {
        // no heavy download: use cached thumbnail/CDN small asset by convention
        System.out.println("Showing CDN thumbnail for " + url);
    }

    @Override public void show() {
        if (real == null) real = new RealImage(url);
        real.show();
    }
}

// Usage
class DemoProxyGallery {
    public static void main(String[] args) {
        Image img = new ImageProxy("https://cdn.app/imgs/sea.jpg");

        // Grid view
        img.showThumbnail(); // instant

        // User clicks to open
        img.show();          // triggers download once
        img.show();          // reuses cached RealImage (no re-download)
    }
}
```

Reasoning to interviewer:

- This is a virtual proxy: it stands in for a heavyweight RealImage and creates it on demand.
- Thumbnails render with zero heavy I/O; the full image loads only on click, improving initial page performance.
- The proxy is a single control point to add caching, memoization, permissions, or retries without touching RealImage.
- Memory stays predictable—unused images remain lightweight proxies.

Future scope:

- Add eviction/soft references for RealImage to prevent memory bloat in long sessions.
- Prefetch a few likely-next images during idle time for perceived speed.
- Track metrics (download time, cache hit rate) inside the proxy.

- Gate show() behind an auth/entitlement check for premium images.

Scenario 48: Strategy — pluggable discount logic in travel booking

Discount rules change frequently (Festival, Loyalty, Student). Checkout code shouldn't know each formula; it should just ask a strategy to compute the discount and be able to swap at runtime.

Answer:

I'll define a DiscountStrategy with apply(subtotal, context). Each rule is a concrete strategy. BookingCart holds a reference to the active strategy and delegates the discount computation. A small DiscountContext carries dynamic data (user tier, student flag, promo window) so strategies stay pure and testable.

Code:

```
import java.time.LocalDate;

// Strategy
interface DiscountStrategy {
    double apply(double subtotal, DiscountContext ctx);
}

// Context for dynamic inputs
record DiscountContext(boolean isLoyal, boolean isStudent, LocalDate today) {}

// Concrete strategies
class FestivalOffer implements DiscountStrategy {
    @Override public double apply(double subtotal, DiscountContext ctx) {
        // 15% off during festival week (demo)
        return round(subtotal * 0.85);
    }
}

class LoyaltyDiscount implements DiscountStrategy {
    @Override public double apply(double subtotal, DiscountContext ctx) {
        double off = ctx.isLoyal() ? 250 : 0;
        return round(Math.max(0, subtotal - off));
    }
}

class StudentDiscount implements DiscountStrategy {
    @Override public double apply(double subtotal, DiscountContext ctx) {
        if (!ctx.isStudent()) return subtotal;
        // 10% capped at ₹500
        double discounted = subtotal * 0.90;
        return round(Math.max(subtotal - 500, discounted));
    }
}

// Cart / Context holder
```

```
class BookingCart {
    private DiscountStrategy strategy;

    public void use(DiscountStrategy s) { this.strategy = s; }

    public double total(double subtotal, DiscountContext ctx) {
        if (strategy == null) throw new IllegalStateException("No discount strategy
set");
        return strategy.apply(subtotal, ctx);
    }

    private static double round(double v) { return Math.round(v * 100.0) / 100.0; }
}

// Optional: simple selector
enum Rule { FESTIVAL, LOYALTY, STUDENT }
class DiscountSelector {
    static DiscountStrategy choose(Rule r) {
        return switch (r) {
            case FESTIVAL -> new FestivalOffer();
            case LOYALTY  -> new LoyaltyDiscount();
            case STUDENT  -> new StudentDiscount();
        };
    }
}

// Usage
class DemoStrategyDiscounts {
    public static void main(String[] args) {
        BookingCart cart = new BookingCart();
        DiscountContext ctx = new DiscountContext(true, true, LocalDate.now());

        cart.use(DiscountSelector.choose(Rule.LOYALTY));
        System.out.println("Loyalty total:  " + cart.total(3200, ctx));

        cart.use(DiscountSelector.choose(Rule.STUDENT));
        System.out.println("Student total:  " + cart.total(3200, ctx));

        cart.use(DiscountSelector.choose(Rule.FESTIVAL));
        System.out.println("Festival total: " + cart.total(3200, ctx));
    }
}
```

Reasoning to interviewer:

- Strategy keeps each discount rule isolated; checkout remains oblivious to formulas and easy to maintain.

- Runtime swapping is a one-liner (use(...)), enabling A/B tests, policy changes, or per-segment rules.
- DiscountContext cleanly carries flags and dates without turning strategies into god objects.
- Each strategy is trivial to unit test with boundary cases (caps, floors).

Future scope:

- Compose multiple strategies (e.g., best-of festival vs loyalty) via a composite or rule engine.
- Externalize rates and caps to configuration; hot-reload without redeploys.
- Add explainability (breakdown() returning line items) for invoices and customer support.
- Implement eligibility guards so ineligible users fall back to a default “no discount” strategy.

Scenario 49: Observer — stock portfolio subscriptions

Users subscribe to specific tickers (Apple, Tesla). When a ticker's price changes, only its subscribers should be notified, and they can subscribe/unsubscribe anytime.

Answer:

I'll model each ticker as a "subject" with a subscriber list. A Subscriber interface represents any UI/client wanting updates. Ticker stores the current price and, on setPrice, notifies a snapshot of its subscribers. A simple MarketHub will manage many tickers so callers don't reimplement subject logic per symbol.

Code:

```
import java.util.*;

// Observer
interface Subscriber {
    void onPrice(String symbol, double price);
}

// Concrete observer
class UserClient implements Subscriber {
    private final String name;
    public UserClient(String name) { this.name = name; }
    @Override public void onPrice(String symbol, double price) {
        System.out.println(name + " notified: " + symbol + " = " + price);
    }
}

// Subject
interface Ticker {
    String symbol();
    void subscribe(Subscriber s);
    void unsubscribe(Subscriber s);
    void setPrice(double price);
    double getPrice();
}

class StockTicker implements Ticker {
    private final String symbol;
    private final List<Subscriber> subs = new ArrayList<>();
    private double price;

    public StockTicker(String symbol) { this.symbol = symbol; }
    @Override public String symbol() { return symbol; }
    @Override public double getPrice() { return price; }

    @Override public void subscribe(Subscriber s) {
subs.add(Objects.requireNonNull(s)); }
```

```
    @Override public void unsubscribe(Subscriber s) { subs.remove(s); }

    @Override public void setPrice(double price) {
        this.price = price;
        // notify over a snapshot to avoid ConcurrentModification during callbacks
        for (Subscriber s : List.copyOf(subs)) s.onPrice(symbol, price);
    }
}

// Coordinator for many tickers
class MarketHub {
    private final Map<String, Ticker> tickers = new HashMap<>();
    public Ticker ticker(String symbol) {
        return tickers.computeIfAbsent(symbol.toUpperCase(Locale.ROOT),
StockTicker::new);
    }
    public void update(String symbol, double price) {
        ticker(symbol).setPrice(price);
    }
}

// Usage
class DemoObserverStocks {
    public static void main(String[] args) {
        MarketHub hub = new MarketHub();
        Ticker aapl = hub.ticker("AAPL");
        Ticker tsla = hub.ticker("TSLA");

        Subscriber rita = new UserClient("Rita");
        Subscriber dev  = new UserClient("Dev");

        aapl.subscribe(rita);
        tsla.subscribe(rita);
        tsla.subscribe(dev);

        hub.update("AAPL", 232.15); // Rita
        hub.update("TSLA", 258.40); // Rita + Dev

        tsla.unsubscribe(rita);
        hub.update("TSLA", 261.05); // Dev only
    }
}
```

Reasoning to interviewer:

- Subject/observer cleanly decouples the data producer (ticker) from consumers (clients, webhooks, dashboards).

- Per-ticker subjects ensure fan-out is scoped correctly (AAPL updates don’t hit TSLA subscribers).
- Snapshot iteration avoids concurrent modification if a callback unsubscribes.
- MarketHub centralizes ticker creation and symbol normalization, keeping call sites simple.

Future scope:

- Make notifications asynchronous via an event queue so a slow client doesn’t block price updates.
- Add per-subscriber filters (only alert above/below thresholds, percentage moves).
- Batch updates (coalesce multiple ticks per second) for high-volume markets.
- Persist subscriptions so they survive restarts and support multi-device login.

Scenario 50: Command — drawing actions with undo

A drawing app needs Draw Line, Draw Circle, and Erase Shape. Every action should be an object that knows how to do and undo itself; the editor keeps an undo stack.

Answer:

I'll define a DrawCommand with execute() and undo(). The Canvas is the receiver that stores shapes. Each command captures the state it needs to reverse itself (e.g., the shape id it added, or the erased shape snapshot). The Editor (invoker) runs commands and maintains a history stack.

Code:

```
import java.util.*;
import java.util.concurrent.atomic.AtomicLong;

// Receiver & model
interface Shape { long id(); String describe(); }

class Line implements Shape {
    private final long id; private final int x1,y1,x2,y2;
    Line(long id, int x1, int y1, int x2, int y2) { this.id=id; this.x1=x1; this.y1=y1;
this.x2=x2; this.y2=y2; }
    public long id() { return id; }
    public String describe() { return "Line#" + id + "
("+x1+","+y1+")→("+x2+","+y2+")"; }
}

class Circle implements Shape {
    private final long id; private final int cx,cy,r;
    Circle(long id, int cx, int cy, int r) { this.id=id; this.cx=cx; this.cy=cy; this.r=r; }
    public long id() { return id; }
    public String describe() { return "Circle#" + id + " center("+cx+","+cy+") r="+r; }
}

class Canvas {
    private final Map<Long, Shape> shapes = new LinkedHashMap<>();
    private final AtomicLong seq = new AtomicLong(1);

    public long addShape(Shape s) { shapes.put(s.id(), s); return s.id(); }
    public Shape remove(long id) { return shapes.remove(id); }
    public long nextId() { return seq.getAndIncrement(); }

    public void render() {
        System.out.println("Canvas:");
        shapes.values().forEach(s -> System.out.println("  " + s.describe()));
    }
}
```

```
// Command
interface DrawCommand { void execute(); void undo(); }

// Concrete commands
class DrawLine implements DrawCommand {
    private final Canvas canvas;
    private final int x1,y1,x2,y2;
    private Long id; // assigned on execute

    public DrawLine(Canvas c, int x1, int y1, int x2, int y2) {
        this.canvas=c; this.x1=x1; this.y1=y1; this.x2=x2; this.y2=y2;
    }
    public void execute() {
        id = canvas.nextId();
        canvas.addShape(new Line(id, x1, y1, x2, y2));
    }
    public void undo() { if (id != null) canvas.remove(id); }
}

class DrawCircle implements DrawCommand {
    private final Canvas canvas;
    private final int cx,cy,r;
    private Long id;

    public DrawCircle(Canvas c, int cx, int cy, int r) {
        this.canvas=c; this.cx=cx; this.cy=cy; this.r=r;
    }
    public void execute() {
        id = canvas.nextId();
        canvas.addShape(new Circle(id, cx, cy, r));
    }
    public void undo() { if (id != null) canvas.remove(id); }
}

class EraseShape implements DrawCommand {
    private final Canvas canvas;
    private final long targetId;
    private Shape removedBackup; // so we can restore on undo

    public EraseShape(Canvas c, long targetId) { this.canvas=c;
this.targetId=targetId; }
    public void execute() { removedBackup = canvas.remove(targetId); }
    public void undo() { if (removedBackup != null)
canvas.addShape(removedBackup); }
}

// Invoker
class Editor {
    private final Deque<DrawCommand> history = new ArrayDeque<>();
    public void run(DrawCommand cmd) { cmd.execute(); history.push(cmd); }
```

```
    public void undo() { if (!history.isEmpty()) history.pop().undo(); }
}

// Usage
class DemoCommandDrawing {
    public static void main(String[] args) {
        Canvas canvas = new Canvas();
        Editor editor = new Editor();

        DrawLine line = new DrawLine(canvas, 0,0, 100,100);
        editor.run(line);
        editor.run(new DrawCircle(canvas, 50,50, 20));
        canvas.render();

        // Erase the line we just drew
        editor.run(new EraseShape(canvas, 1)); // assuming first assigned id was 1
        canvas.render();

        // Undo erase → line comes back
        editor.undo();
        canvas.render();

        // Undo circle draw
        editor.undo();
        canvas.render();
    }
}
```

Reasoning to interviewer:

- Each action is a command object that knows how to apply and undo itself; the editor just orchestrates.
- Commands capture the minimal state needed for undo (shape id or full shape backup).
- The receiver (Canvas) is a simple shape store; commands keep business logic out of it.
- The history stack gives multi-level undo with zero coupling between actions.

Future scope:

- Add a redo stack; after an undo, pushing a new command clears the redo stack.
- Macro commands (draw house = many lines and circles) that undo as a group.

- Snapshot/memento support for heavy operations or selection moves.
- Persist command logs to enable collaborative editing or session replay.

Scenario 51: Prototype — rich document templates

A document editor offers heavy templates (resume, invoice, legal letter) with styles, headers, watermarks, and default sections. Creating these from scratch every time is wasteful. We want to clone a prebuilt prototype and then tweak it.

Answer:

I'll keep a Document prototype with a copy() method. Each template instance holds both intrinsic parts (e.g., an immutable watermark image we can shallow-share) and mutable parts (styles map, sections) that need deep copies. A small TemplateRegistry exposes preconfigured prototypes; creating a new file is just registry.get("RESUME").copy().

Code:

```
import java.util.*;

// Prototype
interface Document {
    Document copy();              // deep/shallow as appropriate
    void setTitle(String title);
    void addSection(String section);
    void updateStyle(String key, String value);
    void render();
}

// Value objects
final class Header { final String text; Header(String t){ this.text=t; } }
final class Watermark {             // heavy/immutable (e.g., image bytes)
    private final byte[] png;
    public Watermark(byte[] png) { this.png = png; }
    public byte[] bytes(){ return png; } // do not expose mutable internals in real code
}

// Concrete prototype
class TemplateDocument implements Document {
    private String title;
    private Header header;              // small immutable -> shallow copy is fine
    private final Watermark watermark;     // heavy immutable -> shallow share
    private final Map<String,String> styles;   // mutable -> deep copy
    private final List<String> sections;       // mutable -> deep copy

    public TemplateDocument(String title, Header header, Watermark watermark,
                            Map<String,String> styles, List<String> sections) {
        this.title = title;
        this.header = header;
        this.watermark = watermark;
        this.styles = new HashMap<>(styles);          // defensive copy
        this.sections = new ArrayList<>(sections);    // defensive copy
```

```
    }

    // Copy constructor (controls deep vs shallow)
    private TemplateDocument(TemplateDocument src) {
        this.title = src.title;
        this.header = src.header;                    // shallow (immutable)
        this.watermark = src.watermark;              // shallow (immutable, large)
        this.styles = new HashMap<>(src.styles);     // deep (new map)
        this.sections = new ArrayList<>(src.sections);// deep (new list)
    }

    @Override public Document copy() { return new TemplateDocument(this); }
    @Override public void setTitle(String title) { this.title = title; }
    @Override public void addSection(String section) { sections.add(section); }
    @Override public void updateStyle(String k, String v) { styles.put(k, v); }

    @Override public void render() {
        System.out.println("Doc: " + title);
        System.out.println("Header: " + header.text);
        System.out.println("Styles: " + styles);
        System.out.println("Sections: " + sections);
        System.out.println("Watermark bytes: " + watermark.bytes().length);
    }
}

// Registry for prototypes
class TemplateRegistry {
    private final Map<String, Document> prototypes = new HashMap<>();
    public void register(String key, Document proto) { prototypes.put(key, proto); }
    public Document get(String key) {
        Document proto = prototypes.get(key);
        if (proto == null) throw new IllegalArgumentException("Unknown template: " +
key);
        return proto.copy(); // callers always receive a fresh clone
    }
}

// Usage
class DemoPrototypeDocs {
    public static void main(String[] args) {
        Watermark wm = new Watermark(new byte[1_000_000]); // large, immutable
        Header hdr = new Header("ACME Corp.");
        Map<String,String> resumeStyles = Map.of("font","Serif","size","12pt");
        List<String> resumeSections = List.of("Summary","Experience","Education");

        TemplateDocument resumeProto =
            new TemplateDocument("Resume Template", hdr, wm, resumeStyles,
resumeSections);

        TemplateRegistry reg = new TemplateRegistry();
```

```
        reg.register("RESUME", resumeProto);

        // Create new docs quickly
        Document d1 = reg.get("RESUME");
        d1.setTitle("Resume - Rajat G.");
        d1.addSection("Skills: Java, Spring, Kubernetes");
        d1.updateStyle("size", "11pt");
        d1.render();

        Document d2 = reg.get("RESUME"); // fresh clone unaffected by d1 mutations
        d2.setTitle("Resume - Anita S.");
        d2.render();
    }
}
```

Reasoning to interviewer:

- Prototype avoids rebuilding complex objects (styles, sections, watermarks) each time—cheap cloning wins.
- Deep-copy mutable collections to prevent clones from interfering; shallow-share large immutable assets to save memory.
- The registry gives a single source of truth for templates and hides construction complexity.
- Copy constructor centralizes the deep/shallow policy; future fields follow the same rule.

Future scope:

- Add deepCopy() vs shallowCopy() variants if callers need explicit control.
- Version templates in the registry (e.g., “LEGAL_V2”) and support migration.
- Lazy-load heavy parts (like watermark bytes) with a proxy if first-render only.
- Add afterClone hooks for per-document initialization (IDs, timestamps).

Scenario 52: Composite — nested dashboard widgets

An analytics dashboard nests containers and leaves: a Grid contains Cards; Cards contain Charts or Tables. We'd like to call render() and refresh() on any widget uniformly and let composites delegate to their children.

Answer:

I'll define a Widget interface with render() and refresh(). Leaf widgets (Chart, Table) implement behavior directly. Composite widgets (Card, Grid) keep a list of child widgets and forward operations to them. The client treats containers and leaves the same way—classic Composite.

Code:

```
import java.util.*;

// Component
interface Widget {
    void render();
    void refresh(); // e.g., re-query data
}

// Leaf widgets
class Chart implements Widget {
    private final String title;
    public Chart(String title) { this.title = title; }
    @Override public void render()  { System.out.println("Render Chart: " + title); }
    @Override public void refresh() { System.out.println("Refresh Chart data: " +
title); }
}

class Table implements Widget {
    private final String title;
    public Table(String title) { this.title = title; }
    @Override public void render()  { System.out.println("Render Table: " + title); }
    @Override public void refresh() { System.out.println("Refresh Table data: " +
title); }
}

// Composite base (optional utility)
abstract class CompositeWidget implements Widget {
    protected final List<Widget> children = new ArrayList<>();
    public void add(Widget w)    { children.add(Objects.requireNonNull(w)); }
    public void remove(Widget w) { children.remove(w); }
    @Override public void render()  { children.forEach(Widget::render); }
    @Override public void refresh() { children.forEach(Widget::refresh); }
}

// Concrete composites
```

```
class Card extends CompositeWidget {
    private final String header;
    public Card(String header) { this.header = header; }
    @Override public void render() {
        System.out.println("Card: " + header);
        super.render();
    }
    @Override public void refresh() {
        System.out.println("Card refresh: " + header);
        super.refresh();
    }
}

class Grid extends CompositeWidget {
    private final int columns;
    public Grid(int columns) { this.columns = columns; }
    @Override public void render() {
        System.out.println("Grid (" + columns + " cols)");
        super.render();
    }
    @Override public void refresh() {
        System.out.println("Grid refresh");
        super.refresh();
    }
}

// Usage
class DemoCompositeDashboard {
    public static void main(String[] args) {
        Widget revenueChart = new Chart("Monthly Revenue");
        Widget topSkusTable = new Table("Top SKUs");

        Card kpiCard = new Card("KPI Overview");
        kpiCard.add(revenueChart);
        kpiCard.add(topSkusTable);

        Card opsCard = new Card("Ops Health");
        opsCard.add(new Chart("Error Rate"));
        opsCard.add(new Table("Open Incidents"));

        Grid grid = new Grid(2);
        grid.add(kpiCard);
        grid.add(opsCard);

        // Uniform operations
        grid.render();
        grid.refresh();
    }
}
```

Reasoning to interviewer:

- Composite gives one interface for both containers and leaves—client code doesn’t branch on type.
- Composites simply aggregate and delegate; they can add container-specific behavior (headers, layout).
- Extending the tree is trivial: add a Tabs composite or Gauge leaf without touching the rest.
- This fits dashboards well where refresh/render naturally fan out to nested widgets.

Future scope:

- Add find(predicate) traversal to target specific widgets (e.g., refresh only charts).
- Support partial refresh (dirty flags) and asynchronous refresh for slow data sources.
- Introduce lifecycle hooks (mount/unmount) to manage resources like websockets.
- Combine with Flyweight for repeated leaf widgets sharing heavy style/skin data.

Scenario 53: Bridge — media app controls vs devices

We have two axes that change independently: control UIs (BasicControls, ProControls) and playback devices (SmartTV, Projector, BluetoothSpeaker). We don't want a class per combination. Bridge lets us keep "controls" as the abstraction and "device" as the implementation; either side can evolve without touching the other.

Answer:

I'll define a Device implementor with operations like powerOn, setVolume, play. The abstraction Controls holds a Device reference and defines high-level actions. Concrete controls (basic vs pro) implement user flows (mute, quick-seek, EQ presets) by delegating to the Device. Now adding a new device or a new control surface is just a new class on its side of the bridge.

Code:

```
// Implementor
interface Device {
    void powerOn();
    void powerOff();
    void setVolume(int level); // 0..100
    void play(String title);
}

// Concrete Implementors
class SmartTV implements Device {
    private int volume;
    public void powerOn()  { System.out.println("SmartTV ON"); }
    public void powerOff() { System.out.println("SmartTV OFF"); }
    public void setVolume(int level) { volume = clamp(level);
System.out.println("SmartTV vol=" + volume); }
    public void play(String title) { System.out.println("SmartTV playing: " + title); }
    private int clamp(int v) { return Math.max(0, Math.min(100, v)); }
}

class Projector implements Device {
    private int volume;
    public void powerOn()  { System.out.println("Projector ON (lamp warmup)"); }
    public void powerOff() { System.out.println("Projector OFF (cooldown)"); }
    public void setVolume(int level) { volume = Math.max(0, Math.min(100, level));
System.out.println("Projector vol=" + volume); }
    public void play(String title) { System.out.println("Projector playing: " + title); }
}

class BluetoothSpeaker implements Device {
```

```
    private int volume;
    public void powerOn()  { System.out.println("BT Speaker ON (paired)"); }
    public void powerOff() { System.out.println("BT Speaker OFF"); }
    public void setVolume(int level) { volume = Math.max(0, Math.min(100, level));
System.out.println("BT Speaker vol=" + volume); }
    public void play(String title) { System.out.println("BT Speaker streaming: " +
title); }
}

// Abstraction
abstract class Controls {
    protected final Device device;
    protected Controls(Device device) { this.device = device; }
    public abstract void onPower();
    public abstract void play(String title);
    public abstract void volumeUp();
    public abstract void volumeDown();
}

// Refined Abstractions
class BasicControls extends Controls {
    private int volume = 30;
    public BasicControls(Device device) { super(device); }
    public void onPower() { device.powerOn(); device.setVolume(volume); }
    public void play(String title) { device.play(title); }
    public void volumeUp()   { volume = Math.min(100, volume + 5);
device.setVolume(volume); }
    public void volumeDown() { volume = Math.max(0,   volume - 5);
device.setVolume(volume); }
}

class ProControls extends Controls {
    private int volume = 50;
    public ProControls(Device device) { super(device); }
    public void onPower() { device.powerOn(); device.setVolume(volume); }
    public void play(String title) { device.play("[HiFi] " + title); }
    public void volumeUp()   { volume = Math.min(100, volume + 2);
device.setVolume(volume); }
    public void volumeDown() { volume = Math.max(0,   volume - 2);
device.setVolume(volume); }
    // extra pro feature
    public void mute() { volume = 0; device.setVolume(0); }
}

// Usage
class DemoBridge {
    public static void main(String[] args) {
        Controls ui = new BasicControls(new SmartTV());
        ui.onPower();
        ui.play("Planet Earth");
```

```
        ui.volumeUp();

        Controls studio = new ProControls(new BluetoothSpeaker());
        studio.onPower();
        studio.play("Jazz Classics");
        studio.volumeDown();
        ((ProControls) studio).mute();

        // Swap either side freely:
        Controls theater = new ProControls(new Projector());
        theater.onPower();
        theater.play("Sci-Fi Night");
    }
}
```

Reasoning to interviewer:

- Controls depend only on the Device interface; devices don't know about controls.
- Combinatorial explosion is avoided (no SmartTVBasicControls, SmartTVProControls, etc.).
- Adding a new device (e.g., "Soundbar") or a new control surface ("KidsControls") doesn't require touching the other side.
- Behavior differences (e.g., projector warmup) live in the implementor where they belong.

Future scope:

- Add more device ops (seek, captions) without changing BasicControls—only the abstraction uses what it needs.
- Compose "macro" controls (movie mode = power on + set volume + play).
- Inject devices via DI and hot-swap controls at runtime (e.g., remote vs mobile app).
- Add a TelemetryDevice decorator to collect play/volume metrics without changing controls.

Scenario 54: Flyweight — memory-efficient character rendering

Rendering millions of characters naively creates huge duplication of font/glyph data (same “A” metrics repeated everywhere). Flyweight shares that intrinsic data; we pass varying bits (position, color) as extrinsic state at render time.

Answer:

I’ll define a Glyph flyweight that stores intrinsic state: font family, style, size, and the glyph outline/metrics for a specific character code. A GlyphFactory caches and returns shared Glyph instances keyed by (char, font,size,style). A lightweight GlyphRun or RenderedChar holds only extrinsic state: x/y position, color, and effects. Rendering becomes glyph.draw(x, y, color). Memory drops dramatically because millions of characters now share thousands of glyphs.

Code:

```
import java.util.*;

// Flyweight (intrinsic state)
final class Glyph {
    private final char codePoint;
    private final String font;
    private final int size;
    private final int style; // 0=normal,1=bold,2=italic... (demo)
    // Pretend this is a heavy outline/metrics blob:
    private final byte[] outlineData;

    private Glyph(char codePoint, String font, int size, int style) {
        this.codePoint = codePoint;
        this.font = font;
        this.size = size;
        this.style = style;
        this.outlineData = generateOutline(codePoint, font, size, style);
    }

    static Glyph of(char c, String font, int size, int style) {
        return new Glyph(c, font, size, style);
    }

    // heavy generation (simulated)
    private byte[] generateOutline(char c, String f, int size, int style) {
        return new byte[2048]; // pretend curve tables, hinting, kerning
    }

    // Render using extrinsic state
    public void draw(int x, int y, int argbColor) {
        // draw using outlineData at (x,y) with argbColor
        // (omitted actual graphics)
    }
```

```
    @Override public String toString() {
        return "Glyph[" + codePoint + "," + font + "," + size + "," + style + "]";
    }
}

// Flyweight Factory with caching
final class GlyphFactory {
    private static final class Key {
        final char c; final String f; final int s; final int st;
        Key(char c, String f, int s, int st) { this.c=c; this.f=f; this.s=s; this.st=st; }
        @Override public boolean equals(Object o) {
            if (this == o) return true; if (!(o instanceof Key)) return false;
            Key k = (Key) o; return c==k.c && s==k.s && st==k.st && f.equals(k.f);
        }
        @Override public int hashCode() { return Objects.hash(c,f,s,st); }
    }

    private final Map<Key, Glyph> cache = new HashMap<>();

    public Glyph get(char c, String font, int size, int style) {
        Key k = new Key(c, font, size, style);
        return cache.computeIfAbsent(k, key -> Glyph.of(c, font, size, style));
    }

    public int flyweightCount() { return cache.size(); }
}

// Extrinsic holder (very light)
record RenderedChar(Glyph glyph, int x, int y, int argbColor) {
    void paint() { glyph.draw(x, y, argbColor); }
}

// Usage / demo
class DemoFlyweightEditor {
    public static void main(String[] args) {
        GlyphFactory factory = new GlyphFactory();
        List<RenderedChar> screen = new ArrayList<>();

        String text = "AABBAA\nAABBAA"; // repeated glyphs
        int y = 10;
        for (String line : text.split("\n")) {
            int x = 10;
            for (char c : line.toCharArray()) {
                Glyph g = factory.get(c, "JetBrainsMono", 14, 0);
                screen.add(new RenderedChar(g, x, y, 0xFF222222)); // extrinsic
pos/color
                x += 8;
            }
            y += 16;
```

```
        }

        // Paint all (simulated)
        screen.forEach(RenderedChar::paint);

        System.out.println("Unique flyweights created: " + factory.flyweightCount()); //
small
        System.out.println("Rendered chars on screen:  " + screen.size());          //
large
    }
}
```

Reasoning to interviewer:

- Intrinsic state (glyph outline/metrics) is shared across all occurrences; extrinsic (x/y/color) is passed in at draw time.
- The factory ensures there is exactly one flyweight per (char,font,size,style) key, dramatically reducing memory.
- Rendering remains simple: create many RenderedChar objects that refer to a small glyph pool.
- This fits editors, map labels, or terminals where repeated symbols dominate.

Future scope:

- Add kerning/ligature lookup inside Glyph while keeping run-time positions extrinsic.
- Use weak references in the factory to let rare glyphs be GC’d when memory is tight.
- Batch draw calls per glyph to leverage GPU instancing (perf win).
- Layer a text-shaping engine (HarfBuzz) ahead of flyweights for complex scripts while still sharing glyph data.

Scenario 55: Facade — “one-click export” across 5 subsystems

You’ve got a pipeline: fetch data → transform → render PDF → upload to cloud → email the link. Product wants a single button; engineering wants subsystems decoupled and testable.

Answer

I’ll expose one simple API on a Facade, say exportReport(reportId, recipient). Internally, the Facade coordinates independent services (DataService, Transformer, PdfRenderer, CloudStorage, Mailer). Each service is injected into the Facade (constructor DI), so they’re mockable in tests. The Facade owns orchestration and error mapping; subsystems remain focused and reusable.

Code

```
// Result types
record ExportResult(boolean success, String downloadUrl, String message) {}

// Subsystems (contracts)
interface DataService { ReportData fetch(String reportId); }
interface Transformer { byte[] transform(ReportData raw); }
interface PdfRenderer { byte[] render(byte[] normalizedData); }
interface CloudStorage { String upload(String path, byte[] pdfBytes); } // returns
URL
interface Mailer { void send(String to, String subject, String body); }

// Simple DTO
record ReportData(String id, String payloadJson) {}

// Concrete stubs (for demo)
class DataServiceImpl implements DataService {
    public ReportData fetch(String reportId) { return new ReportData(reportId,
"{...json...}"); }
}
class TransformerImpl implements Transformer {
    public byte[] transform(ReportData raw) { return ("TRANSFORMED:" +
raw.payloadJson()).getBytes(); }
}
class PdfRendererImpl implements PdfRenderer {
    public byte[] render(byte[] normalized) { return ("PDF:" + new
String(normalized)).getBytes(); }
}
class S3Storage implements CloudStorage {
    public String upload(String path, byte[] pdf) { return "https://cdn.example.com/" +
path; }
}
class SmtpMailer implements Mailer {
    public void send(String to, String subject, String body) {
        System.out.printf("Email to %s: %s%n%s%n", to, subject, body);
```

```
    }
}

// Facade
class ReportExporterFacade {
    private final DataService data;
    private final Transformer transformer;
    private final PdfRenderer renderer;
    private final CloudStorage storage;
    private final Mailer mailer;

    public ReportExporterFacade(DataService data, Transformer transformer,
PdfRenderer renderer,
                                CloudStorage storage, Mailer mailer) {
        this.data = data; this.transformer = transformer; this.renderer = renderer;
        this.storage = storage; this.mailer = mailer;
    }

    public ExportResult exportReport(String reportId, String recipientEmail) {
        try {
            ReportData raw = data.fetch(reportId);
            byte[] normalized = transformer.transform(raw);
            byte[] pdf = renderer.render(normalized);

            String key = "reports/" + reportId + "-" + System.currentTimeMillis() + ".pdf";
            String url = storage.upload(key, pdf);

            mailer.send(recipientEmail, "Your report is ready", "Download: " + url);
            return new ExportResult(true, url, "Export successful");
        } catch (Exception e) {
            // map/normalize errors; keep subsystems decoupled from callers
            return new ExportResult(false, null, "Export failed: " + e.getMessage());
        }
    }
}

// Usage
class DemoFacade {
    public static void main(String[] args) {
        ReportExporterFacade facade = new ReportExporterFacade(
            new DataServiceImpl(), new TransformerImpl(), new PdfRendererImpl(),
            new S3Storage(), new SmtpMailer()
        );
        ExportResult r = facade.exportReport("Q3-2025", "ops@example.com");
        System.out.println(r);
    }
}
```

Reasoning to interviewer

- The Facade gives product a single “one-click” API while preserving clean, testable services.
- Constructor DI keeps subsystems swappable/mocked; orchestration is the only responsibility of the Facade.
- Errors are normalized in one place; subsystem changes don’t ripple to callers.
- If any step changes (e.g., swap S3 with GCS), only DI wiring changes—no changes to controllers/UI.

Future scope

- Add async/batch exports; facade returns a job ID and emits completion events.
- Observability hooks (metrics/spans) live in the Facade—one place to measure the end-to-end path.
- Feature flags (e.g., canary a new renderer) by choosing different injected implementations.
- Idempotency keys to avoid duplicate emails/uploads on retries.

Scenario 56: Chain of Responsibility — API security pipeline

Incoming requests must pass Auth → Rate-Limit → Data Validation → Business Rules. Any step can fail fast; ops may reorder or insert steps later.

Answer

I'll model each step as a Handler with handle(ctx, next) (or classic setNext). A Chain assembles handlers dynamically and invokes them in order. Each handler decides to either short-circuit (on failure) or delegate to the next. This keeps steps independent and reconfigurable.

Code

```
import java.util.ArrayList;
import java.util.List;
import java.util.function.BiConsumer;

// Request/Response context
class RequestContext {
    final String token;
    final String path;
    final String body;
    int userId = -1;
    int httpStatus = 200;
    String message = "OK";

    RequestContext(String token, String path, String body) {
        this.token = token; this.path = path; this.body = body;
    }
    void fail(int status, String msg) { this.httpStatus = status; this.message = msg; }
    boolean isFailed() { return httpStatus >= 400; }
}

// Handler SPI (functional: handler receives ctx + next)
@FunctionalInterface
interface Handler extends BiConsumer<RequestContext, Runnable> {}

// Concrete handlers
class AuthHandler implements Handler {
    public void accept(RequestContext ctx, Runnable next) {
        if (ctx.token == null || !ctx.token.startsWith("Bearer ")) {
            ctx.fail(401, "Unauthorized");
            return; // short-circuit
        }
        ctx.userId = 42; // demo: extracted from token
        next.run();
    }
}
```

```
class RateLimitHandler implements Handler {
    public void accept(RequestContext ctx, Runnable next) {
        // demo: block specific path occasionally
        if ("/heavy".equals(ctx.path)) {
            ctx.fail(429, "Rate limit exceeded");
            return;
        }
        next.run();
    }
}

class ValidationHandler implements Handler {
    public void accept(RequestContext ctx, Runnable next) {
        if (ctx.body == null || ctx.body.isBlank()) {
            ctx.fail(400, "Invalid request body");
            return;
        }
        next.run();
    }
}

class BusinessRulesHandler implements Handler {
    public void accept(RequestContext ctx, Runnable next) {
        if (ctx.body.contains("forbidden")) {
            ctx.fail(403, "Business rule violation");
            return;
        }
        next.run();
    }
}

// Chain builder
class HandlerChain {
    private final List<Handler> handlers = new ArrayList<>();

    public HandlerChain add(Handler h) { handlers.add(h); return this; }

    // Compose into one Runnable pipeline
    public void handle(RequestContext ctx) {
        Runnable composed = () -> {}; // terminal (no-op)
        // build in reverse so the first added runs first
        for (int i = handlers.size() - 1; i >= 0; i--) {
            Handler current = handlers.get(i);
            Runnable next = composed;
            composed = () -> {
                if (!ctx.isFailed()) current.accept(ctx, next);
            };
        }
        composed.run();
    }
```

```
}

// Usage
class DemoChain {
    public static void main(String[] args) {
        HandlerChain chain = new HandlerChain()
            .add(new AuthHandler())
            .add(new RateLimitHandler())
            .add(new ValidationHandler())
            .add(new BusinessRulesHandler());

        RequestContext ok = new RequestContext("Bearer abc", "/search",
"{\"q\":\"laptops\"}");
        chain.handle(ok);
        System.out.println(ok.httpStatus + " " + ok.message); // 200 OK

        RequestContext rl = new RequestContext("Bearer abc", "/heavy",
"{\"q\":\"x\"}");
        chain.handle(rl);
        System.out.println(rl.httpStatus + " " + rl.message); // 429 Rate limit exceeded

        // Reorder dynamically (e.g., validate before rate limit)
        HandlerChain alt = new HandlerChain()
            .add(new AuthHandler())
            .add(new ValidationHandler())
            .add(new RateLimitHandler())
            .add(new BusinessRulesHandler());

        RequestContext bad = new RequestContext("Bearer abc", "/search", "");
        alt.handle(bad);
        System.out.println(bad.httpStatus + " " + bad.message); // 400 Invalid request
body
    }
}
```

Reasoning to interviewer

- Each handler is single-purpose and stateless; the chain composes them declaratively.
- Short-circuiting is natural: a handler simply doesn’t call next.run().
- Dynamic reordering/insertion is just rebuilding the chain list—no changes to handlers.
- The context object carries shared state and the failure contract (status/message).

Future scope

- Add async/non-blocking handlers (e.g., Redis rate limiter) by adopting CompletionStage or reactive types.
- Per-route chains (different pipelines for /admin vs /public).
- Metrics/tracing handler to time each step and add spans.
- A policy DSL to generate chains from config for true runtime reconfiguration.

Scenario 57: Interpreter — mini rule language for discounts/flags

A business team wants rules like: amount > 1000 AND country == "IN" OR vip == true. We need to parse and evaluate these against an input context (e.g., a map from field → value), and we should be able to add new operators later (like contains, between) without editing existing operator code.

Answer:

I'll model the language as an AST of Expression nodes. Terminals read values from the context; non-terminals apply an Operator. Parsing will use a Pratt/precedence parser with an operator registry. New operators are added by registration (token → precedence → factory) rather than changing core classes.

Code:

```
import java.util.*;
import java.util.function.BiFunction;

// ===== Core AST =====
interface Expression {
    Object eval(Map<String, Object> ctx);
}

// Literal and identifier terminals
final class Literal implements Expression {
    private final Object value;
    Literal(Object value) { this.value = value; }
    public Object eval(Map<String, Object> ctx) { return value; }
}

final class Identifier implements Expression {
    private final String name;
    Identifier(String name) { this.name = name; }
    public Object eval(Map<String, Object> ctx) { return ctx.get(name); }
}

// Operator interface (binary for demo; unary can be added similarly)
interface Operator {
    String token();
    int precedence();      // higher binds tighter
    boolean rightAssoc();  // for right-associative ops if any
    Expression bind(Expression left, Tokenizer tz, OperatorRegistry reg);
    Object apply(Object left, Object right);
}

// Generic binary operator
abstract class BinaryOperator implements Operator {
    private final String token; private final int prec; private final boolean right;
```

```
    BinaryOperator(String token, int prec, boolean rightAssoc) { this.token = token;
this.prec = prec; this.right = rightAssoc; }
    public String token() { return token; }
    public int precedence() { return prec; }
    public boolean rightAssoc() { return right; }

    public Expression bind(Expression left, Tokenizer tz, OperatorRegistry reg) {
        Expression right = Parser.parseExpression(tz, reg, precedence() -
(rightAssoc() ? 1 : 0));
        BinaryOperator self = this;
        return new Expression() {
            public Object eval(Map<String, Object> ctx) {
                return self.apply(left.eval(ctx), right.eval(ctx));
            }
            public String toString() { return "(" + left + " " + token + " " + right + ")"; }
        };
    }
}

// ===== Operator registry =====
final class OperatorRegistry {
    private final Map<String, Operator> ops = new HashMap<>();
    public void register(Operator op) { ops.put(op.token(), op); }
    public Operator get(String t) { return ops.get(t); }
    public boolean isOperator(String t) { return ops.containsKey(t); }
    public Collection<Operator> operators() { return ops.values(); }
}

// Built-in operators (extensible)
final class OpEq extends BinaryOperator {
    OpEq() { super("==", 40, false); }
    public Object apply(Object l, Object r) { return Objects.equals(l, r); }
}
final class OpGt extends BinaryOperator {
    OpGt() { super(">", 50, false); }
    public Object apply(Object l, Object r) { return ((Number)l).doubleValue() >
((Number)r).doubleValue(); }
}
final class OpAnd extends BinaryOperator {
    OpAnd() { super("AND", 20, false); }
    public Object apply(Object l, Object r) { return ((Boolean)l) && ((Boolean)r); }
}
final class OpOr extends BinaryOperator {
    OpOr() { super("OR", 10, false); }
    public Object apply(Object l, Object r) { return ((Boolean)l) || ((Boolean)r); }
}

// Example of adding a new operator without touching above: CONTAINS
final class OpContains extends BinaryOperator {
    OpContains() { super("CONTAINS", 45, false); }
```

```
    public Object apply(Object l, Object r) {
        if (l instanceof String && r instanceof String) return ((String)
l).contains((String) r);
        if (l instanceof Collection<?> c) return c.contains(r);
        return false;
    }
}

// ===== Tokenizer (very small) =====
final class Tokenizer {
    private final List<String> tokens;
    private int i = 0;

    Tokenizer(String src) {
        this.tokens = lex(src);
    }

    private List<String> lex(String s) {
        List<String> out = new ArrayList<>();
        StringBuilder buf = new StringBuilder();
        for (int p = 0; p < s.length(); ) {
            char c = s.charAt(p);
            if (Character.isWhitespace(c)) { p++; continue; }
            if (c == '(' || c == ')') { out.add(String.valueOf(c)); p++; continue; }
            if (c == '>' || c == '<' || c == '=') {
                if (p + 1 < s.length() && s.charAt(p + 1) == '=') { out.add("" + c + "="); p
+= 2; }
                else { out.add("" + c); p++; }
                continue;
            }
            if (c == "" || c == '\'') { // string literal
                char quote = c; p++;
                StringBuilder sb = new StringBuilder();
                while (p < s.length() && s.charAt(p) != quote) sb.append(s.charAt(p++));
                p++; out.add("\"" + sb + "\""); continue;
            }
            // keyword/ident/number
            buf.setLength(0);
            while (p < s.length() && !Character.isWhitespace(s.charAt(p)) &&
"()".indexOf(s.charAt(p)) < 0) {
                char ch = s.charAt(p);
                if (ch == '>' || ch == '<' || ch == '=') break;
                buf.append(ch); p++;
            }
            out.add(buf.toString());
        }
        return out;
    }

    boolean has() { return i < tokens.size(); }
```

```
    String peek() { return tokens.get(i); }
    String next() { return tokens.get(i++); }
    boolean match(String t) { if (has() && peek().equals(t)) { i++; return true; } return
false; }
}

// ===== Parser (Pratt) =====
final class Parser {
    static Expression parse(String src, OperatorRegistry reg) {
        Tokenizer tz = new Tokenizer(src);
        Expression expr = parseExpression(tz, reg, 0);
        if (tz.has()) throw new IllegalArgumentException("Unexpected token: " +
tz.peek());
        return expr;
    }

    static Expression parseExpression(Tokenizer tz, OperatorRegistry reg, int
minPrec) {
        Expression left = parsePrimary(tz, reg);

        while (tz.has() && reg.isOperator(tz.peek())) {
            Operator op = reg.get(tz.peek());
            if (op.precedence() < minPrec) break;
            tz.next(); // consume operator
            left = op.bind(left, tz, reg);
        }
        return left;
    }

    private static Expression parsePrimary(Tokenizer tz, OperatorRegistry reg) {
        if (!tz.has()) throw new IllegalArgumentException("Unexpected end");
        String t = tz.peek();

        if (t.equals("(")) {
            tz.next();
            Expression e = parseExpression(tz, reg, 0);
            if (!tz.match(")")) throw new IllegalArgumentException("Expected ')'");
            return e;
        }
        // string literal
        if (t.startsWith("\"")) { tz.next(); return new Literal(t.substring(1, t.length()-1)); }
        // boolean literal
        if (t.equalsIgnoreCase("true") || t.equalsIgnoreCase("false")) { tz.next(); return
new Literal(Boolean.parseBoolean(t)); }
        // number
        if (t.matches("-?\\d+(\\.\\d+)?")) { tz.next(); return new
Literal(Double.valueOf(t)); }
        // identifier
        tz.next(); return new Identifier(t);
    }
```

```
}

// ===== Demo =====
class DemoInterpreter {
    public static void main(String[] args) {
        OperatorRegistry reg = new OperatorRegistry();
        reg.register(new OpEq());
        reg.register(new OpGt());
        reg.register(new OpAnd());
        reg.register(new OpOr());
        reg.register(new OpContains()); // added later without touching parser/core

        String rule = "amount > 1000 AND country == \"IN\" OR vip == true";
        Expression expr = Parser.parse(rule, reg);

        Map<String,Object> ctx1 = Map.of("amount", 1200.0, "country", "IN", "vip",
false);
        Map<String,Object> ctx2 = Map.of("amount", 500.0,  "country", "US", "vip",
true);
        Map<String,Object> ctx3 = Map.of("amount", 500.0,  "country", "US", "vip",
false);

        System.out.println(expr.eval(ctx1)); // true
        System.out.println(expr.eval(ctx2)); // true (vip)
        System.out.println(expr.eval(ctx3)); // false
    }
}
```

Reasoning to interviewer:

- Terminals (Identifier, Literal) keep data access simple; non-terminals delegate to Operator.
- The Pratt parser + operator registry gives precedence control and pluggability—adding BETWEEN, !=, or unary NOT is just a new Operator with precedence and an apply(); no changes to existing operators.
- Evaluation runs against a Map<String,Object> context, so we can feed it from HTTP requests, DB rows, or DTOs.
- This design scales from basic predicates to richer DSLs (regex match, date math) while keeping core stable.

Future scope:

- Add unary operators (NOT) and parentheses handling (already supported).
- Short-circuit evaluation within boolean operators.

- Strong typing via a Value wrapper to avoid raw Object.
- Compile to bytecode/lambdas for hot paths, or precompute rule plans.

Scenario 58: Iterator — lazy PagedCollection over an API

We fetch results page-by-page from an API but want clients to loop naturally with hasNext()/next() without knowing about pagination or network calls.

Answer:

I'll expose a PagedCollection<T> that holds page size and a PageFetcher function. Its iterator() returns a stateful Iterator<T> that buffers the current page, fetches next page lazily when the buffer is empty, and hides cursors/tokens. We can also offer forEachBatch(Consumer<List<T>>) for batch processing.

Code:

```
import java.util.*;
import java.util.function.Function;

// Page result from API
record Page<T>(List<T> items, String nextCursor) {}

// Strategy to fetch a page given a cursor (null for first page)
@FunctionalInterface
interface PageFetcher<T> extends Function<String, Page<T>> {
    Page<T> apply(String cursor);
}

// Collection that knows how to create iterators
class PagedCollection<T> implements Iterable<T> {
    private final PageFetcher<T> fetcher;

    public PagedCollection(PageFetcher<T> fetcher) {
        this.fetcher = fetcher;
    }

    @Override
    public Iterator<T> iterator() {
        return new Iterator<>() {
            private final Deque<T> buffer = new ArrayDeque<>();
            private String cursor = null;    // null => first page
            private boolean end = false;

            private void ensureBuffer() {
                if (!buffer.isEmpty() || end) return;
                Page<T> page = fetcher.apply(cursor); // network call happens lazily
                if (page == null || page.items().isEmpty()) {
                    end = true;
                    return;
                }
                buffer.addAll(page.items());
                cursor = page.nextCursor();
```

```
            if (cursor == null) end = true; // no more pages after this buffer drains
        }

        public boolean hasNext() {
            ensureBuffer();
            return !buffer.isEmpty();
        }

        public T next() {
            if (!hasNext()) throw new NoSuchElementException();
            return buffer.removeFirst();
        }

        // Optional: unsupported by many APIs; provide if your backend supports
deletion
        public void remove() {
            throw new UnsupportedOperationException("remove not supported");
        }
    };
  }

  // Optional: process in page-sized batches (avoids per-item overhead)
  public void forEachBatch(java.util.function.Consumer<List<T>>
batchConsumer) {
    String cur = null;
    while (true) {
      Page<T> p = fetcher.apply(cur);
      if (p == null || p.items().isEmpty()) break;
      batchConsumer.accept(p.items());
      cur = p.nextCursor();
      if (cur == null) break;
    }
  }
}

// ===== Demo =====
class DemoIteratorPaging {
  public static void main(String[] args) {
    // Fake API: 3 pages of integers
    PageFetcher<Integer> fetcher = cursor -> {
      if (cursor == null) return new Page<>(List.of(1,2,3), "c1");
      if (cursor.equals("c1")) return new Page<>(List.of(4,5,6), "c2");
      if (cursor.equals("c2")) return new Page<>(List.of(7,8), null);
      return new Page<>(List.of(), null);
    };

    PagedCollection<Integer> pc = new PagedCollection<>(fetcher);

    // Item-wise iteration (lazy page fetch)
    for (int x : pc) {
```

```
            System.out.print(x + " "); // 1 2 3 4 5 6 7 8
        }
        System.out.println();

        // Batch processing
        pc.forEachBatch(batch -> System.out.println("Batch: " + batch));
    }
}
```

Reasoning to interviewer:

- The iterator hides cursors, page sizes, and network calls; it fetches only when needed.
- hasNext() triggers fetches lazily; next() consumes from a buffer—works with for-each naturally.
- Batch processing avoids per-item overhead for analytics/export use cases.
- The PageFetcher strategy decouples iteration from the API client; easy to test with fakes/mocks.

Future scope:

- Add prefetching (fetch next page in background when buffer is half-empty).
- Expose Spliterator with ORDERED | NONNULL and a custom tryAdvance/forEachRemaining for stream support.
- Allow remove() when backend supports item deletion with a token.
- Exponential backoff/retry and rate-limit handling inside the iterator or fetcher.

Scenario 59: Mediator — coordinating a busy chat room

Multiple chat participants (users, bots, scanners) interacting directly gets tangled: mutes/blocks, file-scan flows, auto-replies. A Mediator lets everyone talk through one hub so rules and routing live centrally.

Answer:

I'll introduce a ChatMediator that owns routing and policies. Users never message each other directly; they call mediator.send(from, to, text) or mediator.upload(from, file). The mediator checks mutes/blocks, delivers to recipients, triggers virus scans on uploads, and fans messages out to bots for keyword autoresponses. This keeps participants simple and the orchestration in one place.

Code:

```
import java.util.*;

// Colleague
interface Participant {
    String id();
    void receive(String from, String message);
}

// Concrete colleague
class User implements Participant {
    private final String id;
    private final ChatMediator mediator;
    public User(String id, ChatMediator mediator) { this.id = id; this.mediator =
mediator; }
    public String id() { return id; }

    // outward actions always go via mediator
    public void say(String text) { mediator.broadcast(id, text); }
    public void dm(String to, String text) { mediator.send(id, to, text); }
    public void upload(String filename, byte[] content) { mediator.upload(id,
filename, content); }

    @Override public void receive(String from, String message) {
        System.out.println("[" + id + "] from " + from + ": " + message);
    }
}

// Bot colleague
class KeywordBot implements Participant {
    private final String id = "bot:helper";
    private final ChatMediator mediator;
    public KeywordBot(ChatMediator m) { this.mediator = m; }
    public String id() { return id; }
```

```
    @Override public void receive(String from, String message) {
        if (message.toLowerCase().contains("help")) {
            mediator.send(id(), from, "Here’s the help doc: /help");
        }
    }
}

// Virus scan service (not a Participant, invoked by mediator)
class VirusScanner {
    public boolean clean(byte[] content) { return content.length % 7 != 0; } // demo
logic
}

// Mediator
interface ChatMediator {
    void register(Participant p);
    void mute(String by, String target);
    void block(String by, String target);

    void broadcast(String from, String text);
    void send(String from, String to, String text);
    void upload(String from, String filename, byte[] content);
}

// Concrete mediator
class ChatRoom implements ChatMediator {
    private final Map<String, Participant> peers = new HashMap<>();
    private final Map<String, Set<String>> mutes = new HashMap<>();  // who I
mute (hide messages from)
    private final Map<String, Set<String>> blocks = new HashMap<>(); // who
cannot DM me
    private final VirusScanner scanner = new VirusScanner();

    public void register(Participant p) { peers.put(p.id(), p); }
    public void mute(String by, String target) { mutes.computeIfAbsent(by, k->new
HashSet<>()).add(target); }
    public void block(String by, String target) { blocks.computeIfAbsent(by, k->new
HashSet<>()).add(target); }

    public void broadcast(String from, String text) {
        for (Participant p : peers.values()) {
            if (!p.id().equals(from) && !isMutedFor(p.id(), from)) {
                p.receive(from, text);
            }
        }
    }

    public void send(String from, String to, String text) {
        Participant dst = peers.get(to);
        if (dst == null) return;
```

```
        if (isBlockedBy(to, from)) {
            peers.get(from).receive("system", "Your DM was blocked by " + to);
            return;
        }
        dst.receive(from, text);
    }

    public void upload(String from, String filename, byte[] content) {
        boolean ok = scanner.clean(content);
        String notice = ok ? ("uploaded " + filename) : ("blocked malware in " +
filename);
        broadcast("system", from + " " + notice);
        if (!ok) send("system", from, "Upload rejected: virus suspected.");
    }

    private boolean isMutedFor(String receiver, String sender) {
        return mutes.getOrDefault(receiver, Set.of()).contains(sender);
    }
    private boolean isBlockedBy(String receiver, String sender) {
        return blocks.getOrDefault(receiver, Set.of()).contains(sender);
    }
}

// Usage
class DemoMediatorChat {
    public static void main(String[] args) {
        ChatRoom room = new ChatRoom();
        User a = new User("alice", room);
        User b = new User("bob", room);
        KeywordBot bot = new KeywordBot(room);

        room.register(a); room.register(b); room.register(bot);
        room.mute("bob", "alice");          // Bob ignores Alice in broadcast
        room.block("alice", "bob");         // Alice blocks Bob's DMs

        a.say("hi all, need HELP with invoices");
        b.dm("alice", "Hey Alice, ping");    // blocked by Alice
        a.upload("report.pdf", new byte[12]);
        b.upload("weird.exe", new byte[14]); // length%7==0 -> "infected" -> rejected
    }
}
```

Reasoning to interviewer:

- Participants don’t know each other; they depend only on ChatMediator. This prevents an explosion of cross-references.
- Policies (mutes/blocks, scan outcomes, bot triggers) live in one place and are easy to test.

- Adding new behaviors (moderation, profanity filter, audit log) is a mediator concern—not changes across users/bots.
- External services (scanner) stay decoupled; the mediator composes them per event.

Future scope:

- Event bus inside the mediator (publish/subscribe) for plugins like analytics or moderation.
- Async scan with callbacks (don’t block UI on large files).
- Rooms/channels as separate mediator instances with their own policies.
- Rate limiting and flood control enforced centrally.

Scenario 60: Memento — safe undo/redo for a diagram editor

We need multi-level undo/redo of shapes (position, color, connections), but we must not expose internal structure. Snapshots should be compact and immutable.

Answer:

I'll keep the editor (Originator) with internal mutable state (a list of shapes). It exposes save() returning an opaque Memento value object containing a compact DTO snapshot (no references to live objects) and restore(memento) to roll back. A History (Caretaker) manages two stacks for undo/redo. Only the editor can read/write internals; the caretaker treats mementos as opaque blobs.

Code:

```
import java.util.*;

// Domain model (simplified)
final class Shape {
    final long id;
    String type;       // "RECT","CIRCLE"
    int x, y;          // position
    String color;      // "#RRGGBB"
    List<Long> links;  // connections to other shapes by id

    Shape(long id, String type, int x, int y, String color, List<Long> links) {
        this.id=id; this.type=type; this.x=x; this.y=y; this.color=color;
        this.links = new ArrayList<>(links);
    }

    Shape copy() { // deep copy to detach from live state
        return new Shape(id, type, x, y, color, new ArrayList<>(links));
    }
}

// Originator
class DiagramEditor {
    private final Map<Long, Shape> shapes = new LinkedHashMap<>();
    private long seq = 1;

    // Opaque memento type to outside world
    public static final class Memento {
        private final List<Snapshot> data;    // compact DTOs
        private final long seq;
        private Memento(List<Snapshot> data, long seq) { this.data = data; this.seq =
seq; }
    }
    // Compact DTO for each shape
    private static final class Snapshot {
```

```
        final long id; final String type; final int x,y; final String color; final List<Long>
links;
        Snapshot(Shape s) {
            this.id=s.id; this.type=s.type; this.x=s.x; this.y=s.y; this.color=s.color;
            this.links = List.copyOf(s.links);
        }
        Shape toShape() { return new Shape(id, type, x, y, color, links); }
    }

    public long add(String type, int x, int y, String color) {
        long id = seq++;
        shapes.put(id, new Shape(id, type, x, y, color, List.of()));
        return id;
    }
    public void move(long id, int dx, int dy) { var s = shapes.get(id); if (s != null) { s.x
+= dx; s.y += dy; } }
    public void recolor(long id, String c) { var s = shapes.get(id); if (s != null) s.color
= c; }
    public void link(long a, long b) { var s = shapes.get(a); if (s != null) s.links.add(b);
}

    // Create a compact, immutable snapshot (no live references)
    public Memento save() {
        List<Snapshot> list = new ArrayList<>(shapes.size());
        for (Shape s : shapes.values()) list.add(new Snapshot(s));
        return new Memento(List.copyOf(list), seq);
    }

    // Restore from memento
    public void restore(Memento m) {
        shapes.clear();
        for (Snapshot sn : m.data) shapes.put(sn.id, sn.toShape());
        this.seq = m.seq;
    }

    public void render() {
        System.out.println("Diagram:");
        shapes.values().forEach(s ->
            System.out.println("  " + s.type + "#" + s.id + " ("+s.x+","+s.y+") " + s.color +
" links=" + s.links));
    }
}

// Caretaker
class History {
    private final Deque<DiagramEditor.Memento> undo = new ArrayDeque<>();
    private final Deque<DiagramEditor.Memento> redo = new ArrayDeque<>();

    public void push(DiagramEditor.Memento m) { undo.push(m); redo.clear(); }
    public boolean canUndo() { return !undo.isEmpty(); }
```

```
    public boolean canRedo() { return !redo.isEmpty(); }

    public DiagramEditor.Memento popUndo(DiagramEditor editor) {
        DiagramEditor.Memento m = undo.poll();
        if (m != null) redo.push(editor.save()); // save current for redo before applying
undo
        return m;
    }
    public DiagramEditor.Memento popRedo(DiagramEditor editor) {
        DiagramEditor.Memento m = redo.poll();
        if (m != null) undo.push(editor.save()); // save current for undo before applying
redo
        return m;
    }
}

// Usage
class DemoMementoDiagram {
    public static void main(String[] args) {
        DiagramEditor editor = new DiagramEditor();
        History hist = new History();

        // initial state
        hist.push(editor.save());
        long r1 = editor.add("RECT", 10, 10, "#FF0000");
        long c1 = editor.add("CIRCLE", 50, 50, "#00FF00");
        editor.link(r1, c1);
        hist.push(editor.save());

        editor.move(r1, 5, 5);
        editor.recolor(c1, "#0000FF");
        hist.push(editor.save());

        editor.render(); // changed

        // Undo
        if (hist.canUndo()) { editor.restore(hist.popUndo(editor)); }
        editor.render(); // previous state

        // Redo
        if (hist.canRedo()) { editor.restore(hist.popRedo(editor)); }
        editor.render(); // forward again
    }
}
```

Reasoning to interviewer:

- Editor encapsulates internals; the Memento is opaque outside. Only the editor knows how to serialize/deserialize its state.

- Snapshots carry compact DTOs (no live references), preventing accidental mutation and keeping memory predictable.
- History manages two stacks for multi-level undo/redo; pushing a new action clears the redo stack.
- Copy granularity is per shape DTO—good balance between safety and performance; you can optimize with diffs later.

Future scope:

- Differential mementos (store only changes) to reduce memory for huge diagrams.
- Time-boxed history (cap by count or total bytes).
- Background compression of older snapshots.
- Combine with Command pattern so each action pushes a memento before/after execution for reliable undo/redo.

Scenario 61: State — payment lifecycle (Initialized → Authorized → Captured → Refunded/Failed)

Payments expose different operations depending on where they are in the flow. Instead of a giant switch on status, I'll push behavior into state objects. The Payment context delegates to its current PaymentState. Each concrete state decides what's allowed and what the next state is.

Answer:

I'll define a PaymentState interface with operations like authorize, capture, refund, fail, and status. The Payment context holds a reference to a PaymentState and a few fields (amount, id). State objects call payment.setState(...) to transition. Illegal operations throw a clear exception so the API remains explicit.

Code:

```
// State
interface PaymentState {
    void authorize(Payment p);
    void capture(Payment p);
    void refund(Payment p);
    void fail(Payment p, String reason);
    String status();
}

// Context
class Payment {
    private final String id;
    private final double amount;
    private PaymentState state;

    public Payment(String id, double amount) {
        this.id = id; this.amount = amount;
        this.state = new Initialized();
    }

    // delegate
    public void authorize() { state.authorize(this); }
    public void capture()   { state.capture(this); }
    public void refund()    { state.refund(this); }
    public void fail(String reason) { state.fail(this, reason); }

    void setState(PaymentState s) { this.state = s; }
    public String status() { return state.status(); }
```

```
    // getters if needed
    public String id() { return id; }
    public double amount() { return amount; }
}

// Concrete states
class Initialized implements PaymentState {
    public void authorize(Payment p) { p.setState(new Authorized()); }
    public void capture(Payment p)   { throw new IllegalStateException("Capture
requires Authorized"); }
    public void refund(Payment p)    { throw new IllegalStateException("Nothing to
refund"); }
    public void fail(Payment p, String reason) { p.setState(new Failed(reason)); }
    public String status() { return "INITIALIZED"; }
}

class Authorized implements PaymentState {
    public void authorize(Payment p) { /* idempotent */ }
    public void capture(Payment p)   { p.setState(new Captured()); }
    public void refund(Payment p)    { throw new IllegalStateException("Refund
requires Captured"); }
    public void fail(Payment p, String reason) { p.setState(new Failed(reason)); }
    public String status() { return "AUTHORIZED"; }
}

class Captured implements PaymentState {
    public void authorize(Payment p) { /* no-op */ }
    public void capture(Payment p)   { /* idempotent */ }
    public void refund(Payment p)    { p.setState(new Refunded()); }
    public void fail(Payment p, String reason) { /* capture succeeded; mark dispute
elsewhere */ }
    public String status() { return "CAPTURED"; }
}

class Refunded implements PaymentState {
    public void authorize(Payment p) { throw new IllegalStateException("Refunded
payments cannot be authorized"); }
    public void capture(Payment p)   { throw new IllegalStateException("Refunded
payments cannot be captured"); }
    public void refund(Payment p)    { /* idempotent */ }
    public void fail(Payment p, String reason) { /* no-op */ }
    public String status() { return "REFUNDED"; }
}

class Failed implements PaymentState {
    private final String reason;
    public Failed(String reason) { this.reason = reason; }
    public void authorize(Payment p) { throw new IllegalStateException("Failed
payments cannot be authorized"); }
```

```
    public void capture(Payment p)   { throw new IllegalStateException("Failed
payments cannot be captured"); }
    public void refund(Payment p)    { throw new IllegalStateException("Failed
payments cannot be refunded"); }
    public void fail(Payment p, String r) { /* idempotent */ }
    public String status() { return "FAILED: " + reason; }
}

// Usage
class DemoStatePayment {
    public static void main(String[] args) {
        Payment pay = new Payment("ORD-101", 1499.0);
        System.out.println(pay.status()); // INITIALIZED
        pay.authorize();
        System.out.println(pay.status()); // AUTHORIZED
        pay.capture();
        System.out.println(pay.status()); // CAPTURED
        pay.refund();
        System.out.println(pay.status()); // REFUNDED
    }
}
```

Reasoning to interviewer:

- Behavior sits with state classes; the context is tiny and free of if/else ladders.
- Transitions are explicit and guarded at the source (e.g., capture only from Authorized).
- Idempotency and no-ops are easy to encode per state.
- Adding a new phase (e.g., “Partially Refunded”) is a new state class, not a cross-cutting refactor.

Future scope:

- Emit domain events inside transitions (Authorized→Captured) for downstream systems.
- Persist the state type for resilience, or reconstruct from event history.
- Parameterize operations (partial capture/refund amounts) and validate inside states.
- Share common behavior via an abstract base AbstractPaymentState to reduce repetition.

Scenario 62: Template Method — pluggable import sources under a fixed algorithm

The import flow is fixed: connect → read → map → validate → persist → audit. Sources (CSV, S3 JSON, DB) differ in how they connect/read/map, but the skeleton stays the same.

Answer:

I'll build an abstract ImportJob with a final run() that wires the steps. Subclasses override hook methods: connect(), readRaw(), mapRecords(), validate(), persist(). Common cross-cutting bits (timing, metrics, auditing, error handling) live in the template. This keeps the path identical for all sources while allowing variation where needed.

Code:

```
import java.util.*;

// Template
abstract class ImportJob<T, R> {
    // Template Method
    public final void run() {
        long t0 = System.currentTimeMillis();
        try {
            connect();
            T raw = readRaw();
            List<R> mapped = mapRecords(raw);
            validate(mapped);
            int n = persist(mapped);
            audit(true, n, null);
        } catch (Exception e) {
            audit(false, 0, e.getMessage());
            throw e;
        } finally {
            disconnectQuietly();
            System.out.println("Import took " + (System.currentTimeMillis() - t0) + "
ms");
        }
    }

    // Hooks (override as needed)
    protected abstract void connect();
    protected abstract T readRaw();
    protected abstract List<R> mapRecords(T raw);
    protected void validate(List<R> records) {
        // default: no-op; subclasses can tighten rules
        if (records.isEmpty()) throw new IllegalStateException("No records to import");
    }
    protected abstract int persist(List<R> records);
```

```
    protected void audit(boolean success, int count, String error) {
        System.out.println("AUDIT: success=" + success + " count=" + count + " err="
+ error);
    }
    protected void disconnectQuietly() { /* optional */ }
}

// Concrete CSV
class CsvImportJob extends ImportJob<List<String>, Map<String, String>> {
    protected void connect() { System.out.println("Opening local CSV file..."); }
    protected List<String> readRaw() {
        return List.of("id,name", "1,Ana", "2,Raj");
    }
    protected List<Map<String, String>> mapRecords(List<String> raw) {
        String[] header = raw.get(0).split(",");
        List<Map<String,String>> list = new ArrayList<>();
        for (int i = 1; i < raw.size(); i++) {
            String[] cols = raw.get(i).split(",");
            Map<String,String> row = new HashMap<>();
            for (int c = 0; c < header.length; c++) row.put(header[c], cols[c]);
            list.add(row);
        }
        return list;
    }
    protected int persist(List<Map<String, String>> records) {
        System.out.println("Upserting " + records.size() + " CSV rows");
        return records.size();
    }
}

// Concrete S3 JSON
class S3JsonImportJob extends ImportJob<String, Map<String,Object>> {
    protected void connect() { System.out.println("Connecting to S3..."); }
    protected String readRaw() { return
"[{\"id\":3,\"name\":\"Lee\"},{\"id\":4,\"name\":\"Mia\"}]"; }
    protected List<Map<String,Object>> mapRecords(String json) {
        // Demo parse; in real life use a JSON library
        List<Map<String,Object>> list = new ArrayList<>();
        list.add(Map.of("id", 3, "name", "Lee"));
        list.add(Map.of("id", 4, "name", "Mia"));
        return list;
    }
    protected void validate(List<Map<String,Object>> records) {
        super.validate(records);
        // extra check: all ids present
        boolean ok = records.stream().allMatch(r -> r.containsKey("id"));
        if (!ok) throw new IllegalArgumentException("Missing id");
    }
    protected int persist(List<Map<String,Object>> records) {
        System.out.println("Persisting " + records.size() + " JSON docs");
```

```
        return records.size();
    }
    protected void disconnectQuietly() { System.out.println("Closing S3 client"); }
}

// Concrete DB
class DbTableImportJob extends ImportJob<Void, Map<String,Object>> {
    protected void connect() { System.out.println("Opening JDBC connection..."); }
    protected Void readRaw() { return null; } // we'll stream rows directly
    protected List<Map<String,Object>> mapRecords(Void ignored) {
        return List.of(Map.of("id", 5, "name", "Noah"));
    }
    protected int persist(List<Map<String,Object>> records) {
        System.out.println("Bulk insert into target table: " + records);
        return records.size();
    }
    protected void disconnectQuietly() { System.out.println("Closing JDBC
connection"); }
}

// Usage
class DemoTemplateImport {
    public static void main(String[] args) {
        new CsvImportJob().run();
        new S3JsonImportJob().run();
        new DbTableImportJob().run();
    }
}
```

Reasoning to interviewer:

- The algorithm order is locked by final run(), guaranteeing consistency across sources.
- Variation points are cleanly isolated via hooks (readRaw, mapRecords, validate, persist).
- Common cross-cutting (timing, auditing, error normalization) lives once in the template.
- New sources (Google Sheets, FTP) are new subclasses—no edits to existing logic.

Future scope:

- Add optional hooks (beforePersist, afterPersist) for idempotency and dedupe.
- Support streaming reads (iterator/spliterator) to avoid large in-memory lists.

- Make audit pluggable (send metrics to Prometheus, traces to OpenTelemetry).
- Introduce a small policy object for validation rules, keeping subclasses thin.

Scenario 63: Visitor — extensible cart operations (GST, discounts, shipping) without touching item classes

We've got a fixed hierarchy of cart items (Book, Food, Electronics), but the list of operations on them will keep growing: today it's GST, discounts, shipping; tomorrow it might be analytics, CSV export, fraud scoring. Visitor fits perfectly: item types stay closed, new operations arrive as new visitors.

Answer:

I'll make each item implement CartItem with accept(CartVisitor v). The CartVisitor interface declares an overloaded visit per concrete item. I'll add concrete visitors like GstVisitor, DiscountVisitor, and ShippingVisitor, each carrying its own accumulation state. To compute a cart, we just pass each visitor across all items. When product asks for "analytics/export," we add a new visitor—no edits to Book, Food, or Electronics.

Code:

```
import java.util.*;

// ------------- Element hierarchy (closed for modification) -------------
interface CartItem {
    void accept(CartVisitor v);
    String name();
    double basePrice();
}

class Book implements CartItem {
    private final String title; private final double price; private final boolean
educational;
    public Book(String title, double price, boolean educational) { this.title = title;
this.price = price; this.educational = educational; }
    public boolean isEducational() { return educational; }
    public String name() { return "Book: " + title; }
    public double basePrice() { return price; }
    public void accept(CartVisitor v) { v.visit(this); }
}

class Food implements CartItem {
    private final String label; private final double price; private final boolean
perishable;
    public Food(String label, double price, boolean perishable) { this.label = label;
this.price = price; this.perishable = perishable; }
    public boolean isPerishable() { return perishable; }
    public String name() { return "Food: " + label; }
    public double basePrice() { return price; }
    public void accept(CartVisitor v) { v.visit(this); }
}
```

```
class Electronics implements CartItem {
    private final String model; private final double price; private final int
warrantyMonths;
    public Electronics(String model, double price, int warrantyMonths) { this.model =
model; this.price = price; this.warrantyMonths = warrantyMonths; }
    public int warrantyMonths() { return warrantyMonths; }
    public String name() { return "Electronics: " + model; }
    public double basePrice() { return price; }
    public void accept(CartVisitor v) { v.visit(this); }
}

// ------------- Visitor (open for extension) -------------
interface CartVisitor {
    void visit(Book b);
    void visit(Food f);
    void visit(Electronics e);
}

// GST visitor (example rates)
class GstVisitor implements CartVisitor {
    private double totalGst = 0.0;
    public void visit(Book b) {
        // many regions: educational books GST=0%
        double rate = b.isEducational() ? 0.0 : 0.05; // 5% for non-educational books
(demo)
        totalGst += b.basePrice() * rate;
    }
    public void visit(Food f) {
        double rate = f.isPerishable() ? 0.05 : 0.12; // demo slab
        totalGst += f.basePrice() * rate;
    }
    public void visit(Electronics e) {
        totalGst += e.basePrice() * 0.18; // typical higher slab
    }
    public double totalGst() { return round(totalGst); }
    private static double round(double x){ return Math.round(x*100.0)/100.0; }
}

// Discount visitor (e.g., promo rules)
class DiscountVisitor implements CartVisitor {
    private double totalDiscount = 0.0;
    public void visit(Book b) {
        // 10% promo on books, capped at ₹100 per item (demo)
        double raw = b.basePrice() * 0.10;
        totalDiscount += Math.min(raw, 100.0);
    }
    public void visit(Food f) {
        // no discount on perishables, 5% otherwise
        if (!f.isPerishable()) totalDiscount += f.basePrice() * 0.05;
```

```
    }
    public void visit(Electronics e) {
        // ₹500 off if warranty >= 24 months
        if (e.warrantyMonths() >= 24) totalDiscount += 500.0;
    }
    public double totalDiscount() { return round(totalDiscount); }
    private static double round(double x){ return Math.round(x*100.0)/100.0; }
}

// Shipping visitor (simple logic)
class ShippingVisitor implements CartVisitor {
    private double totalShipping = 0.0;
    public void visit(Book b) {
        totalShipping += 40.0; // flat for books
    }
    public void visit(Food f) {
        totalShipping += f.isPerishable() ? 80.0 : 50.0; // cold-chain cost for perishable
    }
    public void visit(Electronics e) {
        totalShipping += 120.0; // fragile packaging
    }
    public double totalShipping() { return totalShipping; }
}

// Optional: a “composed” visitor that computes a full price breakdown in one pass
class PriceBreakdownVisitor implements CartVisitor {
    private double subtotal = 0, gst = 0, discount = 0, shipping = 0;
    private final GstVisitor gstV = new GstVisitor();
    private final DiscountVisitor discV = new DiscountVisitor();
    private final ShippingVisitor shipV = new ShippingVisitor();

    public void visit(Book b)        { subtotal += b.basePrice(); gstV.visit(b);
discV.visit(b); shipV.visit(b); }
    public void visit(Food f)        { subtotal += f.basePrice(); gstV.visit(f);
discV.visit(f); shipV.visit(f); }
    public void visit(Electronics e) { subtotal += e.basePrice(); gstV.visit(e);
discV.visit(e); shipV.visit(e); }
    public double subtotal()  { return round(subtotal); }
    public double gst()       { return gstV.totalGst(); }
    public double discount()  { return discV.totalDiscount(); }
    public double shipping()  { return shipV.totalShipping(); }
    public double grandTotal(){ return round(subtotal() + gst() + shipping() -
discount()); }
    private static double round(double x){ return Math.round(x*100.0)/100.0; }
}

// ---------- Client ----------
class Cart {
    private final List<CartItem> items = new ArrayList<>();
    public void add(CartItem i) { items.add(i); }
```

```
    public void accept(CartVisitor v) { items.forEach(i -> i.accept(v)); }
}

// Usage
class DemoVisitorCart {
    public static void main(String[] args) {
        Cart cart = new Cart();
        cart.add(new Book("Clean Code", 799, true));
        cart.add(new Food("Olive Oil", 499, false));
        cart.add(new Electronics("Noise-Cancelling Headphones", 5999, 24));

        // Run individual visitors
        GstVisitor gst = new GstVisitor();
        DiscountVisitor disc = new DiscountVisitor();
        ShippingVisitor ship = new ShippingVisitor();
        cart.accept(gst); cart.accept(disc); cart.accept(ship);

        System.out.println("GST:       ₹" + gst.totalGst());
        System.out.println("Discount:  ₹" + disc.totalDiscount());
        System.out.println("Shipping:  ₹" + ship.totalShipping());

        // Or compute a full breakdown in one pass:
        PriceBreakdownVisitor full = new PriceBreakdownVisitor();
        cart.accept(full);
        System.out.println("Subtotal:  ₹" + full.subtotal());
        System.out.println("GST:       ₹" + full.gst());
        System.out.println("Discount:  ₹" + full.discount());
        System.out.println("Shipping:  ₹" + full.shipping());
        System.out.println("Grand:     ₹" + full.grandTotal());
    }
}
```

Reasoning to interviewer:

- Double dispatch via item.accept(visitor) + visitor.visit(concreteType) gives type-specific logic without instanceof chains.
- Items remain stable and simple (data + accept); we add new operations by adding new visitors.
- Each visitor holds its own accumulation state, which makes combining or sequencing operations straightforward.
- The composed PriceBreakdownVisitor shows you can reuse smaller visitors without changing the item model.

Future scope:

- Add AnalyticsVisitor to count category mix, ExportVisitor to produce CSV, or FraudVisitor—no item edits.
- Introduce VisitableCollection for nested carts/bundles; the pattern still applies.
- If visitors explode, consider a “visitor registry” to run selected operations based on feature flags.
- For performance-heavy paths, precompute category indices and let visitors consult them, still without touching item classes.

Scenario 64: Producer–Consumer (BlockingQueue) — back-pressure aware thumbnail pipeline

Kafka can burst; S3 writes and resizes are slower. We need a bounded buffer so producers don't overwhelm memory, and consumers scale without starving other threads.

Answer:

I'll build a small pipeline with:

1. a bounded BlockingQueue<Job> to absorb short spikes,
2. a dedicated, size-limited consumer pool for CPU+IO resize/upload,
3. back-pressure via blocking put() (or offer(.., timeout, ..) with metrics) so producers slow down when the queue is full,
4. graceful shutdown and visibility (metrics on queue depth, rejects, latencies).

If we need non-blocking back-pressure, I'd switch to Flow/Reactive Streams with a Subscriber that requests n items at a time.

Code:

```
import java.util.concurrent.*;
import java.util.*;

record Job(String key, byte[] imageBytes) {}

class ThumbnailService {
    // Bounded queue => back-pressure
    private final BlockingQueue<Job> queue = new ArrayBlockingQueue<>(2_000);

    // Separate pool sized for mixed CPU (resize) + IO (S3)
    private final ThreadPoolExecutor consumers;
    private final ExecutorService producers; // Kafka pollers

    public ThumbnailService(int consumerThreads, int producerThreads) {
        consumers = new ThreadPoolExecutor(
                consumerThreads, consumerThreads,
                0L, TimeUnit.SECONDS,
                new LinkedBlockingQueue<>(), // internal hand-off of runnables
                r -> new Thread(r, "thumb-consumer-" + UUID.randomUUID()),
                new ThreadPoolExecutor.AbortPolicy()   // consumers should not be
overloaded
        );
        producers = Executors.newFixedThreadPool(producerThreads, r -> new
Thread(r, "kafka-poller"));
```

```
    }

    // Producer: called by Kafka poll loop
    public void onKafkaMessage(String key, byte[] image) throws
InterruptedException {
        // back-pressure: block if buffer is full (or use offer with timeout)
        queue.put(new Job(key, image));
    }

    public void start() {
        for (int i = 0; i < consumers.getCorePoolSize(); i++) {
            consumers.submit(this::consumeLoop);
        }
    }

    private void consumeLoop() {
        while (!Thread.currentThread().isInterrupted()) {
            try {
                Job j = queue.take(); // blocks when empty
                process(j);
            } catch (InterruptedException ie) {
                Thread.currentThread().interrupt();
                break;
            } catch (Exception e) {
                // log + DLQ if needed
            }
        }
    }

    private void process(Job j) {
        // 1) resize (CPU) -> ideally use a fast library with bounded threads
        byte[] resized = resize(j.imageBytes());
        // 2) upload (IO) -> S3 client with its own connection pool
        uploadToS3("thumbs/" + j.key(), resized);
    }

    private byte[] resize(byte[] bytes) { /* ... */ return bytes; }
    private void uploadToS3(String key, byte[] data) { /* ... */ }

    public void shutdown() {
        consumers.shutdownNow();
        producers.shutdownNow();
    }
}
```

Reactive Streams flavor (back-pressure pull from consumers):

```
import java.util.concurrent.Flow.*;
```

```
class JobPublisher implements Publisher<Job> {
    private final Iterable<Job> source;
    JobPublisher(Iterable<Job> source){ this.source=source; }
    public void subscribe(Subscriber<? super Job> s) {
        s.onSubscribe(new Subscription() {
            final Iterator<Job> it = source.iterator();
            volatile boolean canceled;
            public void request(long n) {
                long emitted = 0;
                while (emitted < n && !canceled && it.hasNext()) {
                    s.onNext(it.next());
                    emitted++;
                }
                if (!it.hasNext()) s.onComplete();
            }
            public void cancel() { canceled = true; }
        });
    }
}
```

Reasoning to interviewer:

- Bounded BlockingQueue gives natural back-pressure; put() blocks producers during spikes instead of OOM-ing.
- Separate, right-sized consumer pool prevents resize/upload from stealing threads from unrelated parts of the JVM.
- Kafka side: if put() blocks, the poll loop slows, which is okay—Kafka can handle temporary lag; commit offsets after enqueue to avoid loss/dup.
- Visibility: export queue depth, consumer lag, processing time; alarm on sustained high depth.

Future scope:

- Prioritize VIP tenants with multiple queues (e.g., Queue<Job> per tenant + fair poll).
- Use a bounded Semaphore instead of a big queue to cap inflight jobs end-to-end.
- Swap to Reactive Streams end-to-end if we need non-blocking IO and back-pressure (request(n)) through the chain.
- Add a dead-letter path for persistent failures and idempotent uploads.

Scenario 65: Thread Pool / Executor tuning — thousands of short tasks (DB + small CPU)

Most time is I/O wait on DB; CPU per task is tiny. The right size depends on cores, DB pool, and latency.

Answer:

I'd size the executor using the I/O-bound rule of thumb and the DB constraints:

1. Measure one task: average service time = CPU + IO_wait.
2. For I/O-bound pools: threads ≈ cores × (1 + wait/compute). With tiny compute and significant DB wait, this tends to a larger number—but…
3. Hard cap by DB connection pool and rate limits. If the DB pool is 100 connections, spawning 500 threads just time-slices; you'll queue at the DB anyway.

So I usually set:

- DB connection pool ≈ 100 (example),
- executor threads ≈ min(3×cores, 2×DB_pool) but never exceed DB_pool by much unless queries are pipelined or use async I/O,
- bounded queue to smooth bursts,
- CallerRunsPolicy (or custom) to push back to the request thread if overloaded—this raises caller latency instead of dropping memory on the floor.

Code (configurable executor):

```
import java.util.concurrent.*;

class ExecutorsConfig {

    static ThreadPoolExecutor ioBoundPool(int cores, int dbPoolSize) {
        // Example sizing: allow mild over-subscription beyond DB pool
        int core = Math.min(Math.max(cores * 2, 32), dbPoolSize * 2);
        int max  = Math.min(core * 2, dbPoolSize * 3);

        // Bounded queue; choose capacity ≈ a few seconds of work
        int queueCapacity = 10_000;

        return new ThreadPoolExecutor(
            core,
```

```
            max,
            60L, TimeUnit.SECONDS,
            new ArrayBlockingQueue<>(queueCapacity),
            r -> {
                Thread t = new Thread(r);
                t.setName("api-io-" + t.getId());
                t.setDaemon(true);
                return t;
            },
            // Back-pressure to callers when saturated
            new ThreadPoolExecutor.CallerRunsPolicy()
        );
    }
}
```

Proving it under load (what I'd do in practice):

1. Establish a baseline: run a load test (Gatling/JMeter) that generates realistic API traffic.
2. Instrument everything: record per-task timing breakdown (queue wait, executor execution time, DB round-trip), pool gauges (getActiveCount(), getPoolSize(), queue size, rejection count).
3. Sweep settings: vary core, max, and queue capacity; find the "knee of the curve" where p95 latency stops improving but CPU/DB hit saturation.
4. Validate stability: long-run (30–60 min) with jitter, ensure no unbounded queue growth, no GC pressure spikes, and zero starvation (watch other thread pools and CPU steal).
5. Guardrails in prod:
    - Circuit breaker on DB latency spikes.
    - Timeouts on futures.
    - Bulkhead: keep this executor separate from CPU-heavy pools (e.g., image processing) to prevent starvation.
    - SLO alerts on p95/p99, queue depth, and rejection rate.

Queue choice & rejection policy:

- Queue: ArrayBlockingQueue (bounded, lower GC) for predictable back-pressure; LinkedBlockingQueue only if you really need larger capacity (still bounded).

- Rejection:
    - CallerRunsPolicy pushes back immediately (good when the caller can afford extra latency and you want natural throttling).
    - AbortPolicy if you prefer to fail fast (pair with clear 503s and retry headers).
    - Custom policy that drops low-priority tasks first if you classify work.

Small CPU inside tasks:

- Keep CPU hogs out of this pool; use a dedicated fixed pool for CPU work sized ≈ number of cores.
- If DB is the bottleneck, raising maxPoolSize above the DB connection count rarely helps; it increases contention. Instead, improve SQL and add caching.

Future scope:

- Switch to async DB client (R2DBC/driver async) and use fewer threads with higher concurrency (CompletableFuture/reactive), then tune with event-loop + small worker pool.
- Adaptive sizing: adjust pool sizes based on observed queue time and DB latency (PID-like controller).
- Per-tenant queues/executors to isolate noisy neighbors.
- Add a token-bucket limiter before enqueuing to cap worst-case queuing delay.

Scenario 66: Read–Write Lock — hot catalog: many reads, few writes

Catalog data is read on every product page, while updates (price change, inventory sync) are occasional. We want cheap concurrent reads but strictly serialized writes.

Answer

I'd choose ReentrantReadWriteLock when:

- Reads dominate (≫ 80%) and are independent.
- Critical section is non-trivial (copying maps, filtering), so a single synchronized lock would bottleneck.
- We can structure code to avoid "read→write upgrade".

If the section is tiny or updates are frequent, synchronized is simpler and often faster due to JVM optimizations (biased/Lightweight locking). For starvation: use a fair lock (new ReentrantReadWriteLock(true)) so queued writers don't starve behind a read stampede. Prefer lock downgrading (write→read) over upgrading (read→write, which is unsafe).

Code

```
import java.util.*;
import java.util.concurrent.locks.*;

class HotCatalog {
    private final Map<String, String> products = new HashMap<>(); // id -> name
    private final ReentrantReadWriteLock rw = new ReentrantReadWriteLock(true);
// fair to prevent writer starvation
    private final Lock r = rw.readLock();
    private final Lock w = rw.writeLock();

    // Fast path: many concurrent readers
    public String getName(String id) {
        r.lock();
        try { return products.get(id); }
        finally { r.unlock(); }
    }

    // Occasional writer: exclusive
    public void putOrUpdate(String id, String name) {
        w.lock();
        try { products.put(id, name); }
        finally { w.unlock(); }
    }
```

```
    // Compute-if-absent without read→write upgrade pitfalls
    public String getOrLoad(String id, java.util.function.Function<String, String>
loader) {
        String val;
        r.lock();
        try {
            val = products.get(id);
            if (val != null) return val;
        } finally { r.unlock(); }

        // Acquire write; double-check
        w.lock();
        try {
            val = products.get(id);
            if (val == null) {
                val = loader.apply(id);
                products.put(id, val);
            }
            // Downgrade: keep a read lock while releasing write if you need to
continue reading
            // r.lock(); // optional
        } finally {
            // if you took r.lock() above, release write first, then r later
            w.unlock();
        }
        return val;
    }

    // Snapshot read with copy-on-write technique (shorter write hold)
    public Map<String,String> snapshot() {
        r.lock();
        try { return Map.copyOf(products); }  // small, immutable snapshot
        finally { r.unlock(); }
    }
}
```

Reasoning to interviewer

- ReentrantReadWriteLock lets many readers proceed concurrently, which matches the read-heavy workload; synchronized would serialize them.
- Fair mode avoids the classic “writer starvation under constant read traffic.”
- No read→write upgrade: we exit the read lock before taking the write lock, and we double-check inside write to avoid redundant work (check-then-act pattern).

- For short reads or frequent writes, the overhead of RW locks can exceed benefits—then plain synchronized (or StampedLock for optimistic reads) is better.

Future scope

- Consider StampedLock for optimistic reads on extremely hot paths (retry on validation failure).
- Shard the catalog by key space into N locks to reduce write contention.
- Expose metrics: read/write hold time, contention, queued writers; flip fair/unfair based on data.
- If updates are batched, use copy-on-write (build new map, single write swap) to minimize write lock time.

Scenario 67: CompletableFuture Composition — fan-out to 3 services with timeouts/fallbacks

We call Inventory, Pricing, and Reviews in parallel, combine the results, and treat slow/failed calls with timeouts and fallbacks. Each service should be isolated so one pool's saturation can't stall the others.

Answer

I'll submit each call to its own ExecutorService (bulkhead isolation). Use supplyAsync to fan out, compose with thenCombine/allOf, guard with orTimeout/completeOnTimeout, and attach exceptionally/handle for fallbacks. If any one is optional, wrap with completeOnTimeout + default value; if mandatory, fail fast with orTimeout.

Code

```
import java.time.Duration;
import java.util.concurrent.*;
import java.util.function.Supplier;

// DTOs
record Inventory(int qty) {}
record Price(double amount) {}
record Reviews(double rating) {}
record ProductView(Inventory inv, Price price, Reviews rev) {}

// Bulkhead executors (tune sizes per service)
class Pools {
    static final Executor INV_POOL = new ThreadPoolExecutor(16, 32, 60,
TimeUnit.SECONDS,
            new ArrayBlockingQueue<>(1000), new
ThreadPoolExecutor.CallerRunsPolicy());
    static final Executor PRI_POOL = new ThreadPoolExecutor(16, 32, 60,
TimeUnit.SECONDS,
            new ArrayBlockingQueue<>(1000), new
ThreadPoolExecutor.CallerRunsPolicy());
    static final Executor REV_POOL = new ThreadPoolExecutor(8,  16, 60,
TimeUnit.SECONDS,
            new ArrayBlockingQueue<>(500),  new
ThreadPoolExecutor.CallerRunsPolicy());
}

class Services {
    static Inventory fetchInv(String sku) { sleep(80); return new Inventory(12); }
    static Price fetchPrice(String sku)   { sleep(120); return new Price(1999.0); }
    static Reviews fetchReviews(String s) { sleep(350); return new Reviews(4.6); }
    static void sleep(long ms){ try { Thread.sleep(ms); } catch (InterruptedException
ignored) {} }
}
```

```
class ProductFacade {
   private static final Duration MANDATORY_TIMEOUT = Duration.ofMillis(300);
   private static final Duration OPTIONAL_TIMEOUT  = Duration.ofMillis(250);

   public CompletableFuture<ProductView> view(String sku) {
      CompletableFuture<Inventory> invF = supply("inv", () ->
Services.fetchInv(sku), Pools.INV_POOL)
            .orTimeout(MANDATORY_TIMEOUT.toMillis(),
TimeUnit.MILLISECONDS); // must succeed

      CompletableFuture<Price> priceF = supply("price", () ->
Services.fetchPrice(sku), Pools.PRI_POOL)
            .orTimeout(MANDATORY_TIMEOUT.toMillis(),
TimeUnit.MILLISECONDS);

      // Reviews are optional: timeout -> fallback
      CompletableFuture<Reviews> revF = supply("rev", () ->
Services.fetchReviews(sku), Pools.REV_POOL)
            .completeOnTimeout(new Reviews(0.0), OPTIONAL_TIMEOUT)          //
default if slow
            .exceptionally(ex -> new Reviews(0.0));                         // default on
error

      // Combine all three
      return invF.thenCombine(priceF, ProductView::new) // (Inventory, Price) ->
partial
               .thenCombine(revF, (partial, reviews) ->
                  new ProductView(partial.inv(), partial.price(), reviews))
               .exceptionally(ex -> {
                  // If mandatory paths failed, provide a clear error or fallback strategy
                  throw new CompletionException(new RuntimeException("Product
view failed", ex));
               });
   }

   private static <T> CompletableFuture<T> supply(
         String name, Supplier<T> supplier, Executor pool) {
      return CompletableFuture.supplyAsync(() -> {
         try { return supplier.get(); }
         catch (Exception e) { throw new CompletionException(e); }
      }, pool);
   }
}
```

Using anyOf (race) for redundant endpoints and allOf for “wait for all”:

```
// Race two pricing mirrors; take the first
CompletableFuture<Price> fastPrice =
```

```
    CompletableFuture.anyOf(
        supply("priceA", () -> Services.fetchPrice("SKU1"), Pools.PRI_POOL),
        supply("priceB", () -> Services.fetchPrice("SKU1"), Pools.PRI_POOL)
    ).thenApply(Price.class::cast)
     .orTimeout(250, TimeUnit.MILLISECONDS);

// Wait for multiple optional enrichments (tags, recommendations)
CompletableFuture<Void> allOptional =
    CompletableFuture.allOf(revF, fastPrice.thenApply(p -> null));
```

Timeout helpers (JDK 9+ has orTimeout/completeOnTimeout; for JDK 8 you can emulate with a scheduler). Rejections in pools are handled via CallerRunsPolicy for back-pressure; you could also AbortPolicy and short-circuit with a fast error.

Reasoning to interviewer

- Fan-out with separate executors prevents head-of-line blocking (pricing surge won’t stall inventory).
- orTimeout enforces SLOs on mandatory services; completeOnTimeout gives graceful defaults for optional ones.
- anyOf lets you race mirrors or CDNs; allOf gates on multiple futures.
- Failures are contained: each future handles its own fallback; the aggregator composes them deterministically.

Future scope

- Add circuit breakers/bulkheads per service (resilience4j) and jittered retries for transient errors.
- Structured logging/tracing (handle to record duration, cause) and metrics per service future.
- Cache optional results (e.g., reviews) and short-circuit when stale-ok.
- Move to virtual threads (Project Loom) to simplify executors where latency is I/O-bound.

Scenario 68: Dependency Injection vs Service Locator — refactor legacy static registry

Legacy code reaches into a static ServiceLocator.get("mailer") from deep utility classes. That hides dependencies, hurts testability, and couples everything to a global. I'll refactor to constructor injection so dependencies are explicit and easily mocked.

Answer

Replace global lookups with interfaces injected via constructors (or method parameters for pure functions). Composition/wiring happens at the edges (main/DI framework), not inside utilities. This makes dependencies visible, enforces compile-time safety, and simplifies unit tests.

Code:

```
// BEFORE (Service Locator anti-pattern)
class ServiceLocator {
    private static final Map<String,Object> REG = new HashMap<>();
    public static <T> void register(String key, T svc) { REG.put(key, svc); }
    @SuppressWarnings("unchecked")
    public static <T> T get(String key) { return (T) REG.get(key); }
}

interface Mailer { void send(String to, String msg); }

class OrderUtil {
    // Hidden dependency: hard to test
    public static void notifyCustomer(String email, String text) {
        Mailer mailer = ServiceLocator.get("mailer"); // global reach
        mailer.send(email, text);
    }
}

// AFTER (Constructor Injection)
class OrderNotifier {
    private final Mailer mailer; // explicit dependency
    public OrderNotifier(Mailer mailer) { this.mailer = mailer; }
    public void notifyCustomer(String email, String text) { mailer.send(email, text); }
}

// Wiring (composition root)
class App {
    public static void main(String[] args) {
        Mailer smtp = (to, msg) -> System.out.println("SMTP -> " + to + ": " + msg);
```

```
        OrderNotifier notifier = new OrderNotifier(smtp);
        notifier.notifyCustomer("rajat@example.com", "Your order shipped");
    }
}

// Test becomes trivial
class OrderNotifierTest {
    static class FakeMailer implements Mailer {
        String lastTo, lastMsg;
        public void send(String to, String msg) { lastTo = to; lastMsg = msg; }
    }
    public static void main(String[] args) {
        FakeMailer fake = new FakeMailer();
        OrderNotifier n = new OrderNotifier(fake);
        n.notifyCustomer("t@e.com", "hi");
        assert "t@e.com".equals(fake.lastTo) && "hi".equals(fake.lastMsg);
    }
}
```

Reasoning to interviewer

- Service Locator hides dependencies and can fail late at runtime; DI is explicit and fails fast at compile/wire time.
- DI improves testability: pass fakes/mocks without touching global state.
- No hidden coupling or order-of-registration issues; utilities stop being “reach into the world” functions.
- Dependency graphs are clear; tools/IDE can navigate and enforce them.

Future scope

- Introduce a DI container (Spring/Guice/Dagger) at the composition root for lifecycle, scopes, and conditional bindings.
- Prefer constructor injection; use method/field injection only for optional/rare cases.
- Add interfaces for cross-cutting concerns (logging, metrics) and inject them too; avoid more singletons.

Scenario 69: Repository / DAO — Postgres + Redis behind a clean API

We want to hide persistence details from callers and make testing easy. Reads should use Redis as a cache; Postgres is the source of truth.

Answer

Define a UserRepository interface for use cases (find/save). Implement a caching repository that composes two DAOs: UserPgDao and UserRedisDao. Use read-through/write-through (or write-behind if needed) and TTLs. For tests, plug in in-memory fakes without touching the real DB/Redis.

Code:

```
import java.util.*;

// Domain
record User(String id, String name, int points) {}

// Repository (use-case oriented)
interface UserRepository {
    Optional<User> findById(String id);
    void save(User user);
    void delete(String id);
}

// DAOs per store
interface UserPgDao { Optional<User> select(String id); void upsert(User u); void
delete(String id); }
interface UserRedisDao { Optional<User> get(String id); void set(String id, User u,
long ttlSeconds); void del(String id); }

// Concrete repository with caching
class CachingUserRepository implements UserRepository {
    private final UserPgDao pg;
    private final UserRedisDao redis;
    private final long ttlSec;

    public CachingUserRepository(UserPgDao pg, UserRedisDao redis, long ttlSec)
{
        this.pg = pg; this.redis = redis; this.ttlSec = ttlSec;
    }

    public Optional<User> findById(String id) {
        // 1) Try cache
        Optional<User> cached = redis.get(id);
        if (cached.isPresent()) return cached;

        // 2) Fallback to Postgres, then populate cache
```

```
        Optional<User> db = pg.select(id);
        db.ifPresent(u -> redis.set(id, u, ttlSec));
        return db;
    }

    public void save(User user) {
        // write-through
        pg.upsert(user);
        redis.set(user.id(), user, ttlSec); // update cache eagerly
    }

    public void delete(String id) {
        pg.delete(id);
        redis.del(id);
    }
}

// --- Example DAOs (real impls would use JDBC & Jedis/Lettuce) ---
class PgDaoImpl implements UserPgDao {
    private final Map<String, User> table = new HashMap<>(); // demo only
    public Optional<User> select(String id) { return
Optional.ofNullable(table.get(id)); }
    public void upsert(User u) { table.put(u.id(), u); }
    public void delete(String id) { table.remove(id); }
}

class RedisDaoImpl implements UserRedisDao {
    private final Map<String, User> cache = new HashMap<>(); // demo only
    public Optional<User> get(String id) { return Optional.ofNullable(cache.get(id)); }
    public void set(String id, User u, long ttl) { cache.put(id, u); /* ignore TTL in
demo */ }
    public void del(String id) { cache.remove(id); }
}

// Usage
class DemoRepo {
    public static void main(String[] args) {
        UserPgDao pg = new PgDaoImpl();
        UserRedisDao redis = new RedisDaoImpl();
        UserRepository repo = new CachingUserRepository(pg, redis, 300);

        repo.save(new User("u1", "Anita", 120));
        System.out.println(repo.findById("u1")); // cache miss -> DB -> cache
        System.out.println(repo.findById("u1")); // cache hit
        repo.delete("u1");
        System.out.println(repo.findById("u1")); // empty
    }
}
```

Reasoning to interviewer

- Callers depend on UserRepository, not on JDBC/Redis clients—clean separation.
- The repository composes DAOs to implement read-through/write-through; cache invalidation is localized.
- Testing is trivial: inject in-memory fakes (as above) or mocks. No containers required for unit tests.
- You can add cross-cutting (metrics, tracing, retries) around the repo without leaking storage concerns upward.

Future scope

- Add TTLs, cache stampede protection (single-flight), and negative caching for “not found”.
- Support partial failures: on Redis outage, operate DB-only; on DB outage, serve stale cache if policy allows.
- Introduce specifications or query objects for richer reads (findByEmail, paged queries).
- Split writes for CQRS: command repository (Postgres) and read model (Redis/materialized views) if read patterns diverge.

Scenario 70: Template Method vs Strategy in Spring — fixed payment skeleton + pluggable risk checks

A payment has a predictable pipeline (validate → reserve/authorize → capture/record → notify), but the "risk check" policy varies by market/merchant (basic rules, device fingerprinting, ML score). We'll keep the skeleton stable with Template Method and plug risk as a Strategy. Spring wires the pieces cleanly via constructor injection and bean selection.

Answer:

- Use Template Method for the payment "journey" that should run in the same order across all payment types. Subclasses change only the steps that truly differ (e.g., how to authorize a Card vs UPI).
- Use Strategy for risk evaluation because policies change frequently and you want to swap them at runtime (per-tenant/region/feature flag) without touching the flow code.
- Spring wiring: each PaymentProcessor (template) is a bean. Risk policies are separate beans implementing RiskCheck. Inject by qualifier/primary, or select from a Map<String, RiskCheck> at runtime.

Code (Spring Boot style):

```
// Strategy: pluggable risk policy
public interface RiskCheck {
    RiskDecision evaluate(PaymentContext ctx);
}

public record RiskDecision(boolean approved, String reason) {}

@Component("basicRisk")
class BasicRiskCheck implements RiskCheck {
    @Override public RiskDecision evaluate(PaymentContext ctx) {
        if (ctx.amount() > 50_000 && !ctx.deviceTrusted()) return new
RiskDecision(false, "High amount, untrusted device");
        return new RiskDecision(true, "OK");
    }
}

@Component("mlRisk")
class MlRiskCheck implements RiskCheck {
    private final RiskModelClient model; // imagine an HTTP/GRPC client
```

```
    MlRiskCheck(RiskModelClient model) { this.model = model; }

    @Override public RiskDecision evaluate(PaymentContext ctx) {
        double score = model.score(ctx); // 0..1
        return new RiskDecision(score < 0.8, "Score=" + score);
    }
}

// Template Method: fixed skeleton, overridable hooks
public abstract class PaymentProcessor {

    private final RiskRouter riskRouter; // chooses a RiskCheck strategy
    protected PaymentProcessor(RiskRouter riskRouter) { this.riskRouter =
riskRouter; }

    // Template method - DO NOT override
    public final PaymentResult process(PaymentContext ctx) {
        preValidate(ctx);
        RiskDecision risk = riskRouter.choose(ctx).evaluate(ctx);
        if (!risk.approved()) return PaymentResult.rejected("Risk: " + risk.reason());
        String authId = authorize(ctx);      // varies by instrument
        capture(ctx, authId);                // may differ or be a no-op for UPI
        persist(ctx, authId);                // common persistence
        notifyCustomer(ctx);                 // common
        return PaymentResult.success(authId);
    }

    // Common default hooks
    protected void preValidate(PaymentContext ctx) {
        if (ctx.amount() <= 0) throw new IllegalArgumentException("Invalid amount");
    }
    protected void persist(PaymentContext ctx, String authId) { /* save to DB */ }
    protected void notifyCustomer(PaymentContext ctx) { /* email/webhook */ }

    // Abstract hooks that vary
    protected abstract String authorize(PaymentContext ctx);
    protected abstract void capture(PaymentContext ctx, String authId);
}

// Concrete templates per instrument
@Service("cardProcessor")
class CardPaymentProcessor extends PaymentProcessor {
    private final CardGateway gateway;
    CardPaymentProcessor(RiskRouter rr, CardGateway gw) { super(rr);
this.gateway = gw; }

    @Override protected String authorize(PaymentContext ctx) {
        return gateway.auth(ctx.cardDetails(), ctx.amount()); // returns authId
    }
    @Override protected void capture(PaymentContext ctx, String authId) {
```

```
        gateway.capture(authId, ctx.amount());
    }
}

@Service("upiProcessor")
class UpiPaymentProcessor extends PaymentProcessor {
    private final UpiGateway gateway;
    UpiPaymentProcessor(RiskRouter rr, UpiGateway gw) { super(rr); this.gateway
= gw; }

    @Override protected String authorize(PaymentContext ctx) {
        return gateway.collectRequest(ctx.vpa(), ctx.amount()); // intent/collect flow id
    }
    @Override protected void capture(PaymentContext ctx, String authId) {
        // UPI often completes asynchronously; maybe poll/confirm instead
    }
}

// Context/Result
public record PaymentContext(String merchantId, String region, long amount,
                             boolean deviceTrusted, CardDetails cardDetails, String vpa,
String method) {}
public record PaymentResult(boolean ok, String messageOrAuth) {
    static PaymentResult success(String id) { return new PaymentResult(true, id); }
    static PaymentResult rejected(String msg){ return new PaymentResult(false,
msg); }
}

// Strategy selection (runtime routing of RiskCheck)
@Component
class RiskRouter {
    private final Map<String, RiskCheck> policies; // bean name -> strategy

    RiskRouter(Map<String, RiskCheck> policies) { this.policies = policies; }

    public RiskCheck choose(PaymentContext ctx) {
        // e.g., per-merchant config from DB/Config Server/Feature Flag
        String key = switch (ctx.region()) {
            case "IN" -> "mlRisk";
            default   -> "basicRisk";
        };
        return policies.getOrDefault(key, policies.get("basicRisk"));
    }
}

// Selecting the right processor (factory)
@Component
class PaymentFacade {
    private final Map<String, PaymentProcessor> processors;
```

```
    PaymentFacade(Map<String, PaymentProcessor> processors) { this.processors
= processors; }

    public PaymentResult pay(PaymentContext ctx) {
        PaymentProcessor p = switch (ctx.method()) {
            case "CARD" -> processors.get("cardProcessor");
            case "UPI"  -> processors.get("upiProcessor");
            default     -> throw new IllegalArgumentException("Unsupported method");
        };
        return p.process(ctx);
    }
}
```

Reasoning to interviewer:

- Template Method locks the high-level payment journey so compliance/auditing isn't jeopardized by ad-hoc flows. Each instrument customizes only authorize/capture (and any optional hooks), keeping behavior localized.
- Risk is a classic Strategy: volatile policy, multiple implementations, safe to swap per merchant/region A/B test.
- Wiring with Spring:
    - Strategies are beans; we inject a Map<String, RiskCheck> to choose at runtime (bulkhead against hard-coded qualifiers).
    - Processors are beans that depend on a RiskRouter (composition), not on a particular RiskCheck.
    - A small facade picks a processor by method; no if-else sprawl inside business code.
- Testing is simple:
    - Unit-test CardPaymentProcessor by injecting a stub RiskRouter and a fake CardGateway.
    - Unit-test RiskRouter with a map of fake strategies and table-driven cases for region/tenant selection.
    - Integration-test the full graph with Spring's slice tests and a config that flips strategies.

Future scope:

- Add hook methods (beforeCapture, afterCapture) in the template for 3DS, SCA, or ledger posting; default to no-op.
- Externalize risk routing to a config service and cache decisions; add circuit breakers per risk backend.
- Use @ConditionalOnProperty to switch default RiskCheck in different environments.
- Emit domain events inside the template (authorized, captured) for observability and downstream ledgers.

Scenario 71: Circuit Breaker / Retry / Timeout — stabilizing a flaky Price service

When price calls intermittently stall or fail, checkout latency spikes. I'll wrap the outbound call with: a strict per-call timeout, bounded retries with exponential backoff + jitter, and a circuit breaker (open/half-open/closed) to shed load when the dependency is unhealthy. I'll also isolate execution on a dedicated pool so stuck price calls don't block unrelated work.

Answer

- Timeout first (e.g., 200–300 ms) so the tail doesn't poison p95/p99.
- Retry only idempotent, transient errors (timeouts/5xx), never 4xx; cap attempts (e.g., 2) with backoff + jitter.
- Circuit breaker opens on rolling failure rate/slow-calls; in half-open it probes a few requests before closing.
- Bulkhead: use a small dedicated executor and queue for price calls to avoid head-of-line blocking.
- Expose metrics for each policy stage; alert on failure rate, slow-call rate, open duration, and fallback rate.

Code (vanilla Java + simple breaker; swap with Resilience4j in prod):

```
import java.time.Duration;
import java.util.Random;
import java.util.concurrent.*;
import java.util.function.Supplier;

class PriceDto { final double amount; PriceDto(double a){ amount=a; } }
class PriceClient {
    private final ExecutorService pool = new ThreadPoolExecutor(16, 16, 0L,
TimeUnit.SECONDS,
            new ArrayBlockingQueue<>(1000), new
ThreadPoolExecutor.CallerRunsPolicy());

    private final CircuitBreaker cb = new CircuitBreaker(0.5, 0.5, 20,
Duration.ofSeconds(30)); // 50% fail/slow on 20 samples -> open 30s
    private final Random rnd = new Random();

    public CompletableFuture<PriceDto> getPrice(String sku) {
        Supplier<PriceDto> call = () -> {
            // Simulated flaky dependency
            sleep(80 + rnd.nextInt(250));
            if (rnd.nextDouble() < 0.2) throw new RuntimeException("Upstream 5xx");
            return new PriceDto(1999.0);
```

```
        };

        return cb.execute(() -> retryWithBackoff(
                () -> withTimeout(call, Duration.ofMillis(250)), // hard timeout
                2,                  // max retries
                Duration.ofMillis(50),
                Duration.ofMillis(200)))
            .exceptionally(ex -> { /* fallback policy (e.g., cached price or error) */ return
new PriceDto(-1); });
    }

    private <T> CompletableFuture<T> withTimeout(Supplier<T> s, Duration
timeout) {
        return CompletableFuture.supplyAsync(s, pool)
                .orTimeout(timeout.toMillis(), TimeUnit.MILLISECONDS);
    }

    private static <T> CompletableFuture<T> retryWithBackoff(
            Supplier<CompletableFuture<T>> attempt,
            int maxRetries,
            Duration baseDelay,
            Duration maxDelay) {

        CompletableFuture<T> result = new CompletableFuture<>();
        Random jitter = new Random();

        class Retry {
            int tries = 0;
            void go() {
                attempt.get().whenComplete((val, err) -> {
                    if (err == null) { result.complete(val); return; }
                    if (tries++ >= maxRetries || isNonRetryable(err)) {
                        result.completeExceptionally(err); return;
                    }
                    long backoff = Math.min(
                        (long)(baseDelay.toMillis() * Math.pow(2, tries - 1)),
                        maxDelay.toMillis());
                    long jitterMs = (long)(backoff * (0.5 + jitter.nextDouble() * 0.5)); // 50–
100% jitter
                    scheduler().schedule(this::go, jitterMs, TimeUnit.MILLISECONDS);
                });
            }
        }
        new Retry().go();
        return result;
    }

    private static boolean isNonRetryable(Throwable err) {
        Throwable c = err instanceof CompletionException ? err.getCause() : err;
```

```
        // Example: treat timeouts and RuntimeException as retryable; 4xx would be
non-retryable in real code
        return false;
    }

    private static ScheduledExecutorService scheduler() {
        return LazyScheduler.HOLDER;
    }
    private static class LazyScheduler { static final ScheduledExecutorService
HOLDER =
        Executors.newSingleThreadScheduledExecutor(r -> { Thread t = new
Thread(r, "retry-timer"); t.setDaemon(true); return t; }); }

    private static void sleep(long ms){ try { Thread.sleep(ms); } catch
(InterruptedException ignored) {} }
}

// Minimal circuit breaker for demo (closed → open → half-open)
class CircuitBreaker {
    enum State { CLOSED, OPEN, HALF_OPEN }
    private volatile State state = State.CLOSED;
    private final double failThreshold;     // e.g., 0.5
    private final double slowThreshold;     // e.g., 0.5 slow calls
    private final int windowSize;           // e.g., last 20 calls
    private final Duration openFor;
    private final long slowCallMillis = 300;

    private final ConcurrentLinkedQueue<Boolean> outcomes = new
ConcurrentLinkedQueue<>();
    private volatile long reopenAt = 0L;

    CircuitBreaker(double failThreshold, double slowThreshold, int windowSize,
Duration openFor) {
        this.failThreshold = failThreshold;
        this.slowThreshold = slowThreshold;
        this.windowSize = windowSize;
        this.openFor = openFor;
    }

    public <T> CompletableFuture<T> execute(Supplier<CompletableFuture<T>>
supplier) {
        long now = System.currentTimeMillis();
        if (state == State.OPEN) {
            if (now < reopenAt) return failed(new RejectedExecutionException("CB
open"));
            // half-open probe
            state = State.HALF_OPEN;
        }
        long start = now;
        CompletableFuture<T> f = supplier.get();
```

```
        return f.whenComplete((v, e) -> {
            boolean failed = e != null;
            boolean slow = (System.currentTimeMillis() - start) > slowCallMillis;
            recordOutcome(!failed && !slow, slow, failed);
        });
    }

    private void recordOutcome(boolean ok, boolean slow, boolean failed) {
        outcomes.add(!(failed || slow)); // success if not failed and not slow
        while (outcomes.size() > windowSize) outcomes.poll();

        long fails = outcomes.stream().filter(b -> !b).count();
        double failRate = (windowSize == 0) ? 0 : (double) fails / outcomes.size();
        double slowRate = 0; // simplified: treat slow as failure above; prod would
track separately

        if (state == State.CLOSED && (failRate >= failThreshold || slowRate >=
slowThreshold)) {
            state = State.OPEN; reopenAt = System.currentTimeMillis() +
openFor.toMillis();
        } else if (state == State.HALF_OPEN) {
            if (!ok) { state = State.OPEN; reopenAt = System.currentTimeMillis() +
openFor.toMillis(); }
            else { state = State.CLOSED; outcomes.clear(); }
        }
    }

    private static <T> CompletableFuture<T> failed(Throwable t) {
        CompletableFuture<T> f = new CompletableFuture<>();
f.completeExceptionally(t); return f;
    }
}
```

Reasoning to interviewer

- Timeout caps worst-case latency; retries are bounded and jittered to avoid thundering herds.
- Circuit breaker protects the service and the dependency during sustained failures and lets it recover via half-open probes.
- Bulkhead executor prevents a stuck price tier from starving other outbound calls.
- Clear fallback path keeps checkout functional (e.g., cached/prior price) with visible degradation.

Metrics & alerts I'd expose

- Per endpoint: success rate, error rate by cause, timeout count, retry count/attempts histogram, fallback count.
- Circuit breaker: state (open/half-open/closed), transitions, open duration, slow-call rate.
- Latency: p50/p90/p99 end-to-end and broken down (queue wait, execution, upstream).
- Pool health: active threads, queue depth, rejections.
- Alerts: breaker stuck open > N minutes, fallback rate > X%, p99 > SLO for Y minutes, retries > baseline.

Future scope

- Replace demo breaker with Resilience4j (TimeLimiter, Retry, CircuitBreaker, Bulkhead) and bind to Micrometer.
- Per-tenant configs for limits; dynamic config via a central store.
- Stale-while-revalidate cache for prices to mask short outages.

Scenario 72: Bulkhead & Rate Limiter — isolating a noisy tenant

One client floods the API. I'll isolate tenants with separate bulkheads (thread pools/queues) and cap their QPS, so others stay healthy. If a tenant exceeds limits, they get 429s or queued delay—never at the expense of neighbors.

Answer

- Bulkhead per tenant: a ThreadPoolExecutor (or semaphore) and bounded queue per tenant ID.
- Rate limiter: token bucket per tenant (QPS + burst). Exceeding tokens → immediate 429 or controlled delay.
- Global guardrails: overall max concurrency to protect the node.
- Prioritization: premium tenants get larger pools/bursts.

Code (per-tenant pool + token bucket):

```
import java.time.Instant;
import java.util.Map;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicLong;

class TenantLimits {
    final int core; final int max; final int queue; final double qps; final int burst;
    TenantLimits(int core, int max, int queue, double qps, int burst) {
        this.core=core; this.max=max; this.queue=queue; this.qps=qps;
this.burst=burst;
    }
}

class TenantBulkhead {
    final ThreadPoolExecutor exec;
    final TokenBucket limiter;

    TenantBulkhead(TenantLimits lim) {
        this.exec = new ThreadPoolExecutor(lim.core, lim.max, 30,
TimeUnit.SECONDS,
                new ArrayBlockingQueue<>(lim.queue),
                r -> { Thread t = new Thread(r, "tenant-pool"); t.setDaemon(true); return
t; },
                new ThreadPoolExecutor.AbortPolicy()); // reject when saturated
        this.limiter = new TokenBucket(lim.qps, lim.burst);
    }
}

class TokenBucket {
    private final double ratePerSec; // refill rate
```

```
    private final int burst;
    private double tokens;
    private long lastRefillNanos = System.nanoTime();

    TokenBucket(double ratePerSec, int burst) {
        this.ratePerSec = ratePerSec; this.burst = burst; this.tokens = burst;
    }

    synchronized boolean tryAcquire() {
        long now = System.nanoTime();
        double refill = (now - lastRefillNanos) / 1_000_000_000.0 * ratePerSec;
        if (refill > 0) {
            tokens = Math.min(burst, tokens + refill);
            lastRefillNanos = now;
        }
        if (tokens >= 1.0) { tokens -= 1.0; return true; }
        return false;
    }
}

class BulkheadGateway {
    private final ConcurrentHashMap<String, TenantBulkhead> tenants = new
ConcurrentHashMap<>();
    private final AtomicLong rejectedQps = new AtomicLong();

    public void registerTenant(String id, TenantLimits limits) {
        tenants.put(id, new TenantBulkhead(limits));
    }

    public <T> CompletableFuture<T> submit(String tenantId, Callable<T> task) {
        TenantBulkhead bh = tenants.get(tenantId);
        if (bh == null) return failed(new IllegalArgumentException("unknown tenant"));

        if (!bh.limiter.tryAcquire()) { // rate limit
            rejectedQps.incrementAndGet();
            return failed(new RejectedExecutionException("429 Too Many
Requests"));
        }
        CompletableFuture<T> f = new CompletableFuture<>();
        try {
            bh.exec.submit(() -> {
                try { f.complete(task.call()); }
                catch (Exception e) { f.completeExceptionally(e); }
            });
        } catch (RejectedExecutionException rex) {
            rejectedQps.incrementAndGet();
            return failed(new RejectedExecutionException("503 Queue full for
tenant"));
        }
        return f;
```

```
    }

    public Map<String, Object> snapshot() {
        return Map.of(
            "time", Instant.now().toString(),
            "tenants", tenants.size(),
            "rejected", rejectedQps.get()
        );
    }

    private static <T> CompletableFuture<T> failed(Throwable t) {
        CompletableFuture<T> f = new CompletableFuture<>();
f.completeExceptionally(t); return f;
    }
}

// Usage demo
class DemoBulkheadRateLimit {
    public static void main(String[] args) {
        BulkheadGateway gw = new BulkheadGateway();
        gw.registerTenant("gold",  new TenantLimits(16, 32, 1000, 200.0, 400));
        gw.registerTenant("bronze",new TenantLimits(4,  8,  100,  30.0,  60));

        // Submit tasks; bronze will cap at ~30 req/s while gold enjoys higher limits…
    }
}
```

Reasoning to interviewer

- Per-tenant executors + bounded queues isolate contention: a flood in Tenant A fills A's queue only; Tenant B's pool remains available.
- Token bucket enforces steady-state QPS with bursts; failure mode is explicit (429/503), not cascading latency.
- Different SKUs get different limits—just provision different TenantLimits.
- Metrics at the bulkhead boundary let you prove fairness and spot noisy neighbors quickly.

Metrics & alerts

- Per tenant: queue depth, active threads, task latency, rejection counts (rate-limit vs queue-full), throughput.
- Global: total concurrency, CPU, GC, DB connection usage.
- Alerts: sustained rejections for a tenant, queue depth > threshold, latency SLO breaches.

Future scope

- Replace custom token bucket with Resilience4j RateLimiter and Bulkhead (thread-pool or semaphore style) + Micrometer.
- Dynamic quotas from a config service; hot-reload limits without restart.
- Weighted fair-queuing across tenants; or a central scheduler that drains per-tenant queues fairly.
- Per-endpoint limits (write vs read), and adaptive throttling based on downstream health signals.

Scenario 73: Idempotency — duplicate payment callbacks

Payment gateways can deliver the same webhook twice (network retries, “at-least-once” semantics). I want the handler to be safe if the same callback hits us multiple times and to publish downstream events exactly once.

Answer

I’ll enforce idempotency with a key (e.g., gatewayEventId:merchantRef) and a unique row in an idempotency table. The callback handler runs in a DB transaction:

1. Try to create an idempotency row with status IN_PROGRESS. If the insert hits the unique constraint, we load the existing row and short-circuit (return the cached response).
2. If this is the first time, perform the business update (e.g., mark payment captured) in a way that’s itself idempotent (guarded update).
3. In the same transaction, write an outbox event (e.g., PaymentCaptured) and mark the idempotency row COMPLETE while storing the HTTP response we’ll send back to the gateway.
4. A separate outbox dispatcher reliably publishes events to Kafka and marks them SENT. If we crash after commit but before publish, the dispatcher will pick them up; if the callback is redelivered, steps 1–2 do nothing and no duplicate outbox row is created.

This gives “effectively-exactly-once” inside our system: each callback key updates our DB at most once and produces one downstream event even if the webhook is retried.

Code

```
// --- DDL (illustrative) ---
// create table idempotency (
//   idem_key varchar(128) primary key,
//   first_seen_at timestamptz not null default now(),
//   status varchar(16) not null,       -- IN_PROGRESS | COMPLETE
//   response_code int,                 -- cached HTTP status to return
//   response_body text
// );
//
```

```
// create table payments (
//   id bigserial primary key,
//   merchant_ref varchar(64) unique,
//   status varchar(16) not null,       -- PENDING | AUTHORIZED | CAPTURED |
REFUNDED...
//   amount bigint not null
// );
//
// create table outbox (
//   id bigserial primary key,
//   aggregate_type varchar(32) not null,   -- "PAYMENT"
//   aggregate_id bigint not null,
//   event_type varchar(64) not null,       -- "PaymentCaptured"
//   payload jsonb not null,
//   published boolean not null default false,
//   created_at timestamptz not null default now()
// );
// create index on outbox(published, id);

// --- DTO from gateway ---
record GatewayCallback(String eventId, String merchantRef, String status, long
amount) {}

// --- Service (Spring style) ---
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.transaction.annotation.Transactional;

class PaymentCallbackService {
    private final JdbcTemplate jdbc;

    PaymentCallbackService(JdbcTemplate jdbc) { this.jdbc = jdbc; }

    @Transactional
    public HttpResponse handle(GatewayCallback cb) {
        String idemKey = cb.eventId() + ":" + cb.merchantRef();

        // 1) Try to register idempotency (first writer wins)
        try {
            jdbc.update(
              "insert into idempotency(idem_key, status) values(?, 'IN_PROGRESS')",
              idemKey);
        } catch (DataIntegrityViolationException duplicate) {
            // Already processed (or in progress). Return cached response if available.
            return jdbc.query(
                "select response_code, response_body, status from idempotency where
idem_key = ?",
                rs -> {
                    if (!rs.next()) return HttpResponse.of(200, "OK"); // fallback
                    String st = rs.getString("status");
```

```
                int code = rs.getInt("response_code");
                String body = rs.getString("response_body");
                if ("COMPLETE".equals(st)) return HttpResponse.of(code, body);
                // IN_PROGRESS: another thread is processing. Return 202/OK
safely.
                return HttpResponse.of(202, "Accepted");
            }, idemKey);
        }

        // 2) Business update (guarded, idempotent)
        if ("CAPTURED".equalsIgnoreCase(cb.status())) {
            // Only advance if not already captured/refunded.
            int updated = jdbc.update(
              "update payments set status = 'CAPTURED' " +
              "where merchant_ref = ? and status in ('PENDING','AUTHORIZED')",
              cb.merchantRef());
            // If updated==0, we were already CAPTURED/REFUNDED; do not emit
duplicate outbox.
            if (updated > 0) {
                // 3) Write outbox in the SAME TX
                jdbc.update(
                  "insert into outbox(aggregate_type, aggregate_id, event_type, payload)
" +
                  "select 'PAYMENT', id, 'PaymentCaptured', " +
                  "       jsonb_build_object('merchantRef', merchant_ref, 'amount',
amount) " +
                  "from payments where merchant_ref = ?",
                  cb.merchantRef());
            }
        }
        // (Other statuses similar: AUTHORIZED, FAILED, REFUNDED with guarded
updates)

        HttpResponse response = HttpResponse.of(200, "OK");

        // 4) Cache the response, mark COMPLETE
        jdbc.update(
          "update idempotency set status='COMPLETE', response_code=?,
response_body=? where idem_key=?",
          response.status(), response.body(), idemKey);

        return response;
    }
}

// --- Outbox dispatcher (reliable publisher) ---
class OutboxDispatcher implements Runnable {
    private final JdbcTemplate jdbc;
    private final Publisher publisher; // wraps Kafka/Rabbit, idempotent on key
```

```
    OutboxDispatcher(JdbcTemplate jdbc, Publisher publisher) {
        this.jdbc = jdbc; this.publisher = publisher;
    }

    @Override public void run() {
        // Loop in a scheduled executor; each iteration uses its own transaction
        jdbc.execute((conn) -> {
            conn.setAutoCommit(false);
            var rows = conn.createStatement().executeQuery(
                "select id, aggregate_type, aggregate_id, event_type, payload " +
                "from outbox where published=false order by id for update skip locked
limit 200");

            while (rows.next()) {
                long id = rows.getLong("id");
                String type = rows.getString("event_type");
                String payload = rows.getString("payload");
                // Publish with an idempotent key (e.g., outbox id)
                publisher.publish(type, payload, String.valueOf(id));
                conn.createStatement().executeUpdate(
                    "update outbox set published=true where id=" + id);
            }
            conn.commit();
            return null;
        });
    }
}

// --- Tiny HTTP response holder for demo ---
record HttpResponse(int status, String body) {
    static HttpResponse of(int s, String b) { return new HttpResponse(s, b); }
}
```

Reasoning to interviewer

- The unique constraint on idempotency.idem_key is the gate: first arrival inserts; duplicates either see COMPLETE and reuse the cached response or get 202 Accepted if the first is still working. No locks are needed across instances.
- The payment update uses a guarded UPDATE … WHERE status in (…) so even if someone replays a “CAPTURED” callback, we won’t move the state twice or emit duplicate outbox rows.
- Transactional outbox ties state change and event creation together atomically; a background dispatcher handles delivery and marking published=true. If the app dies after commit but before publish, events are still in outbox. If the

webhook replays, we won’t insert a second outbox row because the guarded update didn’t change anything.

- Downstream consumers can also be idempotent by using the outbox id or a business key to dedupe—this completes the exactly-once story end-to-end.

Future scope

- Cache previous HTTP responses in Redis keyed by idem_key for faster duplicate handling; write-through from the DB.
- Add a TTL and a cleanup job for old idempotency rows; keep only the last N days per PCI/GDPR policy.
- Record a hash of the request body in idempotency and reject if the same key arrives with a different payload (protects against key misuse).
- If the gateway doesn’t supply a stable event ID, derive idem_key as a hash of relevant fields (gatewayTxnId + merchantRef + amount + status).
- Use FOR UPDATE SKIP LOCKED on the idempotency row if you choose a “claim then work” pattern; or keep the current optimistic “first insert wins” pattern for simplicity.

Scenario 74: Outbox & Saga — Order → Payment → Inventory across services

Think classic checkout: create order, collect money, reserve stock. We want atomic, reliable messaging between services and a clear way to complete or roll back the flow when a step fails.

Answer

I'd solve it in two layers:

1. Outbox for atomic publish (per service): each service (Order, Payment, Inventory) writes state changes and an outbox record in the same DB transaction. A background relay publishes outbox rows to Kafka with an idempotent key; if the app dies, the row is still there to be published later.
2. Saga for cross-service workflow:

- Orchestration: a central "OrderSaga" orchestrator coordinates steps and compensations.
- Choreography: services react to events and emit compensating events on failure.

For payments/inventory (where compensations are real actions, not just deletes), I lean orchestration: the Order service (or a dedicated Saga service) drives the sequence, applies timeouts/retries, and records saga state. If the organization prefers looser coupling, choreography also works—then each service must publish both "done" and "compensate-done" events and the Order service derives the final outcome.

Code (concise, Java/Spring style):

```
// ---------- 1) Outbox: atomic state + event ----------
/*
create table outbox (
  id bigserial primary key,
  aggregate_type varchar(32) not null,  -- "ORDER"
  aggregate_id varchar(64) not null,    -- orderId
  event_type varchar(64) not null,      -- "OrderCreated"
  payload jsonb not null,
  published boolean not null default false,
  created_at timestamptz not null default now()
);
```

```
create index on outbox(published, id);
*/

public record EventEnvelope(
    String id,          // UUID
    String type,        // e.g., "OrderCreated"
    String aggregateId,  // orderId
    String version,     // "v1"
    long occurredAt,    // epoch millis
    Map<String, Object> payload
) {}

@Service
class OutboxRelay { // runs on a scheduled executor
  private final JdbcTemplate jdbc; private final KafkaTemplate<String,
EventEnvelope> kafka;
  @Transactional
  public void flush() {
    jdbc.query("select id, event_type, aggregate_id, payload from outbox " +
            "where published=false order by id for update skip locked limit 200", rs ->
{
      while (rs.next()) {
        long id = rs.getLong("id");
        String type = rs.getString("event_type");
        String agg  = rs.getString("aggregate_id");
        String payload = rs.getString("payload");
        EventEnvelope ev = new EventEnvelope(
            String.valueOf(id), type, agg, "v1", System.currentTimeMillis(),
            Map.of("data", payload));
        kafka.send("order-events", agg, ev).get();       // idempotent key = agg or
outbox id
        jdbc.update("update outbox set published=true where id=?", id);
      }
    });
  }
}
// ---------- 2) Saga (Orchestration flavor) ----------
enum SagaStatus { STARTED, PAYMENT_RESERVED, STOCK_RESERVED,
COMPLETED, COMPENSATING, FAILED }

@Entity class OrderSaga {
  @Id String orderId;
  SagaStatus status;
  String lastError;
  long updatedAt;
}

@Service
class OrderSagaOrchestrator {
```

```
  private final SagaRepo repo; private final CommandBus bus; // bus -> Kafka
command topics

  @Transactional
  public void start(String orderId) {
    repo.save(new OrderSaga(orderId, SagaStatus.STARTED, null,
System.currentTimeMillis()));
    bus.send(new ReservePaymentCmd(orderId));
  }

  @Transactional
  public void onPaymentReserved(String orderId) {
    var s = repo.get(orderId);
    if (s.status() != SagaStatus.STARTED) return; // idempotent
    s.status = SagaStatus.PAYMENT_RESERVED; repo.save(s);
    bus.send(new ReserveStockCmd(orderId));
  }

  @Transactional
  public void onStockReserved(String orderId) {
    var s = repo.get(orderId);
    if (s.status() != SagaStatus.PAYMENT_RESERVED) return;
    s.status = SagaStatus.STOCK_RESERVED; repo.save(s);
    bus.send(new CapturePaymentCmd(orderId));
  }

  @Transactional
  public void onPaymentCaptured(String orderId) {
    var s = repo.get(orderId);
    if (s.status() != SagaStatus.STOCK_RESERVED) return;
    s.status = SagaStatus.COMPLETED; repo.save(s);
    // emit OrderConfirmed via outbox
  }

  // Fail paths (compensation)
  @Transactional
  public void onPaymentFailed(String orderId, String reason) {
    var s = repo.get(orderId);
    s.status = SagaStatus.FAILED; s.lastError = reason; repo.save(s);
    // emit OrderRejected via outbox
  }

  @Transactional
  public void onStockFailed(String orderId, String reason) {
    var s = repo.get(orderId);
    s.status = SagaStatus.COMPENSATING; s.lastError = reason; repo.save(s);
    bus.send(new RefundPaymentCmd(orderId)); // compensate prior success
  }

  @Transactional
```

```
  public void onRefunded(String orderId) {
    var s = repo.get(orderId);
    s.status = SagaStatus.FAILED; repo.save(s);
    // emit OrderRejected via outbox
  }
}
// ---------- Inventory/Payment services (command handlers, outbox per service) ----
------
@Component
class PaymentCommandHandler {
  @Transactional
  public void handle(ReservePaymentCmd cmd) {
    // 1) reserve funds (idempotent by orderId)
    // 2) insert outbox "PaymentReserved" or "PaymentFailed"
  }

  @Transactional
  public void handle(CapturePaymentCmd cmd) {
    // capture, then outbox "PaymentCaptured" (or failure)
  }

  @Transactional
  public void handle(RefundPaymentCmd cmd) {
    // refund, then outbox "PaymentRefunded"
  }
}

@Component
class InventoryCommandHandler {
  @Transactional
  public void handle(ReserveStockCmd cmd) {
    // check & reserve, then outbox "StockReserved" OR "StockFailed"
  }
}
```

Reasoning to interviewer

- Outbox removes the “dual write” race: state change and event publish are atomic at the service boundary; the relay handles delivery and is safe to retry.
- Orchestrated saga keeps the “happy path → error paths → compensations” in one place so timeouts, retries, and visibility are straightforward.
- Every handler and the orchestrator are idempotent (guarded updates, correlation by orderId), so duplicate messages are harmless.
- If you prefer choreography, the same compensations exist—just expressed as reactions: PaymentReserved → Inventory reserves; StockFailed → PaymentRefunded → OrderRejected.

Future scope

- Add timers to detect stuck sagas and trigger compensations automatically.
- Persist step-wise audit and expose a “saga timeline” for support.
- Use Kafka transactions (EOS) for read-process-write inside relay/handlers if you need strict once processing on the bus.
- Multi-warehouse: split “reserve stock” into a sub-saga with best-fit allocation and compensations per warehouse.

Scenario 75: Publisher–Subscriber — audit events to many downstreams

Multiple teams want the audit firehose: storage, SIEM, anomaly detection, BI, alerts. We need clean contracts, safe evolution, and strong isolation so one bad consumer doesn't affect others.

Answer

I'll define a stable event envelope + versioned payload schema, publish to Kafka topics with clear naming, and give each consumer its own group (and often its own topic) to isolate failures. For errors, I use retry topics with backoff and a DLQ. Schema evolution is managed in a registry with backward compatibility; producers never break old consumers.

Code (essentials):

```
// ---------- Envelope (stable) ----------
public record AuditEnvelope(
  String eventId,       // UUID
  String eventType,     // e.g., "USER_LOGIN", "ORDER_UPDATED"
  String version,       // payload schema version, e.g., "v1"
  long occurredAt,      // epoch millis (UTC)
  String tenantId,
  String source,        // service name
  String traceId,
  Map<String,Object> payload // versioned content
) {}
```

Topic design (Kafka):

- audit.events.v1 (primary)
- audit.events.retry.1m, audit.events.retry.5m, audit.events.retry.30m (delayed retry topics)
- audit.events.dlq (dead letter)

Partition key: tenantId (keeps tenant ordering); if you need per-user order, key by tenantId:userId.

Schema/versioning:

- Payload uses Avro/Protobuf/JSON Schema in a registry. Set compatibility to “backward” (new fields optional with defaults). Envelope fields rarely change; if they do, bump envelope version and topic (e.g., audit.events.v2).

Publisher (idempotent by eventId):

```
class AuditPublisher {
  private final KafkaTemplate<String, AuditEnvelope> kafka;
  public void publish(AuditEnvelope ev) {
    kafka.send("audit.events.v1", ev.tenantId(), ev); // key isolates per tenant
  }
}
```

Consumer pattern with retries + DLQ:

```
@Component
class AuditConsumerSIEM {
  private final KafkaTemplate<String, AuditEnvelope> kafka;

  @KafkaListener(topics = "audit.events.v1", groupId = "siem")
  public void onMessage(AuditEnvelope ev,
                        @Header(KafkaHeaders.RECEIVED_TOPIC) String topic,
                        @Header("x-retry") Integer attempt) {
    try {
      // Parse payload by ev.version(); route to SIEM
      processForSiem(ev);
    } catch (TransientException te) {
      int n = attempt == null ? 1 : attempt + 1;
      String retryTopic = n == 1 ? "audit.events.retry.1m"
                        : n == 2 ? "audit.events.retry.5m"
                        :          "audit.events.retry.30m";
      // add header to track attempts; a delay layer routes with delay or you can use
a scheduler
      kafka.send(retryTopic, ev.tenantId(), withAttemptHeader(ev, n));
    } catch (PoisonPillException pe) {
      // bad data -> DLQ with reason
      kafka.send("audit.events.dlq", ev.tenantId(), ev);
    }
  }
}
```

Failure isolation:

- Each downstream has its own consumer group and its own processing pool (bulkhead).

- Sensitive pipelines can subscribe to a *derived* topic (e.g., audit.events.parsed) produced by an ETL service, so malformed raw events never hit them.
- If one consumer falls behind, others are unaffected (independent groups). If one consumer needs strict SLAs, give it a dedicated topic via a transformer that fans out.

Dead-letter handling:

- DLQ carries the full envelope plus error metadata (error type, stack hash, attempt count).
- A DLQ viewer service lets teams inspect, reprocess, or purge with a reason.

Observability:

- Per-topic and per-consumer metrics: lag, throughput, p50/p95/p99 processing time, success/error/retry/DLQ counts.
- Alerts: sustained lag > threshold, DLQ rate spikes, schema-compatibility violations.

Reasoning to interviewer

- The versioned payload + stable envelope de-risks evolution; older consumers keep working.
- Retry topics with backoff avoid hot-looping on bad dependencies; DLQ captures poison messages without blocking the stream.
- Partitioning by tenant preserves in-tenant ordering and enables per-tenant throttling/analytics.
- Isolation by consumer group and sometimes by derivative topics prevents one workload from degrading others.

Future scope

- Add multi-tenant quotas and per-tenant DLQs if noisy customers frequently produce bad events.
- Encrypt sensitive fields in payload (FPE/KMS) and manage field-level access for downstreams.

- Introduce exactly-once semantics where needed (Kafka transactions end-to-end) or keep at-least-once with idempotent sinks.
- Governance: event catalog (owner, schema, PII flags), contract tests in CI to block breaking changes before deploy.

Scenario 76: Object Pool (non-GoF) — expensive but short-lived parsers

Creating parser instances is costly (grammar tables, I/O warmup), but each parse is brief. Should we pool?

Answer

Use pooling only when the object is truly expensive to construct and either (a) not trivially GC-friendly or (b) binds scarce resources (e.g., direct buffers, native handles). On modern JVMs, pooling cheap, heap-only objects is often harmful: the allocator is fast (TLABs), escape analysis removes allocations, and pools add contention and retention. Prefer immutable, thread-safe singletons first (e.g., DateTimeFormatter, many parser combinator objects). If the parser is not thread-safe but reusable after reset(), use per-thread or sharded pools to avoid a global bottleneck.

Code

```
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.ThreadLocalRandom;
import java.util.function.Supplier;

// Example "expensive" parser
final class ExpensiveParser {
    private boolean dirty;
    ExpensiveParser() { heavyInit(); }
    private void heavyInit() { try { Thread.sleep(10); } catch (InterruptedException
ignored) {} }
    void parse(byte[] bytes) { /* ... */ dirty = true; }
    void reset() { dirty = false; }
}

// Stripe-sharded pool to avoid a single hot lock
final class ParserPool {
    private final int stripes;
    private final ConcurrentLinkedQueue<ExpensiveParser>[] queues;
    private final Supplier<ExpensiveParser> factory;
    private final int maxPerStripe;

    @SuppressWarnings("unchecked")
    ParserPool(int stripes, int maxPerStripe, Supplier<ExpensiveParser> factory) {
        this.stripes = stripes;
        this.maxPerStripe = maxPerStripe;
        this.factory = factory;
        this.queues = new ConcurrentLinkedQueue[stripes];
        for (int i = 0; i < stripes; i++) queues[i] = new ConcurrentLinkedQueue<>();
    }

    private int stripe() {
```

```
        // xorshift-ish: cheap spread across stripes
        int h = System.identityHashCode(Thread.currentThread());
        h ^= (h << 13); h ^= (h >>> 17); h ^= (h << 5);
        return (h & 0x7fffffff) % stripes;
    }

    public ExpensiveParser borrow() {
        var q = queues[stripe()];
        ExpensiveParser p = q.poll();
        return (p != null) ? p : factory.get();
    }

    public void release(ExpensiveParser p) {
        p.reset(); // make it safe for reuse
        var q = queues[stripe()];
        if (q.size() < maxPerStripe) q.offer(p); // else drop on the floor; let GC reclaim
    }
}

// Usage with try-with-resources style wrapper
final class Pooled<T> implements AutoCloseable {
    private final T value;
    private final java.util.function.Consumer<T> releaser;
    Pooled(T v, java.util.function.Consumer<T> r) { this.value = v; this.releaser = r; }
    public T get() { return value; }
    public void close() { releaser.accept(value); }
}

class ParserPoolDemo {
    public static void main(String[] args) {
        ParserPool pool = new
ParserPool(Runtime.getRuntime().availableProcessors(),
                                   16, ExpensiveParser::new);

        // borrow/use/release without global contention
        try (Pooled<ExpensiveParser> p = new Pooled<>(pool.borrow(),
pool::release)) {
            p.get().parse(new byte[ThreadLocalRandom.current().nextInt(100, 200)]);
        }
    }
}
```

Reasoning to interviewer

- When pooling is safe: objects with costly construction (loading grammar tables, JNI init), objects pinning native/OS resources, or large direct ByteBuffers.

- When pooling is harmful: small/immutable Java objects, or those the JIT can scalar-replace. A pool keeps them alive longer (hurts GC), adds contention, and risks leaks/reset bugs.
- Avoid contention: use striped or ThreadLocal pools (bulkhead across cores) and drop extras instead of blocking; blocking pools reintroduce latency spikes.
- Safety: every release must sanitize (reset()), and objects must not escape while “leased”. Prefer AutoCloseable wrappers to ensure release even on exceptions.

Future scope

- Replace custom pools with tested libs (Netty’s Recycler, Agrona) when throughput matters.
- Telemetry: borrow latency, hit/miss rate, live vs. max per stripe; alert on “borrow timeout” if you choose to block.
- Consider a hybrid: immutable parser + pooled scratch buffers (direct) for large temporary memory.
- Re-benchmark regularly; if parser init becomes cheap (JIT warmed), remove the pool.

Scenario 77: Flyweight Pitfalls — keep extrinsic state out of equality

In a text editor, glyph flyweights share intrinsic data (code point, font, size, style). Position, color, selection flags are extrinsic and must not affect identity. Mixing extrinsic into equals/hashCode causes cache misses and subtle bugs.

Answer

Make flyweights immutable, with equals/hashCode based only on intrinsic fields, and store extrinsic state outside (caller passes it into draw(x, y, color, selection)). The factory's cache key must also be intrinsic only; never include coordinates or runtime styling. Validate with tests that two glyphs with different positions but same intrinsic attributes are equal and return the same pooled instance.

Code

```
import java.util.*;

// Intrinsic state only
final class Glyph {
    private final int codePoint;
    private final String font;
    private final int size;
    private final int style; // bitmask: bold/italic/...

    private Glyph(int cp, String font, int size, int style) {
        this.codePoint = cp; this.font = font; this.size = size; this.style = style;
    }

    // EXTRINSIC is provided here; never stored in the object
    public void draw(int x, int y, int argb, boolean selected) {
        // render outline at (x,y) with color; selection handled here, not part of identity
    }

    // equals/hashCode: intrinsic ONLY
    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Glyph g)) return false;
        return codePoint == g.codePoint && size == g.size && style == g.style &&
font.equals(g.font);
    }
    @Override public int hashCode() { return Objects.hash(codePoint, font, size,
style); }

    // Factory with intrinsic-only key
    static final class Factory {
        private final Map<Glyph, Glyph> cache = new HashMap<>();
        public Glyph get(int cp, String font, int size, int style) {
            Glyph probe = new Glyph(cp, font, size, style);
```

```
            return cache.computeIfAbsent(probe, k -> probe);
        }
        public int uniqueCount() { return cache.size(); }
    }
}

// Bad example: BUGGY equals mixes extrinsic state
final class BuggyGlyph {
    final int cp; final String font; final int size; final int style;
    // EXTRINSIC accidentally captured:
    int x, y, color; boolean selected;

    BuggyGlyph(int cp, String f, int sz, int st) { this.cp=cp; this.font=f; this.size=sz;
this.style=st; }

    // WRONG: includes x,y,color in identity -> same letter at different positions !=
equal
    @Override public boolean equals(Object o) {
        if (!(o instanceof BuggyGlyph b)) return false;
        return cp==b.cp && size==b.size && style==b.style
                && font.equals(b.font)
                && x==b.x && y==b.y && color==b.color && selected==b.selected; // <--
bug
    }
    @Override public int hashCode() { return
Objects.hash(cp,font,size,style,x,y,color,selected); }
}
```

Reasoning to interviewer

- If extrinsic fields sneak into equality, the factory cache explodes (one “A” per position/color), defeating Flyweight and spiking memory. Hash-based structures misbehave (you can’t find the same glyph later because its cursor color changed).
- Keeping flyweights immutable and extrinsic-free ensures canonicalization works: the same intrinsic key yields the same shared instance.
- Pass extrinsic state as parameters to draw() or keep it in lightweight “render commands”/Span objects alongside a glyph reference.

Future scope

- Add a strong unit test suite:

    - same intrinsic → same instance (==) from factory; different extrinsic draws do not create new keys; changing selection color doesn’t affect cache hits.
- Consider interning with WeakReference keys so rarely used glyphs can be GC’d; still intrinsic-only.
- For complex text shaping (ligatures/kerning), keep the shaped “glyph run” as a separate structure with extrinsic layout; glyphs remain pure.
- Batch draw commands by glyph to improve GPU/renderer efficiency without touching flyweight identity.

Scenario 78: Plugin architecture — drop-in pricing/risk rules via Java ServiceLoader

A payments platform wants third-party teams to add new pricing or risk rules without touching core code. Ops should be able to drop a JAR into a plugins folder and the app should discover, rank, and execute those rules safely (timeouts, isolation, no single bad plugin crashing the flow).

Answer

I'll expose a small SPI (service provider interface) like Rule with supports(ctx) and evaluate(ctx) → RuleResult. Implementations live in external JARs and register themselves using META-INF/services/.... At startup (or on a timed refresh), a RuleRegistry uses ServiceLoader to discover rules, captures metadata (priority, name), and executes them in priority order inside a guarded runner (per-rule timeout, exception fencing). This keeps the core stable while allowing new rules by just shipping a JAR.

Code

```
import java.time.Duration;
import java.util.*;
import java.util.concurrent.*;
import java.util.stream.Collectors;

// ---------- SPI (stable contract) ----------
public interface Rule {
    String name();
    int priority(); // smaller = earlier
    boolean supports(EvalContext ctx);
    RuleResult evaluate(EvalContext ctx) throws Exception;
}

public record EvalContext(String merchantId, long amount, String country,
Map<String,Object> attrs) {}
public record RuleResult(boolean passed, String reason) {}

// ---------- Runtime loader ----------
final class RuleRegistry {
    private final List<Rule> all;
    public RuleRegistry(ClassLoader cl) {
        ServiceLoader<Rule> loader = ServiceLoader.load(Rule.class, cl);
        this.all = loader.stream()
                .map(ServiceLoader.Provider::get)
```

```
                .sorted(Comparator.comparingInt(Rule::priority))
                .collect(Collectors.toUnmodifiableList());
    }
    public List<Rule> rulesFor(EvalContext ctx) {
        return all.stream().filter(r -> safeSupports(r, ctx)).collect(Collectors.toList());
    }
    private boolean safeSupports(Rule r, EvalContext ctx) {
        try { return r.supports(ctx); } catch (Throwable t) { return false; }
    }
}

// ---------- Guarded executor (timeouts, isolation) ----------
final class RuleEngine implements AutoCloseable {
    private final ExecutorService pool =
        new ThreadPoolExecutor(8, 8, 0, TimeUnit.SECONDS, new
ArrayBlockingQueue<>(256),
            r -> { Thread t = new Thread(r, "rule-exec"); t.setDaemon(true); return t; },
            new ThreadPoolExecutor.CallerRunsPolicy());

    private final Duration timeout;

    RuleEngine(Duration timeout) { this.timeout = timeout; }

    public List<RuleOutcome> runAll(List<Rule> rules, EvalContext ctx) {
        List<RuleOutcome> out = new ArrayList<>(rules.size());
        for (Rule r : rules) {
            RuleOutcome res = runOne(r, ctx);
            out.add(res);
            if (!res.passed()) break; // stop on first failure (configurable)
        }
        return out;
    }

    private RuleOutcome runOne(Rule rule, EvalContext ctx) {
        Future<RuleResult> f = pool.submit(() -> rule.evaluate(ctx));
        try {
            RuleResult rr = f.get(timeout.toMillis(), TimeUnit.MILLISECONDS);
            return new RuleOutcome(rule.name(), rr.passed(), rr.reason(), null);
        } catch (TimeoutException te) {
            f.cancel(true);
            return new RuleOutcome(rule.name(), false, "timeout", te);
        } catch (ExecutionException | InterruptedException ex) {
            Thread.currentThread().interrupt();
            return new RuleOutcome(rule.name(), false, "exception", ex);
        }
    }

    @Override public void close() { pool.shutdownNow(); }
}
```

```
record RuleOutcome(String rule, boolean passed, String reason, Throwable error)
{}

// ---------- Example built-in rule (plugins ship similar classes in their JARs) ----------
public final class AmountThresholdRule implements Rule {
    @Override public String name() { return "amount-threshold"; }
    @Override public int priority() { return 10; }
    @Override public boolean supports(EvalContext ctx) { return true; }
    @Override public RuleResult evaluate(EvalContext ctx) {
        boolean ok = ctx.amount() <= 50_000;
        return new RuleResult(ok, ok ? "OK" : "Amount too high");
    }
}

// ---------- Demo ----------
class DemoPluginSPI {
    public static void main(String[] args) {
        // In production, you can build a URLClassLoader pointing to /opt/plugins/*.jar
        ClassLoader cl = Thread.currentThread().getContextClassLoader();

        RuleRegistry registry = new RuleRegistry(cl);
        EvalContext ctx = new EvalContext("M-42", 75_000, "IN",
Map.of("deviceTrusted", false));

        try (RuleEngine engine = new RuleEngine(Duration.ofMillis(200))) {
            var outcomes = engine.runAll(registry.rulesFor(ctx), ctx);
            outcomes.forEach(o -> System.out.println(o.rule() + " -> " + o.passed() + "
(" + o.reason() + ")"));
            boolean allPassed = outcomes.stream().allMatch(RuleOutcome::passed);
            System.out.println("Decision: " + (allPassed ? "APPROVE" :
"REVIEW/DENY"));
        }
    }
}
```

To package a plugin JAR, a provider would include:

```
# META-INF/services/Rule
com.example.plugins.DeviceFingerprintRule
```

and implement Rule in com.example.plugins.DeviceFingerprintRule.

Reasoning to interviewer

- ServiceLoader gives late binding: the core does not know concrete rules; it only knows the SPI.

- Priority + supports() lets us cheaply filter and order execution.
- The guarded runner prevents a bad rule from hanging the whole flow (per-rule timeout + exception fencing) and isolates execution on a small pool (bulkhead).
- Loading via a dedicated URLClassLoader pointed at a plugins directory lets ops add/remove rules without redeploying the core.

Future scope

- Hot reload: watch the plugins directory, swap a fresh URLClassLoader, rebuild the RuleRegistry atomically.
- Sandboxing: run high-risk plugins in a separate process (gRPC) or with a restrictive SecurityManager/sandbox (on newer JDKs, prefer process isolation).
- Observability: per-rule metrics (p95, error/timeout rate), circuit breakers for consistently failing rules.
- Versioned SPI: RuleV2 with richer results; keep Rule for backward compatibility and load both.
- Ordering policies: allow rule groups (all must pass / any pass), or a DSL that selects which visitors/rules apply per tenant.